CO-PEER: AUDITING MATHEMATICS WITH PROOFS OF ERROR

Anna Gilbert[1], **Stephen Xia**[1], **Kirill Acharya**[2], **Shayan Hundrieser**[1], **Benjamin Li**[3], **Jan Ondras**[4], **Vaishali Surianarayanan**[1], **Jason Wang**[5], **Joon-Hyeok Yim**[1], **Arman Cohan**[1], **Quanquan C. Liu**[1]

[1]Yale University [2]Stanford University [3]University of Oxford
[4]Massachusetts Institute of Technology [5]The Innovation Fellowship

ABSTRACT

An objection to a mathematical proof requires evidence of its own: a critic can misread a claim, overlook an assumption, or reject a valid step. Counterexamples provide checkable evidence by refuting a false assertion even inside a proof of a true theorem. We introduce Co-PEER, a workflow for constructing and checking *proofs of error*. Each *objection dossier* records the challenged source claim, its assumptions, and a concrete counterexample. It includes a Lean-checked proof that the encoded assumptions hold and the encoded conclusion fails, and a review of whether the formal statement represents the source claim. BugBank stores these dossiers and their revision histories. A human-approved core of 147 dossiers drawn from 137 distinct sources supports a benchmark of 100 proof texts known to contain an error. The strongest evaluated LLM critic selects a reference line (a line annotated as incorrect) as its first choice in 73.0% of texts, compared with 18.0% for the best of four simple text rules. We also test whether a checked counterexample helps models complete Lean repair tasks in which the correction to prove is given. Finally, with the help of AI tools, we develop selected counterexamples into explanations of entire families of failures. We characterize and count the frozen six-colorings of doubled odd cycles of length at least five. We prove that a two-threshold coloring of the integers is the unique optimum within its family and exceeds a conjectured bound. We also determine every input on which a printed integer square-root algorithm fails and prove the correction proposed in the source.

1 AUDITING PROOFS THROUGH COUNTEREXAMPLES

Mathematical arguments, whether written by humans or generated by AI systems (Hubert et al., 2025; Feng et al., 2026), can fail at a single intermediate step while the surrounding argument reads well. A proof may rely on an identity that fails at a boundary case, apply a lemma outside its hypotheses, or claim an invariant that breaks after several steps. Finding and diagnosing such errors is part of mathematical work, and machine-generated arguments add to what must be checked.

Delegating that review to an automated critic raises an immediate problem: a critic's objection can be wrong too. It may challenge a true statement, ignore a hypothesis inherited from an earlier paragraph, or flag a flawed step with an invalid example. Auditing a proof is also different from solving the problem afresh: a new proof of a true theorem does not locate the defect in an earlier argument for it. Useful criticism therefore needs evidence that a reader or tool can inspect directly. A counterexample provides that evidence: a concrete object that satisfies the assumptions of an assertion and violates its conclusion. In *Proofs and Refutations*, Lakatos (1976) analyzes this interplay of proofs, counterexamples, and revisions. Examining why a concrete example breaks an assertion can expose a hidden assumption, guide a repair, or reveal a pattern that extends beyond the original case.

Three questions guide this paper. **Why counterexamples?** A counterexample refutes a specific assertion with a concrete witness that readers can check, even when the theorem being proved is true. **When can we trust a counterexample?** Only after verifying that the witness satisfies the source's assumptions and contradicts its claimed conclusion, and determining whether the failure is local to one step or falsifies the main theorem. **What can we learn from a counterexample?** Analyzing why

a witness works can identify missing hypotheses, guide a repair, and lead to theorems over an entire family.

We introduce **Co-PEER** (Collaborative Proof Expert Evaluation Review), a workflow for constructing and checking *proofs of error*, and **BugBank**, the collection that stores them. Each *objection dossier* in BugBank records the challenged source passage and its assumptions, a concrete counterexample, and a formal *certificate*: a Lean-checked proof that the encoded assumptions hold and the encoded conclusion fails for the witness. Each dossier also records its revision history and a separate review of *source correspondence*, whether the formal statement represents the claim in the source. Lean checks the certificate under its encoded definitions, whereas the source review decides whether the objection applies to the original argument. For our localization benchmark, we select a *human-approved core* of 147 dossiers from 137 distinct sources using recorded human judgments on source interpretation, counterexample validity, and correspondence with the Lean statement (Section 4).

A single counterexample can also open a broader mathematical question. Consider the 18-vertex graph $C_9[K_2]$, formed by replacing each vertex of a 9-cycle with a pair of adjacent vertices and joining each vertex of a pair to both vertices of the neighboring pairs. This graph needs five colors, yet it also admits a proper six-coloring in which every vertex already sees all five other colors among its neighbors, so no single vertex can change color while keeping the coloring proper. Asked to prove or refute a conjecture of Belavadi et al. (2026), with no access to BugBank, a model produced this *frozen* six-coloring of $C_9[K_2]$. A graph is *recolorable* if any two proper colorings can be joined by single-vertex changes whenever the palette exceeds the chromatic number. The conjecture states that if a prime graph and all its induced subgraphs are recolorable, then every clique blowup of the graph is recolorable as well. The base cycle C_9 satisfies these hypotheses, so the frozen coloring of $C_9[K_2]$ refutes the conjecture.

Studying why this coloring freezes yielded a classification for doubled odd cycles $C_n[K_2]$ of length $n \geq 5$. Frozen six-colorings exist if and only if $3 \mid n$, and when $3 \mid n$ there are exactly $90 \cdot 2^n$ such colorings for labeled vertices and colors. The existence construction was known (Shaebani, 2009); our analysis proves necessity and the exact count (Section 5).

Contributions and principal findings. We introduce **Co-PEER** and **BugBank** to make mathematical objections inspectable, checkable, and reusable, and we demonstrate three uses of this evidence. First, from the human-approved core we construct a benchmark of 100 proof texts, each known to contain an error (*certified-claim localization*). Given a numbered proof text, a critic selects up to three candidate lines; a *reference line* is one of the input's accepted error lines. The strongest evaluated LLM critic selects a reference line as its first choice in 73.0% of texts, compared with 18.0% for the best of four simple text rules. A separate LLM-based review of saved critic responses shows why this score alone is insufficient: some responses that select a reference line do not supply a valid objection, while some that miss it do.

Second, we study how counterexamples affect error search, proof repair, and proof revision. Across 100 proofs flagged by the workflow as flawed, counterexample-first search uses 15–31% fewer output tokens than sequential review that must also give a counterexample. The rates of objections confirmed by a reference-assisted model grader show no clear difference between the instructions; confidence intervals for the differences include zero. On 16 Lean repair tasks, models receive the identified defect and the correction to prove. Also supplying the checked counterexample raises first-attempt completions from 13 to 15 for Sonnet 5 and from 1 to 2 for GPT-OSS 120B, while Opus 5.5 completes 15 of 16 in both conditions. In a separate study on 50 paired proof-revision tasks per model using an earlier BugBank collection, retrieving stored objections did not consistently increase the number of proofs that a model grader judged correct. The broader value of retrieval during proof development therefore remains open (Section 4).

Third, with the help of AI tools, we develop selected counterexamples into explanations of entire families of failures. We determine every input on which a printed integer square-root algorithm fails and prove the correction proposed in the source. Building on the known existence construction, we prove necessity and the exact count of frozen six-colorings for doubled odd cycles of length $n \geq 5$. We also prove that one two-threshold coloring of the integers is the unique optimum within its family, a construction and lower bound that independent work also obtained. These follow-ups do not test whether retrieving BugBank records helps mathematical discovery (Section 5).

Section 2 defines proofs of error and their source-correspondence requirements; Section 3 describes Co-PEER and BugBank. Section 4 reports the human reviews, the localization benchmark, and the search, repair, and retrieval studies; Section 5 develops the mathematical follow-ups. Section 6 discusses related work and limitations, and the appendices supply supporting evidence, protocols, and proofs.

2 CLAIMS, COUNTEREXAMPLES, AND PROOFS OF ERROR

When a critic challenges a step in a proof with a counterexample, two distinct questions arise: is the counterexample mathematically valid as formalized, and does that formal statement represent the claim in the source? Together, a formal counterexample checked by Lean and a justified answer to the second question form a *proof of error*.

2.1 CLAIMS AND THEIR ASSUMPTIONS

Let s denote a selected assertion within a surrounding argument, X_s its domain of objects, $A_s(x)$ the assumptions in force at that point, and $C_s(x)$ the claimed conclusion. The assertion has the form

$$Q_s : \quad \forall x \in X_s, \quad A_s(x) \Rightarrow C_s(x). \tag{1}$$

Here $A_s(x)$ collects both the hypotheses inherited from the surrounding argument and any temporary assumptions of the current case. To check a single inference conditionally, one can include earlier assertions of the proof inside $A_s(x)$ and place the new step's conclusion in $C_s(x)$, without proving that those earlier assertions hold. Refuting an assertion under the proof's justified assumptions therefore requires separating those assumptions from intermediate claims that still await proof.

For example, consider an attempted proof that every real-valued function on a domain closed under negation is the sum of an even function and an odd function (one of the 100 texts in our localization benchmark; see Figure 1). The theorem is true, yet the argument asserts the intermediate identity $f(x) = f(-x) + (-f(x))$. On $\mathbb{R}$, the constant function $f \equiv 1$ satisfies the domain assumption and reduces this identity to $1 = 0$. That single function refutes the step, while the theorem itself holds via the decomposition $f(x) = [f(x) + f(-x)]/2 + [f(x) - f(-x)]/2$, whose two summands are even and odd, as exchanging x and $-x$ verifies.

2.2 COUNTEREXAMPLES AND THE SCOPE OF A REFUTATION

A *counterexample* to Q_s is an object $u \in X_s$ satisfying $A_s(u) \wedge \neg C_s(u)$. Both parts are required: an object that violates $A_s(u)$ says nothing about the assertion. In the decomposition example above, A_s requires only a real-valued function on a domain closed under negation. Evenness and oddness are properties required of the desired summands, not hypotheses on f, so $f \equiv 1$ is a valid counterexample to the intermediate identity.

By contrast, a proposed witness can exhibit the claimed failure while violating one of the source's assumptions. In a Shellsort argument examined in Section C.3, the source assumes that a list's first entry is smaller than its last. Under that assumption, it claims that if the first gap-m insertion swaps the first entry with the entry m positions later, that later entry remains first in the final m-sorted list. For $m = 2$, both $(3, 4, 2, 5, 1)$ and $(3, 5, 2, 6, 1, 7, 4)$ first swap 3 and 2 and later move 1 to the front (giving $(1, 5, 2, 6, 3, 7, 4)$ for the second list). Both contradict the predicted first entry 2, but the five-element list violates the endpoint assumption ($3 > 1$), whereas the seven-element replacement satisfies it ($3 < 4$) and is a valid counterexample.

The *scope* of an objection states what the counterexample refutes. Following Lakatos (1976), a *local refutation* falsifies an intermediate assertion, whereas a *global refutation* falsifies the main theorem or conjecture; a single example can do either or both. In the decomposition proof, $f \equiv 1$ is a local refutation of the intermediate identity, not a global refutation of the theorem. Counterexamples expose false assertions; a proof step that is true as stated but lacks justification calls for a different kind of objection.

2.3 FORMAL CERTIFICATES AND SOURCE CORRESPONDENCE

Each dossier expresses its counterexample and the challenged assertion in Lean, using hats to distinguish the formal encoding from the source mathematics. Here $\widehat{X}$ is the Lean type representing the domain X_s, and $\widehat{A}$ and $\widehat{C}$ are the formal predicates encoding the assumptions A_s and conclusion C_s. For a counterexample w expressed in Lean, a *formal certificate* is a proof π checked by Lean's kernel that

$$\widehat{A}(w) \ \wedge \ \neg\widehat{C}(w). \tag{2}$$

Checking π proves that w satisfies $\widehat{A}$ and violates $\widehat{C}$ under the definitions and proof dependencies used, but it does not check whether $\widehat{A}$ and $\widehat{C}$ faithfully represent the original assertion Q_s.

We call that relationship *source correspondence*: whether the formal statement represents the claim in the source. Let $e : \widehat{X} \to X_s$ interpret Lean objects as source objects, so that $e(w)$ is the source-level object represented by w (for instance, a Lean list representing an informal sequence of integers; e need not be implemented in Lean). To conclude that $e(w)$ refutes the source assertion Q_s, two implications must hold at the witness w:

$$\widehat{A}(w) \Rightarrow A_s(e(w)), \qquad C_s(e(w)) \Rightarrow \widehat{C}(w). \tag{3}$$

The first requires that satisfying the encoded assumptions suffices for the source assumptions at $e(w)$. The second requires that if the source conclusion held for $e(w)$, the encoded conclusion $\widehat{C}(w)$ would also hold; because the certificate proves $\neg\widehat{C}(w)$, it follows that the source conclusion is false. Together, the certificate (2) and the two implications (3) give $A_s(e(w)) \wedge \neg C_s(e(w))$, a counterexample to the original assertion (Proposition 4). Only these two one-way implications at the witness w are needed, not equivalence of the formal and informal predicates across the entire domain.

In Co-PEER, separate model reviewers assess whether the encoding and its interpretation justify Equation (3). In the Shellsort example, the first list $(3, 4, 2, 5, 1)$ has a proof checked by Lean, yet the first implication $\widehat{A}(w) \Rightarrow A_s(e(w))$ fails when the formal premise $\widehat{A}$ omits the source's endpoint assumption. A positive review of source correspondence is an interpretive judgment, separate from Lean's kernel check, and is not itself a formal proof. When a review finds a missing assumption or a mismatched interpretation, the encoding and counterexample are revised to match what the source states. Revising the encoding to match the source is distinct from modifying the source claim itself to repair the mathematical error, which requires its own proof.

2.4 WHAT A PROOF OF ERROR ESTABLISHES

A *proof of error* combines a formal certificate (2) with a justified account of source correspondence (3) and an explicit statement of scope (whether the refutation is local, global, or both). It establishes that the named assertion in the source fails under its stated assumptions. It does not establish that a proposed correction works; a repaired claim requires its own proof. Nor does failure to find a counterexample establish that a claim is true. An exhaustive search verifies the claim only over the finite domain checked, and extending that conclusion requires a further argument that those cases cover the full claim. Section 3 describes how Co-PEER constructs these objects and how BugBank preserves them.

3 CO-PEER AND BUGBANK: REUSABLE MATHEMATICAL OBJECTIONS

Co-PEER is our workflow for constructing and checking the proofs of error defined in Section 2, and BugBank is the collection that stores each resulting objection dossier and its revision history for inspection and reuse.

3.1 FROM A SELECTED ASSERTION TO AN OBJECTION DOSSIER

Given a problem statement and a numbered proof, Co-PEER constructs an objection dossier in six stages (prompts, search bounds, and procedures are given in Appendix K; see also McConnell et al., 2011; de Moura & Ullrich, 2021; The mathlib Community, 2020):

- **1. Selecting a candidate assertion.** Two calls to a fixed language model read the problem and the numbered proof and each select an assertion that may admit a short counterexample, quoting the source verbatim and stating its assumptions and conclusion. A deterministic validator checks the exact quotation and output format, and a second model reviews each candidate.
- **2. Searching for a counterexample.** A search agent writes and runs exact-arithmetic programs to test the assertion. It records one of four outcomes: a concrete witness, no witness found after exhaustive search of a specified finite set, an inconclusive result, or a claim outside the permitted search setting.
- **3. Checking the witness independently.** A separately written checker program reconstructs the condition from the quoted claim and tests the witness independently of the search script.
- **4. Formalizing the certificate in Lean.** Human contributors and coding agents construct the formal certificate of Equation (2); Lean checks the proof, and an axiom audit records the assumptions and dependencies used.
- **5. Reviewing the mathematics and source correspondence.** Three fresh model reviews audit the dossier. One searches for a counterexample from the problem and quoted claim alone; one tests the recorded witness with its own program; and one compares the Lean certificate with the quoted claim and witness to assess Equation (3). A further model examines flagged records and recommends keeping, repairing, excluding, or referring them for human review.
- **6. Recording the dossier.** The workflow packages the stage inputs, outputs, certificates, and any recorded review decisions into a persistent objection dossier.

Each *objection dossier* preserves four groups of evidence for inspection:

Dossier component	What a reader can inspect
Source and claim	The saved source version, quoted passage, selected assertion, and the assumptions under which that assertion is claimed to hold.
Counterexample and proof	The concrete witness, formal definitions, and exact named Lean theorem proving Equation (2).
Checking record	File identifiers, dependencies, build logs, and the axioms and assumptions used by the formal proof.
Review and consequence	Assessments of source correspondence (3) and of what the objection shows about the surrounding argument (local or global), with the reviewer (human or model), reviewed version, verdict, and any unresolved concerns recorded for each assessment.

The dossier records what the objection shows about the surrounding argument separately from the checks of the counterexample and its correspondence to the selected source claim.

BugBank. BugBank collects these dossiers. A dossier in BugBank shows what the workflow produced and which reviews, if any, it received; human review is not a condition of membership in the broader collection. Different subsets were reviewed by humans for different studies. Section 4 describes those reviews and defines the 147-dossier *human-approved core* used for the benchmark.

Why preserve the history? A proof checked by Lean does not by itself establish that an objection applies to the source. In the Shellsort example of Section 2, the initial witness $(3, 4, 2, 5, 1)$ had a proof checked by Lean, but source review found that it violates the source's endpoint assumption $(3 > 1)$. The replacement $(3, 5, 2, 6, 1, 7, 4)$ satisfies that assumption $(3 < 4)$, refutes the same step, and also has a Lean certificate. By keeping both versions and the reason for the replacement, the dossier lets a reader check which objection applies under the source's assumptions before reusing it. Section C.3 compares the two witnesses; Appendix K and Section C.7 document construction and replay.

4 Evaluating Critics and Supporting Proof Repair

4.1 Human review and the human-approved core

Human reviewers assessed selected subsets of BugBank by answering three questions. First, does the objection accurately represent the source claim and its assumptions? Second, does the counterexample

satisfy those assumptions and make the claim false? Third, does the Lean formalization correctly represent the counterexample?

Two human-review studies on earlier BugBank collections show why source correspondence requires its own check. In an initial review of 100 dossiers from the 401-dossier census, 76 reviews approved both the counterexample's mathematics and its Lean translation. Of those 76, however, 61 also approved the source interpretation, while seven rejected it and eight were uncertain. A later study audited a stratified random sample of 100 dossiers from the earlier 284-dossier collection using one human reviewer, with earlier verdicts masked in the review packets and prior project exposure recorded for 28 dossiers. In that audit, 91 dossiers received approval on all three questions, and weighting the strata gives an estimated 91.0% approval rate for that collection (95% sampling interval 81.0%–96.5%). The two cohorts have different sampling frames, and neither estimates the quality of the benchmark core below (Appendix C).

Two further checks concern the counterexamples and the references themselves. Across 604 executable search tasks, separately written checker programs confirmed 348 of 351 proposed counterexamples and rejected three (Section K.3). Conversely, errors in the references can also mislead a model grader. In the instruction study of Section 4.5, 13 reference records from an earlier panel later failed source or witness checks. Even so, the reference-assisted model grader had credited 60 of the 78 responses to those texts (three models across two instructions) as matching them (Section D.3).

The human-approved core. For our localization benchmark, we select from BugBank a *human-approved core* of 147 dossiers from 137 distinct sources. Several dossiers may concern different passages or answers from the same source, such as a paper version, a question-and-answer thread, a ProofWiki page, a Reddit post, or a QEDBench problem. To enter the core, a dossier must have a completed human review that answered yes to all three questions above and applies to the current dossier version. It must also have no unresolved negative or uncertain human judgment, and no unresolved concern about whether the objection applies to the selected source claim under its assumptions. For 113 dossiers, the current scientific content is identical to the reviewed version; for the other 34, we confirmed that the earlier positive review still applies. Assessing what the objection shows about the surrounding argument (local step failure or global falsity) was optional and is recorded separately, so core membership does not imply a positive consequence assessment. Every core dossier has a named Lean theorem, a successful build, and inspected proof assumptions. Across the core, 109 of the 147 objections target intermediate assertions, as do 74 of the 100 benchmark inputs below. Appendix C documents the review rounds, exclusions, and version continuity, and Table 2 maps each study to its population. BugBank includes objections to vetted community answers, and the core contains two model-generated proofs from QEDBench (Gonzalez et al., 2026) that had received perfect human rubric scores (Section A.1 and Appendix K).

4.2 Error localization: inputs, outputs, and scoring

In our localization benchmark, which we call *certified-claim localization* (CCL), the critic receives a numbered proof text known to contain an error. The prompt tells the critic that the text contains an error and asks for up to three ranked candidate line numbers with a brief explanation. Each benchmark input is backed by a held-out dossier from the human-approved core with a Lean certificate, which the critic does not see (Figure 1).

The *reference lines* of an input are its reviewed set of accepted error lines; 57 of the 100 primary inputs have more than one. A selected line earns credit if it is a reference line. **Hit@1** is the percentage of inputs whose first choice is a reference line, and **Hit@3** is the percentage with a reference line anywhere in the top three choices. A response *meets the rules* when it follows the response and execution rules: the required format, the token budget, and no disallowed tool use or extra turns. Missing answers, format errors, and budget exhaustion score zero, and an invalid or duplicate line label does not advance later choices (Appendix F). Human approval of a dossier is separate from human review of its reference-line labels. Of the 100 primary inputs, 69 have a recorded positive human check of the current line selection, including 55 checked on the exact current source text and line numbering (Appendix C).

Inputs and comparisons. The localization benchmark comprises 100 proof texts, each with at least ten nonblank lines, grouped into 92 clusters of related sources across community forums, reference

Numbered input (excerpt)
(10 nonblank numbered lines total)

[1] Q1: Suppose a function has the property that whenever "x" is in the domain of "f", then so is "-x". Show that f can be written as the sum of an even function and an odd function.

[3] By definition, an even function has the equation $f(x) = f(-x)$ and by definition, an odd function satisfies the equation $f(-x) = -f(x)$.

[5] Therefore, f can be written such that $f(x) = f(-x) + (-f(x))$.

...

Recorded GPT-OSS 20B selection

`"candidates": [5, 7]`
Score: Hit@1 = 1, Hit@3 = 1

Held-out reference evidence
(not shown to the critic)

Reference line: Line 5
Witness: $D = \{-1, 1\}$, $f \equiv 1$, $x = 1$. Then $f(1) = 1$ while $f(-1) + (-f(1)) = 1 - 1 = 0 \neq 1$.
Lean-checked statement:
The domain is closed under negation, 1 belongs to it, and $f(1) \neq f(-1) + (-f(1))$.

Figure 1: A primary localization input with a false intermediate deduction at line 5 (`reddit_learnmath_dm6itc`). The critic does not see the held-out reference evidence and selects line 5 first. The witness refutes the identity at line 5; the theorem itself is true.

pages, research-paper excerpts, and model-generated proofs from Reddit, Stack Exchange, ProofWiki, arXiv, and QEDBench. These source collections indicate provenance rather than author identity (Section C.7.1). We compare language-model critics against the best of four simple text rules that pick a line by character length, word count, mathematical-symbol count, or TF-IDF centroid similarity to the full text (Salton & Buckley, 1988). Confidence intervals resample the 92 clusters so that related texts stay together (Efron, 1979; Field & Welsh, 2007). Appendix F details the selection mask, statistical methods, budgets, and execution settings for all 13 evaluated configurations (Table 18).

Model configuration	First choice is a reference line Hit@1 (%) [95% CI]	Reference line in top three Hit@3 (%) [95% CI]	Recorded total tokens (lower bound)	Met rules (of 100)
Sonnet 5	61.0 [52.0, 69.6]	76.0 [67.7, 83.7]	697,281	96
Opus 5.5	68.0 [59.8, 76.2]	85.0 [78.6, 91.0]	510,187	96
DeepSeek R1 Qwen	22.0 [14.4, 30.1]	28.0 [20.0, 36.4]	428,199	87
Gemma 3 27B	26.0 [18.3, 33.7]	44.0 [35.7, 52.1]	340,082	100
GLM 4.5 Air	10.0 [4.9, 16.0]	11.0 [5.8, 17.2]	1,658,280	23
GPT-OSS 120B	49.0 [39.6, 58.3]	60.0 [51.0, 69.0]	450,420	97
GPT-OSS 20B	43.0 [34.3, 52.0]	47.0 [37.8, 56.6]	708,462	89
Kimi Linear	14.0 [7.2, 21.4]	27.0 [19.0, 35.7]	241,035	94
Nemotron 3.5 Lightning	27.0 [19.0, 35.6]	27.0 [18.8, 35.4]	1,676,079	41
Nemotron 3 Super	48.0 [40.2, 56.1]	48.0 [40.2, 56.0]	740,935	90
Gemini 3.1 Pro	73.0 [65.1, 80.6]	85.0 [78.2, 91.2]	1,446,420	93
GPT-5.6 Luna	63.0 [54.6, 71.1]	74.0 [66.7, 80.8]	390,142	97
TF-IDF centroid similarity	18.0 [11.0, 25.3]	—	—	—
Random-line expectation	6.7 [5.7, 7.8]	—	—	—

Table 1: Reference-line selection on the 100 primary inputs of the human-approved core. Both Hit@1 and Hit@3 count responses that do not meet the rules as misses. "Met rules" counts responses that follow the response and execution rules (required format, token budget, and no disallowed tool use or extra turns). The token column reports recorded primary input and output tokens (lower bound). The thirteenth configuration, Haiku, has two reporting views shown in Appendix F, which gives all 13 configurations, token components, and failure modes.

4.3 Results: localization, ranking, and response failures

Table 1 reports 12 model configurations on the 100 benchmark texts. Gemini 3.1 Pro achieves the highest Hit@1, selecting a reference line as its first choice on 73.0% of texts (73 of 100). This compares with 18.0% for the best of the four simple text rules (TF-IDF centroid similarity) and an expected 6.7% for random line selection. When three choices are allowed, Gemini 3.1 Pro and Opus 5.5 tie at 85.0% Hit@3. Expanding from one choice to three raises Gemini from 73 to 85 texts and Opus from 68 to 85, showing that identifying a flawed step and ranking it first are distinct

challenges. Rule compliance also shapes the scores: Nemotron 3.5 Lightning produced a response meeting the rules on only 41 of the 100 inputs and exhausted its token budget on the rest. Because all 100 inputs are scored, Hit@1 and Hit@3 measure whether a critic both answers within the rules and locates a reference line. Conditional accuracies restricted to responses meeting the rules have different denominators and are not matched comparisons. The locally run GPT-OSS 120B and 20B reach 49.0% and 43.0% Hit@1 (25 percentage points above the best simple text rule for the 20B model). Appendix F provides complete results, including both reporting views for Haiku.

4.4 Selecting a reference line and supplying a valid objection

Matching a reference line does not guarantee that a critic's explanation is mathematically sound, nor does missing the reference line mean the critic's objection is wrong. A separate LLM-review study of 48 saved responses on 24 texts from an earlier evaluation set (one response per text from Gemini 3.1 Pro and one from Nemotron 3 Super 120B) illustrates both cases. For Nemotron, reviewers accepted a valid objection in 15 of its 18 reference-line matches and in 4 of its 6 misses (for Gemini, 20 of 24 overall). To count as valid, a response had to meet the response and execution rules and refute the selected source claim with mathematically sound evidence under its assumptions, as determined by recorded LLM adjudications and source-correspondence checks (Table 19, Section G.2). Appendix G formalizes this gap between reference-line matching and objection validity and proves bounds on what line-matching scores can establish.

4.5 Search instructions, repair tasks, and retrieval of earlier objections

Three companion studies examine how counterexamples affect error search, formal proof repair, and proof revision; each study has its own input panel and evaluation protocol (Appendix D).

Search instructions. We compared search instructions on 100 proofs flagged by the workflow as containing errors with concrete witnesses (an earlier panel distinct from the localization benchmark). Counterexample-first search instructs the model to look first for a refutable claim and a concrete counterexample. The comparison condition reviews the proof sequentially and requires a counterexample before stopping. A model grader, given the reference counterexample, counts an objection as *confirmed* when it accepts the response's counterexample or self-contained refutation; a correct "invalid" verdict without such support does not count. Counterexample-first search used 31%, 15%, and 29% fewer output tokens than sequential review with a required counterexample for GPT-5.6 Sol, Gemini 3.1 Pro, and Opus 5. The corresponding differences in confirmed-objection rates were $+2$, -5, and $+5$ percentage points, with all 95% intervals including zero. Counterexample-first search used 6–11% more output tokens than ordinary early stopping, which allows an objection without a counterexample. On a separate panel of 60 false mathematical statements presented without proofs, counterexample-first search used 26–52% fewer output tokens than a proof attempt that must supply a counterexample when it rejects a statement (Table 6). Output-token counts include reported reasoning and do not measure end-to-end cost: ratios of recorded token totals depend on prompt caching and recorded tool executions (Section D.2 and Appendix E).

Repair tasks. On 16 Lean repair tasks in which the correction to prove is given, we test whether supplying a checked counterexample helps a model complete the formal repair. Both conditions receive the source context, formal definitions, identified defect, repair template, and target theorem statements; the treatment condition adds the dossier's checked counterexample, and a fixed checker accepts submissions whose proofs establish the required statements (Section D.4). On first attempts, adding the counterexample raised completed repairs from 13 to 15 for Sonnet 5 and from 1 to 2 for GPT-OSS 120B, while Opus 5.5 completed 15 of 16 in both conditions (Table 10). The study tests completing a supplied repair rather than discovering one from scratch. Because the task set is small and the effect varies by model, broader benefits remain to be tested.

Retrieval of earlier objections. Two studies tested giving a model access to earlier BugBank records while it worked on new problems. In a proof-revision study on 50 paired tasks per model under a shared stopping rule, a model grader judged 45 proofs correct with retrieval versus 47 without for Sol, 43 versus 42 for Gemini, and 44 versus 43 for Opus 5. Recorded token totals were higher with retrieval for Sol and Gemini and lower for Opus (Table 12, Section D.5). In an exploratory open-

problem study across 36 target problems, model audits accepted 12 resolutions with retrieval and 11 without, 11 of them shared by both conditions (Table 13, Section D.6). Retrieving stored objections thus produced mixed results; determining which prior objections transfer to a new argument, and when they justify the extra context, remains an open question.

5 From Finding a Failure to Understanding the Mathematics

Each case study below moves from a single counterexample to a theorem that explains an entire family of failures. The initial graph and integer-coloring counterexamples were found by language models asked to prove or refute the statements, without access to BugBank. We developed the family theorems and proofs with the help of AI tools. These follow-ups do not test whether retrieving stored dossiers helps mathematical discovery. Appendices B and I give the complete proofs.

5.1 A square-root algorithm: from one failure to a complete correction

This supplementary analysis concerns a printed integer square-root algorithm whose original dossier is outside the human-approved core because a human reviewer questioned its reading of the source (Appendix A). The results below concern the printed recurrence itself.

Claim and counterexample. The final algorithm in Answer to "Relating a proof to a Haskell program" is intended to compute the integer square root $\lfloor\sqrt{x}\rfloor$ of a nonnegative integer x: the greatest integer whose square does not exceed x. It starts with answer zero at zero. Given answer b at $x-1$, it returns b if $x \le (b+1)^2$ and $b+1$ otherwise. At $x=1$ the weak guard $1 \le (0+1)^2$ holds, so the algorithm wrongly returns zero; at $x=4$ it still returns one, reaching two only at $x=5$.

Structural insight. Because the guard tests $x \le (b+1)^2$ rather than strict inequality, for every positive input x the printed algorithm returns $\lfloor\sqrt{x-1}\rfloor$, lagging the true integer square root by one input. Replacing $\le$ with $<$, the correction proposed in the source, removes this delay. Induction proves both statements. Because $\lfloor\sqrt{x}\rfloor$ increases exactly at positive perfect squares, this one-input lag explains the failure pattern noted in earlier annotations (Appendices A and B).

Proposition 1 (Exact failure set and repair)**.** *The corrected algorithm returns the integer square root of every nonnegative integer. For a positive input x, the original algorithm returns the integer square root of $x-1$. Its errors therefore occur exactly at the positive perfect squares:* $1, 4, 9, 16, \ldots$.

Appendix H records all thirteen dossiers of the earlier 285-dossier collection whose annotations named both a counterexample family and a repair, each paired with Lean-checked family and repair statements.

5.2 Graph colorings: a repeating local pattern explains an entire family

Claim and counterexample. A proper coloring assigns different colors to adjacent vertices; the chromatic number is the fewest colors needed. A graph is *recolorable* if, for every palette size larger than its chromatic number, any two proper colorings can be connected by single-vertex color changes that keep the coloring proper. A proper coloring is *frozen* if every vertex already sees every other available color on a neighbor (Cereceda et al., 2008). For a nonempty graph, a frozen k-coloring with k greater than the chromatic number proves non-recolorability.

A clique blowup replaces each vertex of a base graph by a clique, called a *bag*, and joins every pair of vertices in adjacent bags. Doubling each vertex of an n-cycle gives $C_n[K_2]$. A graph is *prime* if it has no proper subset of at least two vertices whose members have identical neighbors outside that subset. Belavadi et al. (2026, Conjecture 1, §6) conjectured that if a prime graph and all of its induced subgraphs are recolorable, then every clique blowup of the graph is recolorable (the reviewed input was arXiv v2, Belavadi et al., 2024). Every odd cycle of length $n \ge 5$ satisfies those hypotheses, yet $C_9[K_2]$ has 18 vertices, chromatic number five, and a frozen six-coloring.

Structural insight. In a frozen six-coloring of $C_n[K_2]$, every vertex sees all five other colors among the three consecutive bags in its closed neighborhood. Consequently, every window of three consecutive bags uses all six colors once, which forces the color pairs on the bags to repeat with

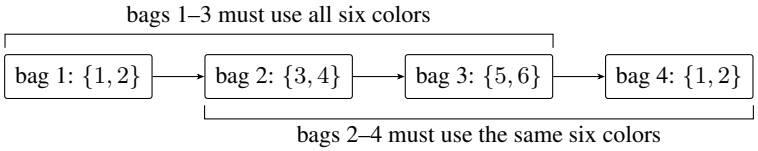


Figure 2: Why the color pairs must repeat every three bags. The two windows share bags 2 and 3, so bag 4 must use the same pair as bag 1. Arrows show the order of the bags, not directed graph edges. The numbers name six distinct colors.

period three around the cycle (Figure 2). Closing that period-three pattern around an n-cycle yields the classification. Existence when $3 \mid n$ follows from Shaebani (2009); our contribution is proving necessity and the exact count for labeled vertices and colors.

Proposition 2 (When the repeating pattern closes)**.** *Let $n \geq 5$ be odd. The doubled n-cycle has a frozen six-coloring exactly when n is a multiple of three. For each such n, there are exactly $90 \cdot 2^n$ such colorings when the vertices and six colors are labeled.*

There are $\binom{6}{2}\binom{4}{2} = 90$ ways to assign disjoint color pairs to the first three bags, and two ways to order the pair inside each of the n labeled bags, giving $90 \cdot 2^n$.

Larger bags yield counterexamples for every odd cycle of length at least nine. For instance, $C_{11}[K_3]$ has chromatic number seven and a frozen eight-coloring. For every odd $n \geq 9$ with $q = \lceil n/3 \rceil$, the blowup $C_n[K_q]$ has chromatic number $2q + 1 < n$ and a frozen n-coloring, refuting recolorability preservation for the base cycle C_n for every odd $n \geq 9$.

5.3 Integer colorings: a better construction with a provable limit

Claim and counterexample. Color the integers $\{1, \ldots, n\}$ ($n \geq 2$) with three colors. An ordered Schur triple $(x, y, x + y)$ with $x, y > 0$ and $x + y \leq n$ is *rainbow* if its three entries receive distinct colors. Parczyk & Spiegel (2026) conjectured that the maximum limiting fraction of rainbow Schur triples as $n \to \infty$ is $2/5$ (40%).

Consider the two-threshold rule that colors every even integer red and colors an odd integer k blue if $k/n \leq a$ or $k/n > b$, and green if $a < k/n \leq b$. Set $a = 4/11$ and $b = 10/11$. For each positive integer m, the coloring at $n = 22m$ produces $99m^2 + 3m$ rainbow triples out of $n(n-1)/2$ ordered Schur triples, a fraction converging to $9/22 \approx 40.9\% > 2/5$. Independent work obtained the same construction and lower bound (Hegde et al., 2026).

Structural insight. To understand why $a = 4/11$ and $b = 10/11$ arise, allow the thresholds to vary over $0 \leq a \leq b \leq 1$ while keeping all even integers red. Every rainbow triple uses one even entry and two odd entries of different colors. Counting the valid mixed-parity and odd–odd pairs via interval lengths and the sum bound $x + y \leq n$ yields a piecewise quadratic limiting rainbow fraction $D(a, b)$, whose maximum is the best this family can do.

Theorem 3 (The best thresholds for this construction)**.** *The unique maximum of $D(a, b)$ is $9/22$, approximately 40.9%, at $a = 4/11$ and $b = 10/11$. The simpler rules with $a = 0$ or $b = 1$, which omit one blue end block, reach at most $2/5$, or 40%.*

This optimum applies to the two-threshold family; a higher limiting fraction requires a rule outside it. Appendix I gives the exact $n = 22m$ counts and the proof of Theorem 3; Appendix J summarizes the further certified counterexample families.

6 Related Work, Limitations, and Conclusion

Proof auditing and error localization. Step-level proof benchmarks such as ProcessBench (Zheng et al., 2025), PRMBench (Song et al., 2025), and Hard2Verify (Pandit et al., 2026) evaluate step-level correctness or error localization. Closer to formal verification, ProofFlow tracks step dependencies during autoformalization (Cabral et al., 2026), Critique-Resilient Benchmarking evaluates adversarial

exchanges through checked objections (Marro et al., 2026), and FaithSieve decomposes informal proofs into local claims that combine Lean checks with semantic alignment assessments (Wang et al., 2026). Co-PEER complements these evaluations by storing the concrete counterexample, its Lean certificate, and the human or model review of source correspondence in a persistent dossier that backs each benchmark reference line.

Counterexample search and formal translation. CounterMath evaluates conceptual reasoning through counterexamples (Li et al., 2025), while Learning to Disprove (Li et al., 2026) and Lean Disprove (Ondras & Freer, 2026) evaluate automated counterexample search. FormalAlign and recent audits of formal math benchmarks show that a statement checked by Lean can still misrepresent the informal problem it translates (Lu et al., 2024; Ammanamanchi et al., 2026). Co-PEER brings those two threads together by separating Lean's check of the formal counterexample from the review of source correspondence. In the spirit of Lakatos (1976), it also uses checked counterexamples to repair proofs and characterize families of failures (see Appendix L for a comparison table of related resources).

Limitations and open questions. Each part of our evaluation has a defined scope. First, BugBank stores successful objections, and the localization benchmark evaluates texts known to contain an error. Measuring end-to-end auditing reliability requires correct-proof controls for false accusations, a defined starting population to estimate how often search, formalization, and source review succeed, and matched computation budgets on unseen sources to compare efficiency. Second, human reviewers and model graders can share blind spots, and annotated reference lines need not exhaust every valid objection in a proof. Third, lower output-token counts under counterexample-first search do not establish lower total computational or human cost once candidate selection, failed searches, Lean formalization, source interpretation, and human review are included. Fourth, the repair and retrieval studies use small task sets and show model-dependent outcomes, so broader trials are needed to determine when supplied witnesses or retrieved dossiers improve an argument.

Conclusion. A counterexample is most useful when a reader can verify both that the witness breaks the formalized claim and that the formalized claim matches the source argument. Co-PEER captures both checks, together with scope and revision history, in an objection dossier, and BugBank's human-approved core of 147 dossiers supports our 100-text localization benchmark. Separate studies test search instructions, repair tasks, and retrieval of earlier objections. Beyond auditing single steps, analyzing why a counterexample works can turn an isolated failure into a complete correction or a theorem for an infinite family.

AI USE STATEMENT

Language models contributed to source triage, counterexample search, executable checkers, Lean development, semantic reviews, qualitative coding, experimental grading, mathematical follow-ups, and manuscript revision. Their contributions included mathematical reasoning and interpretation of source claims. The records identify which assessments were made by human reviewers and which by language models. We developed the analytic propositions in Appendix G with AI tools and give their proofs there. We are responsible for all claims, citations, and released evidence.

ETHICS STATEMENT

Local objections are not blanket judgments about a source author's competence or the validity of an entire work. Source versions, prior corrections, known constructions, and limits of consequence are retained to avoid misleading discovery attribution. Reviewers' identities should remain deidentified in public evidence; source excerpts and redistributed artifacts require appropriate permissions and license compliance. This paper does not claim a representative estimate of source-platform error rates. Recorded human approvals apply to the artifact versions and review questions identified in each dossier; model-assisted judgments are distinguished from human assessments.

REPRODUCIBILITY STATEMENT

The instruction-study prompts, grading rubrics, and token accounting are in Appendices D and E. Corpus construction and executable-check procedures are in Appendix K. Core membership, human-review evidence, certificates, and benchmark score replay are documented in Appendix C and in the repository. Earlier census annotations and coding checks are preserved in `bugbank/annotations/`. Appendices B and I give complete written proofs of the mathematical propositions and theorems.

Data availability. Code, data, Lean proofs, review evidence, and experimental results, with replay instructions and file manifests, are available in the public repository at `https://github.com/qqliu/CoPEER`.

REFERENCES

Musa Al-hassy. Answer to "relating a proof to a Haskell program". Computer Science Stack Exchange, answer 97647, 2018. URL https://cs.stackexchange.com/a/97647. Posted 22 September 2018.

Bowen Alpern and Fred B. Schneider. Defining liveness. *Information Processing Letters*, 21(4): 181–185, October 1985. doi: 10.1016/0020-0190(85)90056-0. URL https://www.cs.cornell.edu/fbs/publications/DefLiveness.pdf.

Pawan Sasanka Ammanamanchi, Siddharth Bhat, and Stella Biderman. Faults in our formal benchmarking: Dataset defects and evaluation failures in Lean theorem proving. *arXiv preprint arXiv:2606.29493*, 2026. URL https://arxiv.org/abs/2606.29493.

Slim Barkallah, Luke Bailey, Kaiyue Wen, Mohammed Abouzaid, and Tengyu Ma. Pseudo-formalization for automatic proof verification. *arXiv preprint arXiv:2605.20531*, 2026.

E. T. Baskoro, C. Dalfó, M. A. Fiol, and R. Simanjuntak. On some metric properties of supertoken graphs. *Utilitas Mathematica*, 123:19–33, 2025. doi: 10.61091/um123-02. URL https://doi.org/10.61091/um123-02.

Manoj Belavadi, Kathie Cameron, and Ni Luh Dewi Sintiari. Recoloring via modular decomposition, 2024. URL https://arxiv.org/abs/2405.06446v2. Version 2, 14 November 2024; Figure 3 and Conjecture 1 in Section 6.

Manoj Belavadi, Kathie Cameron, and Ni Luh Dewi Sintiari. Recoloring via modular decomposition. *Journal of Graph Theory*, 111(4):113–123, 2026. doi: 10.1002/jgt.70005. URL https://onlinelibrary.wiley.com/doi/10.1002/jgt.70005.

Rafael Cabral, Tuan Manh Do, Xuejun Yu, Wai Ming Tai, Zijin Feng, and Xin Shen. ProofFlow: A dependency graph approach to faithful proof autoformalization. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/5fce5198dbf92d5ff45f74504431e2ff-Paper-Conference.pdf.

Luis Cereceda, Jan van den Heuvel, and Matthew Johnson. Connectedness of the graph of vertex-colourings. *Discrete Mathematics*, 308(5–6):913–919, 2008. doi: 10.1016/j.disc.2007.07.028. URL https://doi.org/10.1016/j.disc.2007.07.028.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *International Conference on Automated Deduction (CADE)*, pp. 625–635, 2021. doi: 10.1007/978-3-030-79876-5_37.

Steven T. Dougherty, Suat Karadeniz, and Bahattin Yildiz. The automorphism group of the doubly-even [72,36,16] code can only be of order 1, 3 or 5, 2013. URL https://arxiv.org/abs/1303.4920v1. Version 1, Theorem 3.3; withdrawn in version 2.

J. E. Dunbar, S. M. Hedetniemi, S. T. Hedetniemi, D. P. Jacobs, J. Knisely, R. C. Laskar, and D. F. Rall. Fall colorings of graphs. *Journal of Combinatorial Mathematics and Combinatorial Computing*, 33:257–273, 2000. URL https://combinatorialpress.com/article/jcmcc/Volume%20033/vol-033-paper%2018.pdf.

Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1): 1–26, 1979. doi: 10.1214/aos/1176344552.

Tony Feng, Junehyuk Jung, Sang hyun Kim, Carlo Pagano, Sergei Gukov, Chiang-Chiang Tsai, David Woodruff, Adel Javanmard, Aryan Mokhtari, Dawsen Hwang, Yuri Chervonyi, Jonathan N. Lee, Garrett Bingham, Trieu H. Trinh, Vahab Mirrokni, Quoc V. Le, and Thang Luong. Aletheia tackles FirstProof autonomously. *arXiv preprint arXiv:2602.21201*, 2026.

Christopher A. Field and Alan H. Welsh. Bootstrapping clustered data. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 69(3):369–390, 2007. doi: 10.1111/j.1467-9868.2007.00593.x.

Yuval Filmus. Answer to "why is this sequence of recurrence relevant?". Computer Science Stack Exchange, answer 13122, 2013. URL https://cs.stackexchange.com/a/13122. Posted 7 July 2013.

Mona Gatzweiler and Christian Krattenthaler. A positivity conjecture for a quotient of q-binomial coefficients, 2025. URL https://arxiv.org/pdf/2502.06032v3. Version 3, Conjecture 4.

Santiago Gonzalez, Alireza Amiri Bavandpour, Peter Ye, Edward Zhang, Ruslans Aleksejevs, Todor Antić, Polina Baron, Sujeet Bhalerao, Shubhrajit Bhattacharya, Zachary Burton, John Byrne, Hyungjun Choi, Nujhat Ahmed Disha, Koppány István Encz, Yuchen Fang, Robert Joseph George, Ebrahim Ghorbani, Alan Goldfarb, Jing Guo, Meghal Gupta, Stefano Huber, Annika Kanckos, Minjung Kang, Hyun Jong Kim, Dino Lorenzini, Levi Lorenzo, Tianyi Mao, Giovanni Marzenta, Ariane M. Masuda, Lukas Mauth, Ana Mickovic, Andrés Miniguano-Trujillo, Antoine Moulin, Wenqi Ni, Tomos Parry, Kevin Ren, Hossein Roodbarani, Mathieu Rundström, Manjil Saikia, Detchat Samart, Rebecca Steiner, Connor Stewart, Dhara Thakkar, Jeffrey Tse, Vasiliki Velona, Yunhai Xiang, Sibel Yalçın, Jun Yan, Ji Zeng, Arman Cohan, and Quanquan C. Liu. QEDBENCH: Quantifying the alignment gap in automated evaluation of university-level mathematical proofs. *arXiv preprint arXiv:2602.20629*, 2026. URL https://arxiv.org/abs/2602.20629v3. arXiv:2602.20629v3, 6 July 2026.

Richard H. Hammack and Gregory D. Smith. Cycle bases of reduced powers of graphs. *Ars Mathematica Contemporanea*, 12(1):183–203, 2017. doi: 10.26493/1855-3974.856.4d2. URL https://doi.org/10.26493/1855-3974.856.4d2.

Frank Harary and Robert A. Melter. On the metric dimension of a graph. *Ars Combinatoria*, 2: 191–195, 1976.

Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2. URL https://doi.org/10.1038/s41586-020-2649-2.

Swaroop Hegde, Hitesh Kumar, and Pratibha. A somewhat sure note on an un-Schur problem, 2026. URL https://arxiv.org/abs/2609.18474v1. Version 1, submitted 16 September 2026.

Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z. Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, Yannick Schroecker, Hussain Masoom, Ottavia Bertolli, Tom Zahavy, Amol Mandhane, Jessica Yung, Iuliya Beloshapka, Borja Ibarz, Vivek Veeriah, Lei Yu, Oliver Nash, Paul Lezeau, Salvatore Mercuri, Calle Sönne, Bhavik Mehta, Alex Davies, Daniel Zheng, Fabian Pedregosa, Yin Li, Ingrid von Glehn, Mark Rowland, Samuel Albanie, Ameya Velingker, Simon Schmitt, Edward Lockhart, Edward Hughes, Henryk Michalewski, Nicolas Sonnerat, Demis Hassabis, Pushmeet Kohli, and David Silver. Olympiad-level formal mathematical reasoning with reinforcement learning. Nature, 2025. https://www.nature.com/articles/s41586-025-09833-y.

hypnopump. Counterexample to Conjecture 10 of Meng, "On cube-free problems". GitHub gist, revision 664769b, July 2026. URL https://gist.github.com/hypnopump/8a369af584796b25c9af586b4fe39385.

Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

Omar Kettani. An algorithm for solving the independent set problem. *arXiv preprint arXiv:0709.2689*, 2007. URL https://arxiv.org/abs/0709.2689v1. Claim 1 in version 1; version 3 withdrawn owing to an error in Claim 1 (https://arxiv.org/abs/0709.2689v3).

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP 2023)*, pp. 611–626, 2023. doi: 10.1145/3600006.3613165.

Antoine Labelle. On a specialization of Toda eigenfunctions, 2026. URL https://arxiv.org/html/2502.10655v3. Version 3, Conjecture 7.3.

Imre Lakatos. *Proofs and Refutations: The Logic of Mathematical Discovery*. Cambridge University Press, 1976. doi: 10.1017/CBO9781139171472. Edited by John Worrall and Elie Zahar.

Juho Lauri and Christodoulos Mitillos. Complexity of fall coloring for restricted graph classes, 2019. URL https://arxiv.org/abs/1905.04695v1. Version 1; cycle fall-coloring criterion in Section 4.

Joel Brewster Lewis and Robert Won. Non-attacking rook placements on crossword grids, 2026. URL https://arxiv.org/abs/2609.03081v1. Version 1, Conjecture 3.10.

Yinghui Li, Jiayi Kuang, Haojing Huang, Zhikun Xu, Xinnian Liang, Yi Yu, Wenlian Lu, Yangning Li, Xiaoyu Tan, Chao Qu, Ying Shen, Hai-Tao Zheng, and Philip S. Yu. One example shown, many concepts known! counterexample-driven conceptual reasoning in mathematical LLMs. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, volume 267 of *Proceedings of Machine Learning Research*, pp. 35168–35185, 2025. URL https://proceedings.mlr.press/v267/li25ax.html.

Zenan Li, Zhaoyu Li, Kaiyu Yang, Xiaoxing Ma, and Zhendong Su. Learning to disprove: Formal counterexample generation with large language models. *arXiv preprint arXiv:2603.19514*, 2026.

Lily L. Liu and Yi Wang. On the log-convexity of combinatorial sequences. *Advances in Applied Mathematics*, 39(4):453–476, 2007. doi: 10.1016/j.aam.2006.11.002. URL https://doi.org/10.1016/j.aam.2006.11.002.

Jason Long and Adam Zsolt Wagner. The largest projective cube-free subsets of $\mathbb{Z}_{2^n}$. *European Journal of Combinatorics*, 81:156–171, 2019. doi: 10.1016/j.ejc.2019.05.005. arXiv:1810.01225.

Jianqiao Lu, Yingjia Wan, Yinya Huang, Jing Xiong, Zhengying Liu, and Zhijiang Guo. FormalAlign: Automated alignment evaluation for autoformalization. *arXiv preprint arXiv:2410.10135*, 2024. ICLR 2025.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023. NeurIPS 2023.

Charles F. Manski. *Partial Identification of Probability Distributions*. Springer Series in Statistics. Springer, New York, 2003. ISBN 978-0-387-00454-9. doi: 10.1007/b97478. URL https://link.springer.com/book/10.1007/b97478.

Samuele Marro, Jialin Yu, Emanuele La Malfa, Oishi Deb, Jiawei Li, Yibo Yang, Ebey Abraham, Sunando Sengupta, Eric Sommerlade, Michael Wooldridge, and Philip Torr. Benchmarking at the edge of comprehension. In *International Conference on Machine Learning (ICML)*, 2026. URL https://arxiv.org/abs/2602.14307.

Ross M McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. Certifying algorithms. *Computer Science Review*, 5(2):119–161, 2011. doi: 10.1016/j.cosrev.2010.09.009.

Yuchen Meng. On cube-free problems. *The Electronic Journal of Combinatorics*, 33(1):P1.16, 2026. doi: 10.37236/14052. URL https://doi.org/10.37236/14052.

George C Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pp. 106–119, 1997. doi: 10.1145/263699.263712.

Jan Ondras and Cameron Freer. Lean Disprove: Certified counterexample search for AI-assisted formal mathematics. In *Third AI for Math Workshop at ICML*, 2026. URL https://openreview.net/forum?id=5ck1jRE65S.

Shrey Pandit, Austin Xu, Xuan-Phi Nguyen, Yifei Ming, Caiming Xiong, and Shafiq Joty. Hard2Verify: A step-level verification benchmark for open-ended frontier math. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*, pp. 22502–22517, 2026. doi: 10.18653/v1/2026.acl-long.1031. arXiv:2510.13744.

Olaf Parczyk and Christoph Spiegel. An unsure note on an un-Schur problem. *The Electronic Journal of Combinatorics*, 33(1):P1.45, 2026. doi: 10.37236/13554. URL https://doi.org/10.37236/13554.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/1500000019. URL https://doi.org/10.1561/1500000019.

Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988. doi: 10.1016/0306-4573(88)90021-0. URL https://doi.org/10.1016/0306-4573(88)90021-0.

Saeed Shaebani. On fall colorings of graphs, 2009. URL https://arxiv.org/abs/0909.2769v1. Version 1; Theorem 2 and the example following Corollary 3.

Shiven Sinha, Shashwat Goel, Ponnurangam Kumaraguru, Jonas Geiping, Matthias Bethge, and Ameya Prabhu. Can language models falsify? evaluating algorithmic reasoning with counterexample creation. *arXiv preprint arXiv:2502.19414*, 2025. URL https://arxiv.org/abs/2502.19414.

Peter J. Slater. Leaves of trees. In *Proceedings of the Sixth Southeastern Conference on Combinatorics, Graph Theory, and Computing*, volume 14 of *Congressus Numerantium*, pp. 549–559, 1975.

Mingyang Song, Zhaochen Su, Xiaoye Qu, Jiawei Zhou, and Yu Cheng. PRMBench: A fine-grained and challenging benchmark for process-level reward models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025)*, pp. 25299–25346, 2025. doi: 10.18653/v1/2025.acl-long.1230. arXiv:2501.03124.

Zhi-Wei Sun. A family of polynomials and related congruences and series. *Frontiers in Combinatorics and Number Theory*, 2:1–22, 2026. URL https://arxiv.org/abs/2505.02767v4. arXiv:2505.02767v4 (10 February 2026; first posted 2025). Conjecture 1.1(i), second assertion.

The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pp. 367–381, 2020. doi: 10.1145/3372885.3373824.

Ziyu Wang, Qiming Dai, Yishan Wu, and Zaiwen Wen. FaithSieve: Fine-grained evaluation of math proofs with faithful formal evidence. *arXiv preprint arXiv:2608.26310*, 2026. URL https://arxiv.org/abs/2608.26310.

S. Ole Warnaar and Wadim Zudilin. q-rious unimodality, 2025. URL https://arxiv.org/html/2502.03993v2. Version 2, Conjecture 5.

Freek Wiedijk. Pollack-inconsistency. UITP 2010 / ENTCS 285, 2012. https://doi.org/10.1016/j.entcs.2012.06.008.

Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. ProcessBench: Identifying process errors in mathematical reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025)*, pp. 1009–1024, 2025. doi: 10.18653/v1/2025.acl-long.50. arXiv:2412.06559.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks*, 2023.

Appendix guide. Section 2 gives the essential definitions and source-correspondence requirements; the appendices group the supporting material by purpose, and each study keeps its own population, protocol, and numerical results.

- **Examples and review.** Appendix A works through source-linked counterexamples and what each establishes. Appendix B proves the square-root results, and Appendix C documents the human reviews, the exclusions and replacement witnesses, and the human-approved core.
- **Experiments.** Appendix D gives the protocols and complete results of the search, repair, and retrieval studies. Appendix E provides the instruction study's prompts, grading rubrics, and token accounting. Appendix F details the localization benchmark's protocol and complete results, and Appendix G analyzes what the scores measure.
- **Mathematical follow-ups.** Appendix H records the thirteen explanation packages. Appendix I states, documents, and proves the graph-coloring and rainbow-Schur results, and Appendix J covers the further certified families.
- **Construction and context.** Appendix K records the construction, audit, and executable-check procedures, and Appendix L compares related resources.

A WORKED EXAMPLES: SOURCE-LINKED COUNTEREXAMPLES AND WHAT THEY ESTABLISH

Each example below names the source, the counterexample, the Lean theorem that checks it, and what the objection establishes about the surrounding argument. Five of the examples belong to the 147-dossier human-approved core: the geometric-series formula, the spanning-tree proof, the coding-theory converse, and two QEDBench proofs with perfect prior rubric scores; the handshaking example in Section A.2 is a sixth. The graph-criterion example and the square-root follow-up are supplementary; the reasons are given with each. Appendix B proves the square-root results.

Finding a source and its certificate. A dossier identifier locates its objection at `bugbank/dossiers/<record-id>.json`, its source text at `bugbank/sources/<record-id>.json`, and its formal proof at `bugbank/certificates/<record-id>.lean` in the repository; the README and artifact guide give the layout. Core membership is recorded in `bugbank/core/membership.json`, the 100-input evaluation set in `benchmark/core/README.md`, and the exhibits below, with their recorded builds, in `bugbank/reviews/source_continuity/source_trust/auditing_exhibits.json`.

A three-vertex counterexample to a graph criterion. Dossier `arxiv_0709.2689v3__source_enriched_codex` concerns Claim 1 in version 1 of Kettani (2007). For an n-vertex graph with strictly upper adjacency matrix U, the claim equates the size k of a maximum independent set with the condition $U^{n-k+1} = 0$, where the algorithm tests this power in the supplied vertex order. The path with vertices numbered 1–2–3 has independence number two, but $U^2 \neq 0$. For any numbering of this path, U^2 is nonzero exactly when the degree-two vertex receives the middle number. Using vertices $0, 1, 2$, the theorem `Arxiv07092689v3SourceEnrichedCodex.refutes_claim1_forward_implication_on_ordered_path` proves that $\{0, 2\}$ is independent, that no independent set is larger, and that the specified power is nonzero, using Mathlib's independent sets and matrix powers. Its saved build and axiom audit passed with `propext`, `Classical.choice`, and `Quot.sound`. The identifier's version-three suffix refers to the withdrawal notice of 21 September 2007, which attributes the withdrawal to an error in Claim 1; the notice is preserved in `bugbank/reviews/source_continuity/source_trust/source_notice_0709_2689.json`. This dossier has a certificate but no saved human approval on all three review questions, so it is outside the human-approved core. Section A.2 shows the matrix and places this refutation next to a different use of the same graph.

A community formula with an explicit correction. Dossier `SE_cs_13118__a13122` concerns the CS Stack Exchange answer (Answer to "Why is this sequence of recurrence relevant?"), which sums powers of c from 1 to $n - 1$ but applies a geometric-series quotient with a different lower endpoint. At $c = 2$ and $n = 3$ the sum is 6 and the printed quotient is 7. The theorem `QEDCert.SE_cs_13118__a13122.real_sum_counterexample` checks these values and their

inequality over the reals; its saved build and axiom audit passed with the same three standard axioms. The finite geometric-sum identity gives the correction $(c^n - c)/(c - 1)$ for $n \geq 1$ and $c \neq 1$: the sum starts at 1, not at 0. The certificate proves the concrete discrepancy; the identity supplies the general correction.

A program error with a certified repair. The printed integer-square-root algorithm in Answer to "Relating a proof to a Haskell program" is stored as `SE_cs_97637__a97647`. Its theorem `SE_cs_97637_a97647.refutation` certifies the failure at the natural-number input $x = 1$, where the printed algorithm returns 0 instead of 1; this input also satisfies the source's integer specification. A human reviewer questioned the original dossier's reading of the source, so the dossier is outside the human-approved core (Appendix C). The results that follow concern the printed algorithm itself. The file `mathematics/extensions/sqrt_repair/SE_cs_97637_a97647_repair.lean` preserves the original `prog`, the correction `progStrict`, and the specification $R(x, y) \equiv y^2 \leq x < (y + 1)^2$. The correction proposed in the source changes the test $x \leq (b{+}1)^2$ to $x < (b{+}1)^2$. The theorem `strict_guard_correct` proves this corrected version for every natural-number input, and `error_and_repair` pairs that result with the original failure. The file `mathematics/extensions/sqrt_boundary/SqrtFailureBoundary.lean` proves $\mathrm{prog}(n + 1) = \mathrm{progStrict}(n)$ for every $n \in \mathbb{N}$, and its theorem `complete_diagnosis` combines the universal correctness of the correction with the exact failure set of Equation (4). Earlier annotations had noticed the pattern at perfect squares, and the follow-up proves that it accounts for every failure. Both files have recorded successful builds under Lean 4.30.0 and 4.34.0 using only `propext` and `Quot.sound`, with receipts beside the sources. Appendix B gives the written argument.

A tied cut explains a failed exchange. Dossier `Algorithms_19__sol3` from QED-Bench (Gonzalez et al., 2026) concerns a spanning-tree proof. A triangle with weights $w_{01} = w_{02} = 1$ and $w_{12} = 2$ has the unique minimum spanning tree $\{01, 02\}$, yet the cut $\{0\} \mid \{1, 2\}$ has two tied minimum edges. Deleting 01 and choosing 02 as its replacement leaves a disconnected graph, because 02 was already in the tree. The theorem `QEDReview19cut.refuted` checks the unique minimum spanning tree, the tied cut minima, and the failed exchange using Mathlib graph objects. The mistake is to assume that deleting a tree edge produces the two sides of an arbitrarily chosen cut.

A false converse in coding theory. Dossier `arxiv_1303.4920v2__source_enriched_codex` targets Theorem 3.3 in version 1 of Dougherty et al. (2013). For

$$C = \langle (1, 1) \rangle \leq \mathbb{Z}_4^2,$$

the standard Gray image is the binary self-dual code $\{0000, 0101, 1111, 1010\}$, but C is not quaternary self-dual: $(1, 1) \cdot (1, 1) = 2$ modulo four. The theorem `Arxiv13034920Mathlib.original_witness_refutes` proves this using submodules and orthogonal complements. Binary self-duality of the Gray image therefore does not imply quaternary self-duality of the original code. The numbered theorem asserts this converse for every $\mathbb{Z}_4$ code; the manuscript also considers a more restricted length-72 application. The version-two identifier refers to a later withdrawal notice citing a different lemma.

A.1 Objections in previously reviewed collections

Review by others does not settle individual assertions. The completed 425-record audit retained eight local objections from a 200-answer vetted CS collection and five from 43 accepted math.SE answers that carried no recorded flaw comments; these counts predate the selection of the human-approved core, and the identifiers, denominators, and decisions are recorded in `bugbank/reviews/source_continuity/source_trust/source_trust_reconciliation.json`. Two current-core records, `Algorithms_29__sol5` and `Algorithms_36__sol3`, concern QEDBench proofs that had received perfect human rubric scores (Gonzalez et al., 2026). The reconciliation's denominator of 19 counts retained QEDBench records with linked scores, not perfect-scored proofs, so we report these as two examples rather than a rate. Their original rubric scores and the later reviews of the counterexample dossiers are separate assessments. The geometric-series answer above is a third case of accepted mathematics with a local error.

A.2 One graph, two different conclusions

What a counterexample establishes depends on the assertion being checked. Figure 3 uses the path on three vertices twice. In one source it contradicts a matrix assertion, as in the first example above. In another it shows why a proposed proof step fails although the identity being proved, the handshaking identity, is true. This is the distinction in scope introduced in Section 2: the same object can refute a stated claim or expose a faulty step in a proof of a true statement, and the figure states exactly what each use refutes.

An independent set is a set of vertices with no edges between them, and the independence number $\alpha(G)$ is its largest possible size. For a graph with numbered vertices, the adjacency matrix has entry one where two vertices share an edge and zero elsewhere; its strictly upper part U keeps only the entries above the diagonal. The power U^2 is an ordinary matrix product, and the subscript 13 selects row one, column three.

A false matrix assertion

The path 1–2–3 has $n = 3$ vertices and a largest independent set $\{1, 3\}$ of size $k = 2$. Its strictly upper adjacency matrix is

$$U = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}.$$

The entry $(U^2)_{13} = 1$. This contradicts the printed assertion that an independent set of size k forces $U^{n-k+1} = 0$, for this vertex order. The source's withdrawal notice also identifies a fault in that claim.

A failed step in a proof of a true identity

A leaf is a vertex with one neighbor. Deleting the path's two leaves removes two edges, whereas reversing the proposed construction would require removing just one. Thus the construction misses this graph.

The degrees, or numbers of neighbors, are one, two, and one. With two edges, the handshaking identity holds:

$$1 + 2 + 1 = 4 = 2 \times 2.$$

The graph pinpoints an error in the proposed proof step.

Figure 3: A counterexample's conclusion depends on the assertion being checked. The path contradicts the matrix assertion on the left and the proposed construction on the right. Its degree sum satisfies the handshaking identity.

The proposed handshaking proof. Math Stack Exchange question 782216 proposes building every graph by repeatedly adding two degree-one vertices and one edge. That operation adds an isolated K_2 component, two vertices joined to each other and to nothing else, so reversing it requires an isolated K_2, which the connected three-vertex path does not contain. The original certificate checked a separate base-case error; the later follow-up proves the construction's exact boundary, including the path example (Appendix H). This dossier is in the human-approved core. The matrix assertion on the left is the supplementary dossier of the first example: a valid refutation whose dossier lacks a saved approval on all three review questions. The repository's `ARTIFACTS.md` links the source survey, quoted passages, formal statements, and review records for both.

B Why the square-root algorithm fails, and why the correction works

We prove Proposition 1: the printed algorithm fails exactly at positive perfect squares, and changing one comparison corrects it for every nonnegative integer input. Appendix A names the source and the formal certificates and explains why the original dossier is outside the human-approved core.

Let $f(x)$ and $g(x)$ denote the answers returned by the original and corrected algorithms. Both compute the answer for x from the answer for $x - 1$, either keeping it or increasing it by one. Their common starting value is zero:

$$f(0) = g(0) = 0, \qquad f(x) = \begin{cases} f(x-1), & x \le (f(x-1)+1)^2, \\ f(x-1)+1, & \text{otherwise}, \end{cases}$$

$$g(x) = \begin{cases} g(x-1), & x < (g(x-1)+1)^2, \\ g(x-1)+1, & \text{otherwise}, \end{cases} \qquad x \ge 1.$$

The comparison changes from $\leq$ to $<$ in the source algorithm (Answer to "Relating a proof to a Haskell program"). An answer y is correct when $R(x, y)$ holds, where $R(x, y)$ means $y^2 \leq x < (y+1)^2$. Equivalently, $y = \lfloor \sqrt{x} \rfloor$, the greatest integer whose square is at most x. Proposition 1 states that $g(x) = \lfloor \sqrt{x} \rfloor$ for every nonnegative integer x, and that $f(x) = g(x-1)$ for every $x \geq 1$. Thus the original algorithm is one input behind the corrected algorithm. Its exact failure set is

$$\neg R(x, f(x)) \quad \Longleftrightarrow \quad \exists k \in \mathbb{N},\ k \geq 1 :\ x = k^2. \tag{4}$$

All inputs in this statement are nonnegative integers, the domain of the recurrences above.

Proof of Proposition 1. **Step 1: the corrected algorithm returns the integer square root.** We prove $g(x) = \lfloor \sqrt{x} \rfloor$ by induction on x. At zero, the claim follows from $g(0) = 0$. For $x \geq 1$, let $b = g(x-1)$. The induction hypothesis gives

$$b^2 \leq x - 1 < (b+1)^2.$$

If $x < (b+1)^2$, keeping b gives $b^2 \leq x < (b+1)^2$, as required. Otherwise, the integer $(b+1)^2$ is greater than $x - 1$ and at most x, so it equals x. Increasing the answer to $b + 1$ then gives $(b+1)^2 \leq x < (b+2)^2$. This proves correctness at every input.

Step 2: the original algorithm returns the preceding correct answer. We prove $f(x) = g(x-1)$ by induction for $x \geq 1$. At $x = 1$, $f(1) = 0 = g(0)$. For $x \geq 2$, the induction hypothesis gives $f(x-1) = g(x-2) = b$. The updates for $f(x)$ and $g(x-1)$ make the same choice, because, for integers,

$$x \leq (b+1)^2 \quad \Longleftrightarrow \quad x - 1 < (b+1)^2.$$

Both therefore keep b or both return $b + 1$. Hence $f(x) = g(x-1) = \lfloor \sqrt{x-1} \rfloor$.

Step 3: the delay causes an error exactly at positive squares. For $x \geq 1$, let $b = f(x) = \lfloor \sqrt{x-1} \rfloor$. We have $b^2 \leq x$ and $x \leq (b+1)^2$. The required strict upper bound fails exactly when $x = (b+1)^2$. Conversely, at any positive square $x = k^2$, the original algorithm returns $k - 1$, so its upper-bound condition fails by equality. At zero, the answer is correct. This proves both the exact failure classification and the correction for every allowed input. □

The repository provides the formal proofs in `mathematics/extensions/sqrt_boundary/SqrtFailureBoundary.lean` and `mathematics/extensions/sqrt_repair/SE_cs_97637_a97647_repair.lean`, with toolchains and replay instructions in `bugbank/lean/README.md`.

C HUMAN REVIEW AND THE HUMAN-APPROVED CORE

C.1 WHAT WAS REVIEWED, AND WHY

We reviewed different subsets of BugBank for different purposes: to find failure modes of the construction, to improve the automated filter, to assess the validity of objections, to check the benchmark's reference lines, and to review selected mathematical follow-ups. None of these rounds is a condition of membership in BugBank. Separately, we selected a human-approved core of 147 dossiers, from which the primary and supplementary localization benchmarks were built. Table 2 lists the studies and their populations; Table 3 lists the review rounds.

Nine human reviewers contributed. Every dossier review answered the same three questions:

1. Does the objection accurately represent the original source claim and its assumptions?
2. Does the counterexample satisfy those assumptions and make the claim false?
3. Does the Lean formalization correctly represent the counterexample?

Two further checks were separate from these questions. Reference-line checks asked whether the lines marked in a benchmark text state the challenged claim. Consequence assessment, which asks what the objection shows about the surrounding argument, was optional; its recorded reviewer and verdict apply only to that question and that version, and an absent assessment is not an approval.

The rounds overlap by record identifier (Section C.6), so their counts must not be added to obtain a number of unique reviewed dossiers.

Collection or study	Question it supports
147-dossier human-approved core	What claim, counterexample, formal proof, and source review are available?
100 texts in the main evaluation set; 31 additional texts for supplementary evaluation	Can a model select a line marked incorrect in a text?
100 proofs; 60 statements	How do instructions change the objections produced and the tokens used?
16 Lean repair tasks with the correction given	Can a model complete and prove a specified correction, with or without a supplied counterexample?
50 paired tasks per model	How does retrieving earlier objections affect proof development?
13 explanation packages	What further mathematics follows from every case in an earlier selected family-and-repair group?

Table 2: Each study reports results for its own question and evaluation set. The benchmark measures reference-line selection; the mathematical case studies establish structural theorems and verified repairs.

Round	Frame and selection	Purpose	Outcome
Initial review (5–12 Sept. 2026)	110 packets from the 401-dossier census, eight reviewers; the assignment rule is not recorded in the accessible records	Failure modes; revised filtering prompt	100 returns: 61 all YES, 25 with a NO, 14 uncertain
Further random 30	Stratified random sample (seed 20260921) from the 221 of 285 retained records without an earlier review, by seven venues; one reviewer	Check of the filtered material	27 all YES
Targeted 33	32 selected mathematical artifacts and the documentary supplement D05; one reviewer	Selected records and research extensions	30 all YES
Fixed30	Two-stratum probability sample of the 285 retained records: 20 of the 191 primary (seed 20260922) and 10 of the 94 others (seed 20260923); one reviewer	Resource-wide assessment	29 all YES
Mathematical supplements	Three scoped packets (L01, H09, G07) extending earlier dossiers; one reviewer	Scope of the formal extensions	3 all YES, with consequence
Reference-line rounds	40 drawn uniformly (seed 20260921) from the 279 scorable records with nonempty reference spans; after corrections, a disjoint 40 from the 239 remaining eligible records (seed 3927191731)	Do the marked lines state the challenged claim?	35 of 40, then 38 of 40
Final stratified audit	Stratified random sample without replacement from the 284 retained dossiers: 67 of the 191 primary and 33 of the 93 others; one reviewer; earlier verdicts masked	Population estimate	91 of 100; weighted 91.0% (95% interval 81.0%–96.5%)

Table 3: Human-review rounds. The dossier reviews answered the three questions above for their own populations; the reference-line rounds asked the separate annotation question described in Section C.4. Counts are reviews of the listed artifacts, not unique dossiers; reviewer exposure, version evidence, and overlaps are given in the text.

C.2 Initial review: 100 returns from 110 assignments

Eight reviewers returned 100 completed reviews from 110 assignments between 5 and 12 September 2026. All 100 reviewed identifiers belong to the earlier 401-dossier census. One returned workbook covers 30 records in two batches and the other seven cover ten each; the ten assignment workbooks are blank templates, and only completed returns contribute judgments. The rule used to assign the 110 packets is not recorded in the review records available to us, so we do not describe this round as a random sample.

We recompute each outcome from the explicit answers: three YES answers count as approval, any NO as rejection, and CANNOT DETERMINE without a NO as inconclusive. The table preserves the answers as returned; the accompanying comments are kept in the review evidence.

Initial returned judgment	Records
All three answers YES	61
At least one NO	25
Inconclusive, without NO	14
Completed reviews	100
Assigned without a completed return	10

Of the 76 reviews that approved both the counterexample's mathematics and its Lean translation, 61 also found that the objection accurately represented the original claim and its assumptions; seven did not, and eight were uncertain. The 15 reviews that did not approve the source interpretation reported four omitted hypotheses or domain conditions, one missing definition, one dispute between an asymptotic and a literal reading, one initial-value discrepancy, one wrongly highlighted passage, six sources that were no longer available in full, and one uncertainty about notation or encoding. All 15 dossiers are excluded from the human-approved core.

What the comments contributed. The comments identified issues that neither a calculation nor a successful Lean build could settle. For `Algorithms_35__sol5`, exchanging the two sides of a bipartition did not create the distinct bipartition the claim required. For `Graph_Theory_20__sol4`, the objection omitted a disconnectedness assumption. For `Graph_Theory_12__sol5`, the selected Lean theorem refuted a conjunction where the source asserted an implication, and the reviewer pointed to another theorem in the same file that addressed the implication. These observations shaped the later filtering prompt and the review process described below.

Two rows in the batch-01 return have three YES answers but apparently exchanged comments: the comments for `mse_3703719__ans3707488` (Answers row 8) describe the singleton matroid-rank example in `proofwiki_132447`, and the ProofWiki row (Answers row 4) describes the MSE record's complementary-half-set example. We preserve both returns as written; excluding them leaves 98 reviews with unambiguous comment attribution, including 59 approvals, and both counts are available in the public review tables.

Review conditions and version records. The 110 packets included earlier automated assessments in a background appendix. Reviewers were asked to finish their own judgment before reading it, but the returns do not record when that material was read, so we report this round as unblinded. Each completed row supplies one reviewer's judgment and a completion date rather than time spent. The returns did not include hashes of the particular PDF or Lean file read, so applicability to the current version is documented separately (Section C.6). Forty-four members of the human-approved core are supported by this round. The saved ledger preserves workbook hashes, sheet and row references, comments, and the attribution issue.

Derived filter and public evidence. The reviewers' observations informed a revised guidance prompt, preserved in `bugbank/reviews/filter_prompt/`, for automated filtering across the corpus. The resulting automated audit examined 425 dossiers and retained 285; a subsequent withdrawal left 284. The comments also motivated the comparisons of original and replacement counterexamples in Section C.3. Anonymous row-level answers, attribution flags, and cohort information are in `bugbank/reviews/original100/answers.json` and `bugbank/reviews/co`

horts.json; from the repository root, python -B bugbank/reviews/reproduce.py -check recomputes the recorded counts, including the attribution-clean sensitivity, and the review README explains the exported evidence.

C.3 Source assumptions and checked replacements

Like the initial review above (Section C.2), the certificate comparisons in this subsection concern the earlier construction process. The paired sources, witnesses, certificates, and receipts are indexed in bugbank/reviews/source_continuity/PAIR_INDEX.json.

We compare all 16 records whose counterexamples were replaced during the earlier checks. Five belong to the current 147-dossier core, including the sorting and Gaussian-integer examples below. All 16 pairs retain the same stored problem and source context, and 15 also retain the same challenged quote, so the change in the counterexample is directly inspectable.

The original Lean files were recovered for all 16 records. Four have saved original build and axiom-check receipts; the other 12 were recompiled unchanged under Lean 4.30.0 and 4.34.0, and all 24 compilations succeeded. All 16 replacement certificates also have build receipts. The replays establish that the recovered files compile; the original receipts document which checks were performed at the time. Three pairs preserve the same quote and have both original and replacement build receipts: the endpoint-order example below, an equality interpreted as an eventual asymptotic claim, and a covariance claim tested with an ensemble different from the source's. The fourth pair with original receipts changes its challenged quote. The public pair index records these distinctions alongside the source excerpts, witnesses and audit reasons.

An inspection of all 123 theorem and lemma declarations in the 12 recompiled files found 78 using no axioms, 42 using only propext, and three using propext and Quot.sound; none uses sorryAx or native reduction. Three inspection drivers also queried comment text as declaration names, and an independent review checked that their outputs still covered every genuine declaration. The files, commands and complete outputs are in bugbank/reviews/source_continuity/earlier_versions/.

The replacements address different issues. Four original modules already contained an alternative that satisfies the source assumptions: the arXiv probability family, the ProofWiki induction instance, the six-team tournament and the limit calculation at $x = 3/2$; the first two already select that stronger formal statement, and the other two contain it alongside the disputed statement. Other replacements restore omitted assumptions or conventions, as in the two examples below. Their interpretation follows the recorded audits, and their membership in this comparison is determined by the recorded replacement rather than by random sampling.

An executable check of the omitted source condition. The gap-sorting source explicitly assumes $x_0 < x_n$. Its challenged local step says that after the first exchange with x_m, the final gap-m sorted array begins with that entry x_m. The original witness uses $m = 2$ and $A = (3, 4, 2, 5, 1)$. Its Lean-checked theorem correctly shows that the final first entry is 1 rather than $x_2 = 2$. But $3 < 1$ is false: this array violates the source's endpoint hypothesis. The replacement $A' = (3, 5, 2, 6, 1, 7, 4)$ satisfies $3 < 4$; its first gap-2 insertion exchanges 3 and 2, and the final array is $(1, 5, 2, 6, 3, 7, 4)$, again beginning with 1 rather than $x_2 = 2$. The selected replacement theorem proves the endpoint inequality, the exchange hypothesis and the failed conclusion together.

A supplementary Lean file retains the current certificate's exact bytes as its prefix and adds three axiom-free lemmas. For the explicit predicate $H(a) := a_0 < a_{|a|-1}$, they prove $\neg H(A)$, rule out the conjunction of $H(A)$ with the encoded objection conditions, and prove that A' satisfies those conditions. The file compiles under Lean 4.30.0 with exit code zero. This checks the role of the particular source condition; the correspondence between the quoted prose and H remains an inspectable interpretation, and the file does not certify the entire source proof or claim that its final theorem is false.

An equality case that satisfies the source assumptions. In mse_605342__ans609345, the Gaussian-integer source first chooses α from the lattice generated by $1 + i$ with $N(a - \alpha b) \leq N(b)$, then claims that coprimality makes this strict. The old certificate takes $a = b = -1$, $\alpha = i$, and correctly computes $2 > 1$, outside that weak-bound premise. The current Mathlib certificate keeps

$a = b = -1$ and uses $\alpha = 1 + i$. Its theorem explicitly establishes coprimality, opposite coordinate parities, the allowed multiplier and the weak bound, but proves

$$N(a - \alpha b) = N(b) = 1, \qquad \neg\big(N(a - \alpha b) < N(b)\big).$$

A companion theorem proves $b \mid a$ together with coprimality: divisibility by this unit is no contradiction. The repaired objection therefore identifies the equality boundary in the local strict-descent argument; it does not establish nontermination of the algorithm.

Together, these comparisons show the source check that Co-PEER applies: state the relevant assumptions, prove that the counterexample satisfies them, and establish the contradiction. The human judgments and the certificate comparisons are complementary evidence for these three steps.

C.4 Further random, targeted, and resource-wide reviews

Further random 30. After automated filtering with the revised prompt, one reviewer, who used AI tools, evaluated a stratified random sample of 30 dossiers from the 221 of 285 retained records with no earlier human review. With seed 20260921, the sampling script drew from sorted record identifiers within seven source venues and then shuffled the review order. The sample comprised four Math StackExchange records, five ProofWiki, four QEDBench, seven Reddit, three reference-proof additions, three Stack Exchange, and four arXiv records. The reviewer approved 27 dossiers on all three questions; the individual judgments were 28 YES and 2 NO for source connection (R012, R026), 30 YES for valid mathematics, and 29 YES and 1 NO for Lean statement match (R019). The record notes that the AI assistant saw the prior rationale on its first reading of R001, so we do not claim that earlier judgments were masked in this round. Because the sampling fractions differ across venues, the unweighted approval count describes the reviewed sample rather than a design-weighted estimate for all retained dossiers.

Targeted 33. One reviewer returned judgments for 33 explicitly selected artifacts: 32 mathematical artifacts and the documentary supplement D05. Thirty received YES on all three questions, 29 mathematical artifacts and D05; the three exceptions were E01 (source connection NO), E02 (mathematics NO), and S01 (mathematics not applicable). Fourteen of the artifacts correspond to current record identifiers; the others are research and scope reviews, so the artifact count is not a count of distinct dossiers.

Fixed30. A separate round evaluated a two-stratum probability sample of the then-current 285-record roster. It drew 20 records from the 191-record primary stratum with seed 20260922, retained unchanged, and 10 from the 94-record complement with seed 20260923, both without replacement from sorted record identifiers. One reviewer used AI tools and personally checked the judgments, reporting prior project exposure for six of the 30 records. All 20 primary records and nine of the ten complement records received YES on all three questions; for I025, the objection did not accurately represent the source claim. The prespecified population weights were 191/285 and 94/285, and 29 of 30 describes the reviewed sample. This frame differs from the later 284-record audit frame, whose complement contains 93 records.

Mathematical supplements. Three scoped supplements (L01, H09, G07) received YES on the three questions and on the optional consequence from one reviewer. L01 extends the density analysis of the layered projective-cube construction to all dimensions, with the cited Long–Wagner cube-freeness theorem as an explicit premise; H09 characterizes when the proposed handshaking induction step works while retaining the truth of the handshaking lemma; G07 proves uniqueness of the polynomial representing the fixed fake-Gaussian quotient and its negative degree-43 coefficient, without a minimality claim. They extend earlier mathematical dossiers rather than adding independent records, and the returned packet hashes match the assigned supplements.

Reference-line rounds. These checks asked whether the marked lines of a benchmark text state the challenged claim, whether any such line was missing, and whether an unrelated line was marked; the acceptance criterion was at least 38 of 40 complete line selections with answers YES, NO, NO. The first round sampled 40 records uniformly without replacement (seed 20260921) from the 279 records of the 285-record roster that were eligible for scoring and had nonempty reference

spans, and approved 35 of 40. After corrections, a fresh sample of 40 was drawn from the 239 remaining eligible records (seed 3927191731), disjoint from the first; it approved 38 of 40 and met the criterion. The two remaining concerns were an omitted line (G14) and an extraneous marked line (G12). These judgments concern the marked lines in the reviewed source versions and do not themselves validate the mathematical objections. Both sets of judgments are preserved in `bugbank/reviews/reference_lines/initial40.json` and `bugbank/reviews/reference_lines/fresh40.json`.

Records E01, E02, R012, R019, R026, and I025 are all excluded from the human-approved core.

C.5 Final random audit with masked prior judgments

Finally, we drew a stratified random sample without replacement from the 284 retained dossiers: 67 of the 191 primary records and 33 of the 93 other records. One reviewer used AI tools and personally checked the judgments on accurate source interpretation, witness validity, and correspondence with the Lean statement. The review packets displayed the source and certificate text and masked earlier verdicts and interpretive commentary; the reviewer reported prior project exposure for 28 records.

All three criteria received a YES for 63 of 67 primary and 28 of 33 other records, 91 of 100 in total. Weighting each stratum by its share of the 284-dossier collection gives an estimated approval rate of 91.0%, with a simultaneous 95% sampling interval of [81.0%, 96.5%], obtained by combining two exact 97.5% finite-population intervals by Bonferroni. The component approval counts were 95 of 100 for accurate source interpretation, 98 for witness validity, and 94 for Lean correspondence; the optional reference-line check received 98 YES and two not-applicable responses. The interval summarizes sampling uncertainty for this review procedure on the 284-record population and does not account for reviewer error. The sampling plan and interval method were specified before review; the reviewer-admissibility clarification and the weighted point estimate followed the returned judgments. The design, chronology, and analysis are preserved in `bugbank/reviews/random100/PROTOCOL.md`, `bugbank/reviews/random100/design.json`, and `bugbank/reviews/random100/saved_analysis.json`.

C.6 Overlaps, final dispositions, and the human-approved core

The rounds overlap by record identifier, not necessarily by identical reviewed versions. The further random 30 share no identifiers with the initial returns; Fixed30 shares seven with the initial returns and two with the further random 30; the final audit shares 21 with the initial returns, 12 with the further random 30, and seven with Fixed30. The 14 targeted artifacts with current record identifiers overlap the initial returns, the further random 30, Fixed30, and the final audit in 5, 2, 0, and 7 records.

All nine nonapprovals in the final audit are excluded from the core: four from its primary stratum (A001, A002, A023, A043) and five from its other stratum (A072, A075, A085, A093, A099). Table 4 preserves their three answers and recorded follow-up categories; a previously recorded concern retains its original NO judgment.

The version ledger at `bugbank/reviews/version_resolution/README.md` links every case to its full artifact hashes, source text, selected Lean declaration, original review and final disposition. For all nine, the certificate bytes and source-line arrays match the reviewed versions; exclusion does not overturn the negative judgments. Seven scored cases received YES on the separate line check, which did not resolve their source, witness or Lean concerns.

To enter the human-approved core, a dossier must have a completed review that answered YES to all three questions and applies to the current version. It must also have no retained negative or uncertain human judgment, and no unresolved concern about whether the objection applies to the selected source claim under its assumptions. Among the 162 dossiers with at least one all-YES review, 13 also had an adverse human judgment (ten from the initial returns, two from the targeted review, and one from the final audit), and two had such unresolved concerns. Excluding these 15 leaves the 147-dossier core, drawn from 137 distinct sources. Of the 147, 113 have exact matches between the reviewed scientific contents and the current contents; for the other 34, we confirmed that the saved reviews apply to the current versions, a confirmation recorded separately from exact content matching. The membership ledger and continuity receipt are `bugbank/core/membership.json` and `bugbank/core/author_continuity_receipt.json`. Across the earlier 284-dossier

Table 4: The nine nonapprovals in the final random audit. Q1, Q2 and Q3 check the source interpretation, witness and Lean statement. All nine are excluded from the current 147 dossiers and both benchmark subsets. "Earlier set" refers to the original 191 primary and 255 scorable inputs. Certificate versions are identified by SHA-256 prefixes; the linked version ledger gives full source, packet and certificate hashes. Concerns summarize the recorded reviews, not new adjudications.

Packet	Earlier set	Negative checks	Recorded concern	Certificate version
A001	Primary	Q3	Connection from the source's group construction to Lean is left to prose.	`51284638c1b1`
A002	Primary	Q2, Q3	Zero witness conflicts with an implicit nonzero assumption.	`af06bbfe5425`
A023	Primary	Q1, Q2, Q3	Category-label mismatch is treated as a refutation under a disputed translation.	`c8e308ad5c7d`
A043	Primary	Q3	Lean supplies the size-one transition law rather than deriving it.	`76b30ad2964d`
A072	Other scored	Q3	Boolean instance lacks formal syntax and substitution linking it to the source.	`d47105809371`
A075	Other scored	Q1, Q3	Humorous runtime claim is unspecified; Lean checks a supplied runtime function.	`af1c203b5790`
A085	Other scored	Q1	True equalities are displayed; the false step is implied rather than asserted.	`b0893e69a1ae`
A093	Not scored	Q1	Target is a typing requirement in the model setup, rather than an assertion.	`0e7eea1a1635`
A099	Not scored	Q1	Quoted supposition introduces a case; the gap is its later generalization.	`cf26d14808fe`

roster, 31 dossiers have an adverse human review; none enters the human-approved core. Within the core, 103 dossiers meet the primary-input criteria and 134 are scorable before the separate reference-line exclusions below.

The 100 primary benchmark inputs include 69 with a positive check of the current reference-line selection: 55 have a positive check with the exact current source text and labels, and 27 have a separate positive line-selection review, with overlap between these groups. The older line-review exports identify the reviewed labels but do not establish complete source-version identity. Three dossiers with adverse judgments on the current line selections remain in the human-approved core and are excluded from both evaluation subsets: `arxiv_2207.06033v2__source_enriched_codex`, `proofwiki_12198__codex_expansion`, and `reddit_mathematics_1dz93wr`. This leaves 100 primary and 131 total scorable inputs. The reference-line evidence is preserved in `benchmark/core/reference_line_coverage.json`.

All review-round answers are available under `bugbank/reviews/original100/`, `bugbank/reviews/earlier_selected30/`, `bugbank/reviews/selected33/`, `bugbank/reviews/selected30/`, `bugbank/reviews/scoped_supplements/`, and `bugbank/reviews/random100/`; `bugbank/reviews/README.md` explains the files and reproduction commands. Section C.7 describes the core's documentation.

C.7 The human-approved core and its documentation

The human-approved core contains 147 dossiers drawn from 137 distinct sources, selected from BugBank by the rule of Section C.6; human review is not a condition of membership in BugBank itself. The core contains 147 certificate paths and 141 distinct certificate hashes. After the separate checks of reference-line annotations, it supplies 100 primary benchmark inputs and 131 scorable inputs in total; the 31 additional inputs are shorter source texts. Core membership, review evidence, and benchmark eligibility are recorded for each dossier.

Each dossier connects the source text and quoted claim to a concrete witness, its mathematical explanation, and a Lean certificate. Hashes identify the source packet, witness fields, Lean source, and selected declarations; the saved builds confirm that the selected declarations compile, a separate check lists the axioms used by each selected theorem and confirms that only the allowed foundational

axioms occur, and the build receipts identify the compiler environment, dependencies, commands, and logs. Replacement witnesses appear alongside their predecessors, so a correction and its reason are inspectable; Section C.3 develops two such examples.

C.7.1 Definitions for describing the core

Source identifies where the claim appeared. A *mathematical topic* describes the subject of that claim. A *witness object* describes the counterexample itself, such as a graph, number, function, or algorithm trace. These classifications answer different questions: a graph, for example, can expose an error in an algorithms argument.

An *error mechanism* describes the immediate failed inference: a definition or domain condition, a quantifier, an omitted case, an algebraic or order argument, a count, a structural or analytic inference, or an algorithm invariant. The existing mechanism annotations select the operation closest to the quoted error and retain uncertainty flags.

Source collection	Dossiers	Inputs
QEDBench	25	20
Stack Exchange collection	18	11
math.SE collection	16	10
ProofWiki	28	26
arXiv	15	13
Reddit	42	20
Reference-proof additions	3	0
Total	147	100

Error mechanism	Dossiers	Inputs
Structural inference	38	28
Algebra or order	37	23
Omitted case	28	18
Counting	14	8
Quantifier scope	11	8
Definition or domain	9	6
Algorithm invariant	8	8
Analytic inference	2	1
Total	147	100

Table 5: Composition of the 147-dossier human-approved core and 100 primary benchmark inputs. Each panel counts every record once. Source collections follow the release groups, with the two Reddit groups combined. Error mechanisms use the saved primary annotations; six dossiers, including three primary inputs, retain a mechanism-annotation uncertainty flag.

Reproduction. The repository provides core membership, its inclusion rule, and version evidence in `bugbank/core/`. From the repository root, `python -B bugbank/core/verify.py --release .` reconstructs the core from the saved reviews and checks its scientific content, certificates, and benchmark masks, and `benchmark/core/replay.py` reproduces the benchmark scores; the core README gives the full commands. Earlier census tables, complete annotation evidence, and coding checks are preserved in `bugbank/annotations/`, with portable inspection instructions in its README.

D Search, repair, and retrieval: protocols and complete results

The localization benchmark asks where a critic places an error. The three studies summarized in Section 4.5 ask whether counterexamples help beyond localization: in directing a search, in completing a given correction, and in revising or developing a proof. Each study has its own inputs, conditions, and grading, described below, and none of them uses the 100-input benchmark.

D.1 Counterexample-first search versus sequential review

Inputs and conditions. The instruction study uses 100 proofs sampled from 560 inputs that the construction workflow had labelled as containing errors with concrete witnesses. This panel predates the human-approved core and is distinct from the benchmark. We compared four instructions, each applied to the same proof. *Counterexample-first search* asks the model to find a claim it can refute and stop. *Full review* judges every step. *Ordinary early stopping* may stop at the first failing step without a witness. *Sequential review with a counterexample required* may stop at the first failing step only when it also produces a concrete counterexample, so that it must deliver the same evidence as search. The main comparison is search against this last instruction. Appendix E gives the exact prompts, the execution settings, and the separate 60-statement panel.

Grading. A model grader (Zheng et al., 2023), `gpt-6-astra`, sees the saved reference defect and grades each answer. An objection counts as *confirmed* when the grader accepts the response's counterexample or self-contained refutation as refuting the reference defect (`CAUGHT_MATCHING`) or another defect (`CAUGHT_DIFFERENT`). An objection the grader finds unsupported is recorded as `FALSE_CATCH`. The comparison uses one common grading pass from September for all 100 inputs, including a regrading of the saved Opus 5 answers. Under that pass, Opus 5 has 81, 83, 76, and 76 confirmed objections for search, full review, early stopping, and review with a counterexample required, compared with 83, 84, 78, and 79 under its original grading. Table 6 reports the common September grades for all three models, while the four-instruction Table 7 keeps the original Opus 5 proof grades and uses the September grades for the other rows. The original token summaries use 2,000 bootstrap resamples and the paired comparison 20,000 (Efron, 1979).

	Confirmed objections		Search / sequential usage	
Model	Search	Sequential	Output ratio [95% interval]	Recorded total-token ratio [95% interval]
100 proof inputs per model				
Sol	95	93	.689 [.594,.802]	.958 [.833,1.083]
Gemini	84	89	.854 [.789,.921]	.650 [.441,.941]
Opus 5	81	76	.714 [.620,.814]	.831 [.762,.902]
60 statement inputs per model				
Sol	60	59	.593 [.457,.791]	.959 [.928,.982]
Gemini	58	57	.739 [.569,.889]	.903 [.755,1.063]
Opus	60	60	.476 [.240,.940]	.849 [.685,.983]

Table 6: Counterexample-first search versus sequential review or proof attempt, with a concrete counterexample required in both conditions. Confirmed objections are graded by a model using the reference counterexample. A token ratio below one favors search. Ratios compare summed usage across accepted attempts rather than averaging per-task ratios, and exclude grading and downstream verification. Recorded total tokens are uncached input, cached input, and output (reasoning counted once within output), plus reported auxiliary-model tokens in the 60-statement panel. Opus 5 proof-panel token accounting is provisional.

Results. Search used 31%, 15%, and 29% less recorded output than review with a counterexample required for Sol, Gemini, and Opus 5. The corresponding differences in confirmed objections are $+2$, -5, and $+5$ percentage points, with paired 95% bootstrap intervals $[-2, +6]$, $[-13, +3]$, and $[-2, +12]$; each interval includes zero, so the study does not establish an accuracy difference in either direction. The comparison instruction matters: against ordinary early stopping, which permits an objection without a concrete witness, search uses 6–11% more output. Requiring the same form of evidence in both conditions is therefore central to reading the comparison. On the separate panel of 60 false statements presented without proofs, search also used less output than a proof attempt that must supply a counterexample when it rejects a statement (Table 14 in Appendix E).

Output tokens are not the whole cost. Recorded token totals are uncached input plus cached input plus output, with reasoning counted once within output. Their ratios are 0.958 for Sol and 0.650 for Gemini, the Sol interval including one; on the 96 Sol and 94 Gemini task pairs with no recorded additional execution in either condition they are 0.977 and 0.955. Tables 7 and 8 give every grade and token component for the four instructions on the 100-proof panel; Appendix E gives the 60-statement panel's accounting. The proof totals are reconstructed from saved native usage and Opus 5 telemetry, with a publication receipt covering 1,709 source files; the reconstruction source is `studies/instructions/source/recompute_proofs100_full_components.py`, and `studies/instructions/README.md` gives portable replay instructions.

D.2 Token accounting and sensitivity

Answers and grades. Each model has 100 paired grades and 100 paired usage records for accepted attempts. Search yields 95 confirmed and 3 unsupported objections for Sol, against 93 and 4 for review with a counterexample required; the counts are 84 and 1 against 89 and 0 for Gemini, and 81 and 2 against 76 and 3 for Opus 5. The repository provides the source audit in `stud`

Table 7: All four instruction arms. n is the number graded; strict counts a confirmed objection (an accepted counterexample or refutation), len. also counts a correct unsupported verdict, false counts invalid objections, and unv. counts objects the grader could not check. Mean is accepted-response output tokens. Ratio is counterexample-first output divided by the row's output on paired records, with a 95% record-bootstrap interval; obs. includes recorded rejected-attempt output. Early stop permits stopping at the first failure; early + witness also requires a concrete counterexample. Original Opus proof grades and transcript accounting are retained; all other rows use the common September grades by `gpt-6-astra` and native usage. All comparisons have 100 proof pairs or 60 claim pairs, except original Opus early stop, with 99 token pairs that have recorded usage and 100 graded assessments. Absolute costs are not compared across harnesses. Appendix E gives prompts and accounting.

Input, model	Arm	n	Strict	Len.	False	Unv.	Mean	Ratio [95% CI]	Obs.
proofs, Opus (original)	counterexample	100	83	83	0	0	6952	1 (ref.)	–
	full verify	100	84	84	0	0	9416	0.738 [0.61, 0.88]	–
	early stop	100	78	78	0	0	6313	1.107 [0.96, 1.27]	–
	early + witness	100	79	79	0	0	9737	0.714 [0.62, 0.82]	–
proofs, GPT-5.6 Sol	counterexample	100	95	95	3	0	1229	1 (ref.)	–
	full verify	100	97	97	1	0	3345	0.368 [0.32, 0.43]	0.366
	early stop	100	89	90	6	1	1161	1.059 [0.92, 1.23]	1.070
	early + witness	100	93	93	4	0	1784	0.689 [0.59, 0.80]	0.699
proofs, Gemini 3.1 Pro	counterexample	100	84	84	1	0	11014	1 (ref.)	–
	full verify	100	81	81	5	0	10693	1.030 [0.90, 1.16]	1.038
	early stop	100	83	85	3	0	10150	1.085 [0.95, 1.24]	1.108
	early + witness	100	89	89	0	0	12896	0.854 [0.79, 0.92]	0.872
claims, Opus	counterexample	60	60	60	0	0	1980	1 (ref.)	–
	full proof	60	60	60	0	0	2230	0.888 [0.78, 1.05]	0.888
	early stop	60	60	60	0	0	1461	1.355 [1.18, 1.59]	1.355
	early + witness	60	60	60	0	0	4161	0.476 [0.24, 0.94]	0.235
claims, GPT-5.6 Sol	counterexample	60	60	60	0	0	1555	1 (ref.)	–
	full proof	60	59	60	0	0	2857	0.544 [0.44, 0.67]	0.544
	early stop	60	42	58	1	1	1155	1.347 [1.04, 1.75]	1.347
	early + witness	60	59	59	0	0	2622	0.593 [0.46, 0.79]	0.593
claims, Gemini 3.1 Pro	counterexample	60	58	58	2	0	6407	1 (ref.)	–
	full proof	60	42	57	2	1	7464	0.858 [0.62, 1.14]	0.858
	early stop	60	43	57	0	2	5026	1.275 [0.93, 1.75]	1.276
	early + witness	60	57	57	1	1	8666	0.739 [0.57, 0.89]	0.740

Table 8: Recorded token components for all four proof100 instruction arms. Sums cover accepted attempts with known usage; n is usage coverage out of 100, not grade coverage. Uncached input includes cache creation in the Opus 5 records. Reasoning is included in output once. Additional attempts, graders and downstream checks are excluded. Opus 5 transcript sums can repeat usage and remain provisional; its missing early-stop record is unknown, not zero. Absolute costs are not comparable across harnesses.

Model	Instruction	n	Uncached input	Cache reads	Output	Recorded total tokens
Sol	Search	100	665,020	2,256,000	122,923	3,043,943
Sol	Full review	100	735,037	2,548,992	334,459	3,618,488
Sol	Early stop	100	731,007	2,224,384	116,127	3,071,518
Sol	Early + witness	100	676,289	2,321,408	178,429	3,176,126
Gemini	Search	100	1,240,069	2,242,187	1,101,377	4,583,633
Gemini	Full review	100	1,730,843	4,196,566	1,069,289	6,996,698
Gemini	Early stop	100	1,406,962	3,818,526	1,014,987	6,240,475
Gemini	Early + witness	100	1,482,239	4,285,019	1,289,643	7,056,901
Opus 5	Search	100	2,995,056	13,577,136	695,247	17,267,439
Opus 5	Full review	100	3,293,960	16,204,707	941,599	20,440,266
Opus 5	Early stop	99	2,595,752	14,552,221	625,018	17,772,991
Opus 5	Early + witness	100	3,259,632	16,534,085	973,717	20,767,434

ies/instructions/reference_validity/audit.json and the input manifest in studies/instructions/reproduction/manifest.json.

Paired estimates. A token ratio divides the sum used by search by the sum used by sequential review on the same tasks, so a ratio below one means lower recorded usage for search. For token ratios and for differences in confirmed objections we resample task pairs 20,000 times with seed 20260917 and report the 2.5th and 97.5th percentiles (Efron, 1979); these pointwise intervals describe uncertainty for each comparison separately. The original output-token summaries use 2,000 resamples, which explains small differences in interval endpoints.

Input and cache accounting. Gemini reports uncached input separately from cache reads, so its recorded total adds both components to output; this corrects an earlier summary that subtracted cache reads from uncached input, and the saved receipts preserve the original components. Accepted-attempt ratios measure recorded solver usage; the ledgers separately identify additional attempts and unknown usage. Monetary comparisons would also require prices and billing rules for each execution system.

Connecting the study to the source audit. The saved crosswalk matches each study input to the 425-record source audit by record_id or its explicit original_record_id alias; two Graph Theory records use the alias. This gives 46 retained, 25 excluded, and 29 unmatched records in that earlier audit, against 46, 23, and 31 when only literal identifiers are matched, with the same 21 human-review overlaps either way. These counts describe the earlier audit; current core membership is recorded separately. Establishing that a review applies to an experiment's saved reference also requires comparing the source text and witness, which the next subsection does; the original human returns identify their review packets but do not record hashes of the files the reviewers opened.

Recorded tool use. The instructions allow reading the input and prohibit other code execution, yet the accepted-attempt logs record six additional executions across four Sol task pairs and 48 across six Gemini pairs: for Sol, one search cell and three review cells with execution totals of 2 and 4; for Gemini, two and five cells with totals of 6 and 42. An execution here is any recorded non-read command or tool call and can include work other than mathematical computation. The main comparison keeps all 100 task pairs; the subset of pairs with no recorded additional execution in either condition (96 for Sol, 94 for Gemini) describes observed runs and does not randomize tool permissions, and Opus 5 has no matching execution telemetry. A controlled comparison would assign those permissions before execution. The per-task data are in studies/instructions/reference_validity/toolfree_sensitivity.json.

How one large execution moves the average. For Gemini, the median per-task recorded-total ratio is 0.952, while the ratio of sums is 0.650. The difference arises in part from one sequential review, arxiv_1904.10482v2__source_enriched_codex, which records 24 additional executions and 1,639,229 input tokens, or 1,686,890 including output; omitting that pair gives a ratio of 0.830. The complete 100-pair result and this sensitivity together show how single long runs shape aggregate usage.

D.3 Checking the reference objections

We revisited the reference objections for the 100 proofs of the instruction study. Twenty-two cases had matching full source texts and unchanged witnesses supported by the earlier automated checks: 15 exact text matches and seven complete-text matches after removing line numbers and normalizing whitespace. Cases with only a matching excerpt or a replacement witness were excluded, so this subset was selected by the source-matching rules, not sampled at random. On it, search and review with a counterexample required received 22 and 21 confirmed objections for Sol, 21 and 20 for Gemini, and 19 and 19 for Opus 5; Table 9 gives the paired token comparisons. These are results from the earlier experiment; Section C.7 defines the human-approved core.

What changed when the references were checked? Thirteen original references failed later checks: ten had a recorded source-interpretation problem, and three used witnesses that were replaced as invalid. The saved grader nevertheless marked 60 of their 78 model-and-instruction responses as

Table 9: The 22 source-matched cases from the instruction study. Counts compare search with sequential review with a counterexample required, under the common September grades. Ratios compare accepted solver-attempt token sums. Intervals use the paired 20,000-resample analysis described in Section D.2.

Model	Catches	Δ catches (pp)	Output ratio	Recorded total-token ratio
GPT-5.6 Sol	22/21	+4.5 [+0.0, +13.6]	0.759 [0.633, 0.919]	0.986 [0.981, 0.993]
Gemini 3.1 Pro	21/20	+4.5 [-9.1, +18.2]	0.721 [0.626, 0.819]	0.909 [0.863, 0.951]
Opus 5	19/19	+0.0 [-13.6, +13.6]	0.703 [0.627, 0.784]	0.948 [0.866, 1.031]

matching the reference objection; the other 18 received six supported-alternative, one unsupported-objection, and eleven missed grades. Agreement with a stored reference must therefore be checked against the source. Scores also move with the subset: Gemini has 84 and 89 confirmed objections on all 100 inputs, 35 and 39 on the 42 cases supported by the recorded automated checks, and 21 and 20 on the 22 source-matched cases, search listed first.

Three examples. These cases from the earlier source audit show why an objection's exact target matters.

`mse_25914__ans25916`: a local proof step.
The proposed step requires a product of strictly smaller positive integers. Prime 7 has no such factorization. The source's assertion about expressing an integer as an odd number times a power of two is not disproved. The selected Lean theorem addresses the product claim, including possible factors 1.

`mse_1513740__ans1513768`: a local size comparison.
The proposed strict size comparison fails at $k = 2$: the reduced system has two equations, not fewer than two. This refutes the printed comparison, not the underlying solvability theorem.

`SE_cs_12812__a12823`: an algorithmic claim.
The source proposes satisfying the first disjunct as its procedure. It does not assume that every disjunct is internally consistent. The witness tests that stated procedure rather than adding such an assumption.

The instruction-experiment reproduction guide links the source-matching rules, record lists, saved responses, grades, token components, and example evidence; the original 100-input comparison and every group above remain available there.

D.4 Repair tasks with the correction given

This study asks whether a checked counterexample helps a solver complete a specified mathematical correction. It uses 16 Lean tasks derived from source arguments. Both conditions supply the source context, definitions, identified defect, repair template, and required proof statements; condition A uses this information alone, and condition B adds the checked counterexample. The task's fixed checker accepts a submission when its edits are allowed and its proofs establish the requested correction for all permitted inputs, show that the hypotheses admit an example, and establish any task-specific endpoint properties. Each of three solvers, Opus, Sonnet, and GPT-OSS 120B, received up to three attempts per task and condition.

Table 10 reports first-attempt acceptance; "Absent" means no submission was saved, and missing submissions count as failures. The counts are the same when acceptance also requires compliance with the runtime rules, because a later runtime stop leaves an already saved, accepted submission intact.

Across up to three attempts, Opus obtained an accepted submission on all 16 tasks in both conditions; Sonnet completed 14 of 16 without the counterexample and 16 of 16 with it; GPT-OSS 120B completed 2 of 16 and 3 of 16. Sonnet's first attempts passed in both conditions on 13 tasks, in neither

Table 10: First-attempt checker-accepted submissions on 16 source-derived repair tasks.

Solver	Condition	PASS	FAIL	Absent
Claude Opus 5.5	A	15	0	1
Claude Opus 5.5	B	15	0	1
Claude Sonnet 5	A	13	3	0
Claude Sonnet 5	B	15	1	0
GPT-OSS 120B	A	1	10	5
GPT-OSS 120B	B	2	8	6

on one, and only with the counterexample on two. The paired outcomes show a benefit for Sonnet on these given repairs; the small task set leaves the broader effect unresolved.

Table 11 reports token usage across all 16 selected first attempts for each solver and condition, including failed attempts. Recorded total tokens include output and all reported input components, including cache creation and reads; the released artifact gives the component breakdown. The symbol $\geq$ marks a lower bound where provider usage is unavailable. Earlier attempts made to recover from execution problems, and evaluation costs, are recorded separately. The tasks, checker obligations, saved submissions, and replay instructions are documented in `studies/repairs/README.md`.

Table 11: Token usage for selected first attempts (16 tasks per condition). The last column counts attempts with recorded native usage out of 16.

Configuration	Condition	Recorded total tokens	Output tokens	Attempts with recorded usage
Opus 5.5	A	$\geq$ 598,672	$\geq$ 42,733	15/16
Opus 5.5	B	$\geq$ 746,580	$\geq$ 46,799	15/16
Sonnet 5	A	$\geq$ 9,083,964	$\geq$ 247,815	13/16
Sonnet 5	B	8,819,081	221,411	16/16
GPT-OSS 120B	A	2,086,312	225,343	16/16
GPT-OSS 120B	B	1,720,809	227,225	16/16

D.5 Retrieval during proof revision

This study asks whether retrieved counterexamples help a model develop a proof through repeated revision. Each model worked on the same 50 tasks in two conditions. The BugBank condition could retrieve up to three records for each queried claim from an earlier 661-entry database, ranked by SQLite FTS5 `bm25` (Robertson & Zaragoza, 2009) (`https://sqlite.org/fts5.html#the_bm25_function`); the Plain condition used iterative self-feedback (Madaan et al., 2023). Each row of Table 12 grades all 100 final drafts, 50 from each condition. The shared-rule protocol uses the same revision cap and stopping rule in both conditions; the original-protocol rows retain the earlier stopping rules as a separate comparison. These tasks and the retrieval database belong to this study alone and are separate from the human-approved core and its benchmark.

Under the shared rule the models differ. Opus used fewer recorded tokens with retrieval, a ratio of 0.696 with interval [0.555, 0.887], and had 44 proofs judged correct against 43. Sol and Gemini used more tokens with retrieval, with 45 against 47 and 43 against 42 proofs judged correct. Under the original stopping rules all three ratios exceeded one. The comparison evaluates complete revision procedures, so its reading depends on the feedback source and the stopping rule together.

Grading used 3,312,989 recorded tokens for Sol, 3,536,651 for Gemini, and 3,806,090 for Opus, with 200 grading receipts and complete grading-usage records per model. Additional-attempt records contain two attempts with unknown usage for Sol, none for Gemini, and at least 349,775 tokens for Opus, 25 of whose 26 additional attempts lack complete native usage; these costs are recorded outside the selected-attempt totals and are not allocated to conditions or protocols. The inputs, prompts, grading records, retrieval snapshot, and deterministic replay are documented in `studies/proof_development/README.md`.

Table 12: Proof development outcomes on 50 tasks per model, reported as BugBank / Plain. Correctness is judged by model graders using the recorded rubric. Recorded total tokens are uncached input, cached input, and output (with reported reasoning counted once within output) for the selected completed solver attempts, including auxiliary calls; output tokens also include auxiliary calls. Grading, rejected attempts, and later checks are accounted for separately. Total-token ratios divide the BugBank condition by Plain, with 95% intervals from 2,000 paired bootstrap resamples of tasks.

Model	Protocol	Graded correct (/50)	All-model output tokens	Recorded total tokens	Total-token ratio [95% CI]
GPT-5.6-Sol	shared	45 / 47	315,167 / 339,701	9,762,586 / 7,184,842	1.359 [1.211, 1.511]
Gemini 3.1 Pro	shared	43 / 42	491,041 / 552,991	10,354,373 / 7,949,686	1.302 [1.193, 1.428]
Opus 5	shared	44 / 43	746,839 / 1,023,808	6,768,539 / 9,721,951	0.696 [0.555, 0.887]
GPT-5.6-Sol	original	48 / 48	294,955 / 253,003	10,089,707 / 5,280,740	1.911 [1.634, 2.215]
Gemini 3.1 Pro	original	43 / 42	442,707 / 636,047	8,167,805 / 4,189,656	1.950 [1.773, 2.140]
Opus 5	original	44 / 46	902,816 / 1,094,155	8,160,358 / 7,651,046	1.067 [0.869, 1.349]

D.6 Retrieval in open-problem search

This paired study examined retrieval during longer mathematical searches. Each target was pursued with BugBank retrieval (B) and with self-critique and model auditing alone (P). The main comparison contains 36 distinct final targets; for 22 of them a continued search replaced the original endpoint, so each target is counted once. A model audit accepted a full resolution of the stated target in 12 retrieval runs and 11 self-critique runs: eleven targets were accepted in both conditions, one only with retrieval, and 24 in neither. "Audited-full" in Table 13 denotes this saved model-audit decision; the deciding audits for the 31 original-run targets could see which condition used retrieval, and independent checks of the mathematical claims are a separate step. The table also reports 58 observations from individual stages of these searches; some stages concern the same target, so they are not 58 independent targets. Two further panels are disjoint from the main comparison: six exploratory targets from fresh sources, with no acceptance in either condition, and six targets chosen to test BugBank retrieval, with one acceptance in each.

Component model tokens count completed, valid revision and audit calls through the recorded endpoint; they exclude the shared starting draft and audit, retrieval, invalid or failed attempts, and later work. Original and hard-target rows report cumulative endpoint usage; the continuation row reports only added usage. The Plain condition was required to use at least the BugBank token budget before stopping without a full resolution, so the usage differences partly reflect the execution protocol rather than a causal efficiency gain from retrieval, and because the stages overlap their costs are reported separately rather than summed. The target definitions, saved model audits, continuation rules, and replay instructions are in `studies/open_problem_retrieval/README.md`.

Component	Paired units	Audited-full (B / P)	Both	B-only / P-only	Neither
Distinct targets, final endpoints	36	12 / 11	11	1 / 0	24
Protocol stages, descriptive	58	12 / 11	11	1 / 0	46
Original run	31	9 / 8	8	1 / 0	22
Hard-target run	5	0 / 0	0	0 / 0	5
Fixed-point continuation	22	3 / 3	3	0 / 0	19
Exploratory cohort	6	0 / 0	0	0 / 0	6
Targeted stress test	6	1 / 1	1	0 / 0	5

Table 13: Paired-search components comparing BugBank retrieval (B) against a self-critique/audit condition (P). Component model token usage (B / P): Original $16{,}449{,}383/17{,}590{,}589$; Hard $1{,}683{,}108/2{,}050{,}662$; Continuation (incremental) $19{,}577{,}279/20{,}301{,}967$; Exploratory $1{,}370{,}760/1{,}710{,}782$; Targeted $1{,}372{,}591/1{,}436{,}842$.

E Instruction study: prompts, grading rubrics, and token accounting

This appendix gives the inputs, the exact prompts, the execution settings, the grading rubrics, and the token accounting behind the instruction comparisons of Appendix D and Table 7. The repository

holds the saved inputs, answers, model grades, and usage records in `studies/instructions/`, and its `README.md` explains how to reproduce every table from them. The prompt quotations reproduce `studies/instructions/reproduction/prompts.json`, with only LaTeX escaping and dash formatting changed.

E.1 Study design

Question and comparisons. The study asks how the instruction changes the effort needed to identify a false claim. The initial experiment applied four instructions to 100 proofs with Opus 5 at high effort in the Claude Code execution system; the reproduction ran the same 100 proofs with `gpt-5.6-sol` and Gemini 3.1 Pro, and a separate set of 60 statements without proofs with all three models, reusing the saved Opus 5 proof answers for its paired comparisons. Instructions are compared within each model, with the token accounting that its execution system provides.

The 100-proof input set. Each *proofs100* record contains a problem statement and a numbered proof, stored in the JSON fields `record_id`, `problem`, and `proof_numbered`; a manifest fixes the input list and the input and reference files by SHA-256 hashes. These are the earlier instruction-study inputs, not the 100 benchmark inputs drawn from the human-approved core.

The 60-statement input set. The *claims60* panel contains 60 statements assessed as false in a skeptical review, each with a reference counterexample. Executable checks support 57 of the references, 47 marked fully verified and 10 with exhaustive-search witnesses; the other three rest on the literature, recorded as two partial checks and one citation-only check in the `verification_level` field. Each model receives the statement with its definitions, conventions, and an opaque identifier. The claims and witnesses are separate from BugBank records.

Some definitions hint at a refutation. Before the study ran, we fixed a text rule to identify such hints, the case-insensitive regular expression `/counterexample|claim under test|the claim (asserts|quantifies)|to be refuted|refut/i` applied to the definitions field; it flags 13 of the 60 claims. Section E.2 summarizes the comparisons on the remaining 47 no-hint claims and gives the complete sensitivity table.

Arms and prompts. Each cell sends one prompt. The proof template is

```
You are assessing a mathematical proof for correctness.
Read {input_path} -- it contains the problem statement and a
line-numbered proof.  Use ONLY that file:  no web, no other
files, no code execution.  (Reading that file with your
file-reading tool, or with a single shell command that only
prints it, is allowed and is not code execution.)
{method}
Honesty rules:  do not assume the proof is flawed; many
proofs are fine.  Do not pad.  In your structured output set
record_id to exactly {record_id_json} and arm to exactly
"{arm}".
```

with `{method}` one of the instructions below (the arm), `{input_path}` the record's input file, and `{record_id_json}` and `{arm}` the values the answer must echo. *`refute`*:

```
METHOD you must follow -- COUNTEREXAMPLE-FIRST: read the
proof looking for its riskiest concrete claims, and try to
construct ONE concrete counterexample (explicit numbers,
explicit small structure, explicit trace) to some claim the
proof actually relies on.  The moment you have a
counterexample you have checked carefully, report INVALID
with that counterexample and STOP -- do not audit the rest
of the proof.  If after honest effort you cannot refute
anything and believe the proof is correct, report VALID with
counterexample "".
```

`verify_full`:

```
METHOD you must follow -- COMPLETE VERIFICATION: verify the
proof step by step.  For EVERY numbered step, decide whether
it is justified given the problem and the preceding steps,
and record a one-line justification or objection per step in
assessment_summary.  You must judge every step even after
finding a problem.  Verdict INVALID if any step fails (name
the first failing step in flawed_step), VALID otherwise.
counterexample may be "" -- your job is verification, not
witness construction.
```

verify_earlystop:

```
METHOD you must follow -- VERIFICATION WITH EARLY STOP:
verify the proof step by step in order.  For each numbered
step, decide whether it is justified given the problem and
the preceding steps.  You MAY stop at the first step you are
certain fails; report INVALID with that step in flawed_step.
If you reach the end, report VALID. counterexample may be ""
-- your job is verification, not witness construction.
```

verify_earlystop_witness:

```
METHOD you must follow -- VERIFICATION WITH EARLY STOP,
WITNESS REQUIRED: verify the proof step by step in order.
For each numbered step, decide whether it is justified given
the problem and the preceding steps.  You MAY stop at the
first step you are certain fails -- but a verdict of INVALID
is only complete when you ALSO produce a CONCRETE
counterexample (explicit numbers, explicit small structure,
explicit trace) to that failing step, checked carefully, in
the counterexample field.  This arm exists to equalize
deliverables:  your INVALID must come with the same artifact
a counterexample-first assessment produces.  If you reach
the end without a definitive failure, report VALID with
counterexample "".
```

For the claims the template reads

```
You are assessing a mathematical statement for truth.  Read
{input_path} -- it contains the statement and the
definitions and conventions under which it is to be read.
Use ONLY that file:  no web, no other files, no code
execution.  (Reading that file with your file-reading tool,
or with a single shell command that only prints it, is
allowed and is not code execution.)
{method}
Honesty rules:  do not assume the statement is false; many
statements are true.  Do not pad.  In your structured output
set record_id to exactly {record_id_json} and arm to exactly
"{arm}".
```

and the verification instruction becomes a complete proof attempt; the counterexample-first instruction is unchanged in method: *refute*:

```
METHOD you must follow -- COUNTEREXAMPLE-FIRST: read the
statement looking for its riskiest implication, and try to
construct ONE concrete counterexample (explicit numbers,
explicit small structure, explicit trace) that satisfies
every hypothesis of the statement and violates its
conclusion.  The moment you have a counterexample you have
checked carefully, report INVALID with that counterexample
and STOP -- do not attempt a proof.  If after honest effort
you cannot refute it and believe the statement is true,
report VALID with counterexample "".
```

prove_full:

> METHOD you must follow -- COMPLETE PROOF ATTEMPT: attempt to prove the statement. Write a complete numbered step-by-step argument in assessment_summary and, for EVERY numbered step, decide whether it is justified given the definitions and the preceding steps, recording a one-line justification or objection per step. You must judge every step even after finding a problem. Verdict VALID if every step is justified and the argument proves the statement; INVALID if the argument fails and you conclude the statement is false (name the first failing step in flawed_step). counterexample may be "" -- your job is verification by proof, not witness construction.

prove_earlystop:

> METHOD you must follow -- PROOF ATTEMPT WITH EARLY STOP: attempt to prove the statement step by step in order. For each numbered step, decide whether it is justified given the definitions and the preceding steps. You MAY stop at the first step you are certain cannot be justified; report INVALID with that step in flawed_step. If your argument completes, report VALID. counterexample may be "" -- your job is verification by proof, not witness construction.

prove_earlystop_witness:

> METHOD you must follow -- PROOF ATTEMPT WITH EARLY STOP, WITNESS REQUIRED: attempt to prove the statement step by step in order. For each numbered step, decide whether it is justified given the definitions and the preceding steps. You MAY stop at the first step you are certain cannot be justified -- but a verdict of INVALID is only complete when you ALSO produce a CONCRETE counterexample (explicit numbers, explicit small structure, explicit trace) to the statement, checked carefully, in the counterexample field. This arm exists to equalize deliverables: your INVALID must come with the same artifact a counterexample-first assessment produces. If your argument completes without a definitive failure, report VALID with counterexample "".

Within each input set the four instructions share the surrounding prompt, the output schema, the grading rubric, and the grader; only the method block changes. The shared proof text has 338 characters before the method and 179 after it, and the statement text 367 and 186. Each condition receives one input file and no retrieved BugBank records.

The answer schema requires six string fields, `record_id`, `arm`, `verdict`, `flawed_step`, `counterexample`, and `assessment_summary`, with the verdict one of `VALID`, `INVALID`, or `CANNOT_DECIDE` and no extra fields. The initial study called the last field `reasoning_summary`; the Opus 5 interface rejected that name during the reproduction, so all models used `assessment_summary`.

Execution setup. Each model–input–instruction combination runs as one command-line session under Slurm (`https://slurm.schedmd.com/`) in a bubblewrap sandbox (`https://github.com/containers/bubblewrap`). The sandbox hides the research repository, scratch and temporary directories, and previous sessions. Within the session, it exposes the working directory, the answer schema, and one input file, with credentials supplied separately, while the rest of the host file system and the network remain visible. The prompt permits only reading the input, either with a file-reading tool or with a shell command that prints the file.

The configurations are:

- Opus 5: `claude -p`, high effort, only the `Read` tool, with the CLI's JSON-schema output option.
- GPT-5.6 Sol: `codex exec`, codex-cli 0.154.0, high reasoning effort, approvals disabled, read-only sandbox, and the CLI's output schema.
- Gemini 3.1 Pro: the High variant in Google Antigravity CLI 1.2.3, high effort, JSON-schema output, and a 55-minute print timeout.

Codex and Antigravity retain tools that their interfaces do not let the driver remove, so a rule fixed before the runs flags any Codex shell command other than printing the input and any Antigravity tool step other than viewing the input or finishing. Such calls occur in one to three Codex cells per proof instruction, two to six Gemini cells per proof instruction, and up to three Gemini cells per statement instruction; their effect on the statement comparisons is reported below. Answers returned without reading the input are rejected and retried.

Retries and recovery. The driver runs six concurrent sessions for each Sol run and four for the Opus and Gemini runs and for each grading job, with a 3,600-second limit per session. Transport failures, timeouts, refusals, and invalid answer formats allow up to three attempts. A rate- or usage-limit signal (a structured signal from Claude, or matching CLI error messages from Codex and Antigravity) triggers a 20-minute pause without consuming an attempt. Six consecutive failures trigger a separate 20-minute pause, and three such pauses stop the driver. A cell that exhausts its attempts remains without an answer. Jobs relaunched after the recorded Codex interruption received an enlarged limit of six attempts.

The ledger records every attempt with its prompt hash. Slurm job identifiers in the lock files allow an interrupted run to resume while keeping its completed answers. The driver can also recover an answer from the newest usage record if recording was interrupted, a fallback that no reported run needed.

Token accounting. The main ratios use the output tokens of the accepted attempt: Claude's per-model `outputTokens`, Codex's `turn.completed` output, and Gemini's result usage, each including the engine's reported reasoning. Uncached and cached input come from the same receipts. Additional attempts, and attempts whose usage could not be recovered, are reported separately; when an interrupted shell failed to save a usage file, the available usage was recovered from the event log.

In the initial Opus study the agents also wrote their answers through a tool call, which added that text to output usage; the reproduction emits the structured answer once. This changes the absolute count by roughly one answer per cell in every condition and leaves the paired design intact.

E.2 Token accounting for the 60-statement panel

The accounting joins all 720 accepted usage records to their native receipts and completed ledger attempts, and every answer and grade to its record and instruction; all 120 input and reference files match the source manifest. Recomputing usage leaves the saved answers, grades, and output-token counts unchanged. The complete component table for all four instructions (`claims60_corrected_components.tex`), the full-panel comparisons (`RESULTS.json` and `README.md`), and the 3,018 source-file hashes are in `studies/instructions/corrected_claims60_accounting/`, with portable replay instructions in `studies/instructions/README.md`; the complete subset-sensitivity table is Table 15 below.

Counting each component once. Sol's native input count already includes cached reads, whereas Gemini's `input_tokens` holds only uncached input, so its cached reads must be added; an earlier Gemini summary had subtracted them. Adding the disjoint components gives search/witness totals of 2,480,272/2,745,952 and a ratio of 0.903, correcting 0.872, and changes the search/full-proof ratio from 0.978 to 1.031; output ratios are unchanged. Opus receipts report uncached input, cache creation, cache reads, and output separately. Reasoning is counted once, within output, for every model.

Opus 5 reports usage for the target model and for auxiliary models. Its target-model search/witness totals are 751,620/895,394, a ratio of 0.839; including the reported auxiliary usage gives

Model	Supported S/O/W	Output S/W	Recorded total S/W
Sol	60/42/59	0.593 [0.457, 0.791]	0.959 [0.928, 0.982]
Gemini	58/43/57	0.739 [0.569, 0.889]	0.903 [0.755, 1.063]
Opus	60/60/60	0.476 [0.240, 0.940]	0.849 [0.685, 0.983]

Table 14: The separate 60-statement panel. S is search, O an ordinary early-stop proof attempt, and W a proof attempt that must supply a counterexample. Confirmed-objection counts use the same reference-assisted grader. Ratios compare accepted-attempt token sums. Recorded total tokens are uncached input, cached input, and output (with reported reasoning counted once within output), including reported Opus auxiliary usage. Brackets are pointwise paired 95% intervals; output uses 2,000 resamples and recorded totals 20,000.

824,253/970,902 and 0.849, the all-reported total that Table 14 uses, while the saved accounting also keeps the target-model view. Sol and Gemini supply native aggregate totals without an auxiliary-model breakdown.

Do hints or additional executions change the pattern? Two observational subsets check the pattern: the no-hint subset removes the 13 claims flagged by the pre-run text rule, and the no-additional-execution subset keeps a task pair only when both of its runs recorded no extra execution. Neither subset randomizes tool permissions, and Opus has no execution-count field. For Gemini, excluding the pairs with additional executions changes the output and the all-reported total-token ratios as follows. Against a full proof attempt (59 pairs), the output ratio moves from 0.858 to 0.844 and the total-token ratio from 1.031 to 0.968. Against ordinary early stopping (59 pairs), they move from 1.275 to 1.352 and from 1.210 to 1.046. Against early stopping with a counterexample required (57 pairs), they move from 0.739 to 0.654 and from 0.903 to 0.923. Sol has no such pairs.

On the 47 no-hint claims, the search/witness output ratios are 0.535, 0.689, and 0.396 for Sol, Gemini, and Opus, and the corresponding all-reported total-token ratios are 0.951, 0.942, and 0.809: lower output-token counts can coexist with similar recorded totals. Table 15 gives all three comparisons across all subsets, including the extra output used by search relative to ordinary early stopping.

Table 15: Claims60 instruction and subset sensitivity. S denotes search; F full proof attempt, O ordinary early stop, and W early stop requiring a witness. Counts use preserved grades; token ratios are S/comparator. Tool-free means no recorded additional execution in either paired cell. Exclusions are descriptive. Opus tool use is unrecorded, so no tool-free row is inferred. Output is the target model; recorded total tokens include every model reported in the accepted receipt.

Model	Arm	Subset	n	Catches S/C	Output	Recorded total
Sol	F	All	60	60/59	0.544	0.954
Sol	F	No hint	47	47/47	0.507	0.949
Sol	F	Tool-free	60	60/59	0.544	0.954
Sol	F	Both exclusions	47	47/47	0.507	0.949
Sol	O	All	60	60/42	1.347	1.015
Sol	O	No hint	47	47/36	1.260	1.011
Sol	O	Tool-free	60	60/42	1.347	1.015
Sol	O	Both exclusions	47	47/36	1.260	1.011
Sol	W	All	60	60/59	0.593	0.959
Sol	W	No hint	47	47/46	0.535	0.951
Sol	W	Tool-free	60	60/59	0.593	0.959
Sol	W	Both exclusions	47	47/46	0.535	0.951
Gemini	F	All	60	58/42	0.858	1.031
Gemini	F	No hint	47	46/37	0.768	1.028
Gemini	F	Tool-free	59	57/42	0.844	0.968
Gemini	F	Both exclusions	46	45/37	0.742	0.947
Gemini	O	All	60	58/43	1.275	1.210
Gemini	O	No hint	47	46/36	1.130	1.235
Gemini	O	Tool-free	59	57/43	1.352	1.046
Gemini	O	Both exclusions	46	45/36	1.198	1.027
Gemini	W	All	60	58/57	0.739	0.903
Gemini	W	No hint	47	46/45	0.689	0.942
Gemini	W	Tool-free	57	56/55	0.654	0.923
Gemini	W	Both exclusions	45	45/44	0.610	0.915
Opus	F	All	60	60/60	0.888	0.983

Model	Arm	Subset	n	Catches S/C	Output	Recorded total
Opus	F	No hint	47	47/47	0.845	0.977
Opus	O	All	60	60/60	1.355	1.043
Opus	O	No hint	47	47/47	1.275	1.034
Opus	W	All	60	60/60	0.476	0.849
Opus	W	No hint	47	47/47	0.396	0.809

Additional attempts. The solver ledgers contain 730 attempts: Sol has no rejected attempt, Gemini has three with known usage totaling 29,135 tokens, and Opus has six failed attempts with unknown usage and one rate-limited attempt with known target-model usage of 283,653 tokens. Unknown usage is recorded as unknown, including when a receipt lists zero API messages. The accepted-attempt ratios therefore describe completed solver answers; grading, later checking, and unrecorded interrupted attempts need separate accounting before an end-to-end cost can be estimated.

Grading. The grader, `gpt-6-astra` through Codex CLI at maximum effort, grades the new answers and regrades the 400 original Opus proof answers, receiving the assessment and the held-out reference inline in the same read-only sandbox with its shell tool available. The recorded September 15 pause contains 1,700 completed grading cells, six with shell commands including three attempted arXiv downloads; the completed dataset contains 1,920 cells, seven with shell commands. The extra 220 completions and the additional command-bearing cell are documented in `studies/instructions/reproduction/grader_cohort.json`, with usage and recorded commands in its adjacent `grade_usage/` directory; the 1,700-cell pause cohort was reconstructed from timestamps.

The two saved rubrics follow; their judgments are weighed against the source and witness checks in Section D.3. The proof rubric is:

```
You are grading one proof assessment against held-out ground
truth.
Read these two files:
1.  {assessment_path} - the assessment (verdict, claimed
flawed step, counterexample).  If this file does not exist,
use grade MISSED with justification "assessment absent
(quota interruption)".
2.  {truth_path} - the ground truth:  the confirmed flaw and
its independently verified witness.
Every proof in this sample IS flawed (pipeline-confirmed).
Grade the assessment:
- CAUGHT_MATCHING: it reported INVALID and its flawed
step/counterexample targets the SAME defect as the ground
truth (rephrasing and a different-but-valid witness for the
same claim both count as matching).
- CAUGHT_DIFFERENT: it reported INVALID with a DIFFERENT
flaw, and after checking its support concretely yourself,
that flaw is genuine.
- MISSED: it reported VALID or CANNOT_DECIDE.
- FALSE_CATCH: it reported INVALID but the stated flaw does
not hold up under your concrete check (wrong math, misread
claim, invalid witness).
Be strict about FALSE_CATCH vs CAUGHT_DIFFERENT: verify, do
not take the assessment's word.
Set record_id exactly {record_id_json}, arm "{arm}".
```

The statement rubric adds two outcomes, a correct verdict without a checkable refutation and an object the grader cannot verify; both input sets share this grade list, and in Sol's early-stop proof condition one answer received each added grade, counted separately from confirmed and unsupported objections. The exact statement rubric is:

```
You are grading one assessment of a mathematical statement
against held-out ground truth.
Read these two files:
```

```
1.  {assessment_path} - the assessment (verdict VALID = the
statement is true, INVALID = false, CANNOT_DECIDE; a claimed
failing step; a counterexample).
2.  {truth_path} - the ground truth:  the statement is FALSE
and this file gives an independently verified counterexample
(the reference witness) and, where recorded, why the
statement fails.
Every statement in this sample IS false (reviewer-confirmed,
witness verified).  Grade the assessment:
- CAUGHT_MATCHING: it reported INVALID and its
counterexample (or, if it gave no explicit witness, its
stated reason) is a VALID refutation of the same kind as the
reference (the same object or family, or the same failing
mechanism; a different-but-valid witness of the same kind
counts as matching).
- CAUGHT_DIFFERENT: it reported INVALID with a DIFFERENT
valid refutation (a genuinely different counterexample
object or a different correct reason), and after checking
its support concretely yourself, that refutation is genuine.
- MISSED: it reported VALID or CANNOT_DECIDE.
- FALSE_CATCH: it reported INVALID but the stated
counterexample or reason does not hold up under your
concrete check (wrong math, an object that violates a
hypothesis or satisfies the conclusion, a misread statement,
or no checkable support at all).
- CAUGHT_UNSUPPORTED: it reported INVALID (the correct
verdict) but gave neither a counterexample nor a
self-contained argument that the statement is false.  A
'stated reason' counts as a refutation only if it is itself
a complete argument (for example a parity or counting
argument, or a named object together with why it meets every
hypothesis and fails the conclusion).  'My proof attempt
fails at step k', 'the hypotheses do not force the
conclusion', or naming a gap in a would-be proof is NOT a
refutation.
- UNVERIFIED: it reported INVALID with a named or large
object (a famous graph, a structure with dozens of vertices,
a construction you cannot check by hand from the
assessment's own text) that is NOT the reference object and
that you cannot verify concretely.  Never guess:  use
UNVERIFIED rather than CAUGHT_DIFFERENT or FALSE_CATCH in
that case.  If the object IS the reference object (or an
explicitly stated variant the truth file already covers),
grade CAUGHT_MATCHING.
Be strict about FALSE_CATCH vs CAUGHT_*:  verify concretely,
do not take the assessment's word; a counterexample must
satisfy every hypothesis in the definitions and violate the
conclusion.  FALSE_CATCH is only for INVALID whose supplied
witness or argument is wrong under your concrete check.
Clauses in the definitions that describe what a
counterexample must include are guidance to the solver:
grade on whether the object refutes the statement under the
stated conventions, performing any omitted verification
(trace, tie-break check, non-isomorphism) yourself; omission
of demanded verification text alone never makes a
FALSE_CATCH.
Set record_id exactly {record_id_json}, arm "{arm}".
```

F THE LOCALIZATION BENCHMARK: PROTOCOL AND COMPLETE RESULTS

Current evaluation set. The main evaluation uses 100 texts from the human-approved core, each with at least ten nonblank source lines, grouped into 92 problem clusters; a supplementary evaluation adds 31 shorter texts, for 131 scorable inputs in 123 clusters. The remaining core dossiers are 12 pilot

records, one unscorable record, and three records with unresolved reference-line concerns; the line concerns affect which lines to score, and those dossiers remain in the core. Appendix C describes the reviews, the core's selection, and every exclusion.

The current scores reuse the saved model answers under the original scoring rules; membership was fixed from the review evidence before the answers were rescored. The inclusion ledger is `bugbank/core/membership.json`, and `benchmark/core/README.md` explains how to reproduce both evaluation sets.

Inputs and reference lines. A critic receives the numbered source text and a request for up to three candidate lines, ranked by preference, with a brief explanation. The prompt says that the text contains an error and withholds the dossier's selected passage, counterexample, and Lean certificate. For some arXiv records the supplied text is an excerpt from the paper; the released input is in every case the text the critic was asked to assess.

The reference lines of an input are its accepted error locations, including restatements and instantiations of the challenged claim; selecting any of them earns credit, and 57 of the 100 primary inputs have more than one. Model-assisted review checked and expanded these sets. Of the 100 primary inputs, 69 have a recorded positive human check of the current reference-line selection, 55 of them on the exact current source text and labels; dossier approval and reference-line review are separate checks (Appendix C).

Scoring and uncertainty. Hit@1 counts a match in the first candidate slot, and Hit@3 a match in any of the original first three slots. An invalid, unknown, blank-line, or duplicate label leaves its slot empty, and a later candidate never moves forward to fill it. Missing answers and attempts that break the execution rules or exhaust their budget count as misses. We report these all-input rates together with the number of responses that met the rules; rates conditional on those responses have denominators that differ across configurations.

Four deterministic rules choose the longest line, the line with the most words, the line with the most mathematical symbols, or the line most similar to the document's TF–IDF word profile; the comparison baseline is the strongest of the four on the evaluated population, and random line selection is reported as its exact expectation.

Confidence intervals use 10,000 bootstrap resamples of problem clusters, keeping related texts together within the assignment groups specified in the released scorer (Efron, 1979; Field & Welsh, 2007); the comparison with the strongest rule reselects that rule in each resample. These intervals describe variation across the evaluated texts. The saved parser, scoring code, grouping, and seed settings are in `benchmark/code/`.

Saved responses and reproduction. The release preserves the selected responses, failure categories, and every execution attempt; recorded tokens, lower bounds for incomplete logging, and budget charges are kept as separate quantities. The settings below explain how an attempt is run, and `benchmark/core/README.md` reproduces the current scores from the saved responses.

F.1 Complete current results

Table 16 reports all 13 evaluated configurations on the current 100-input primary set, including both saved Haiku views. It gives intervals for both scores and recorded token components. The two Haiku views share calls, so their costs are not added together. The complete 131-input results and failure counts are in `benchmark/core/expected_scores.json`.

F.2 Evaluated models and execution settings

We evaluated 13 model configurations, five hosted models and eight locally run open-weight models. Haiku has two reporting views of the same calls, an outage-continuation view and a retained-notice view, so the release contains 14 reporting views for 13 configurations; their shared calls and costs are counted once in campaign accounting.

Each repository identifier in Table 17 links to its public model card; hosted model names are the preserved experiment identifiers.

Critic or baseline	Hit@1 (%) [95% CI]	Hit@3 (%) [95% CI]	Recorded total tokens	Recorded output tokens
Haiku 4.5 (retained notice)	31.0 [22.7, 39.8]	40.0 [30.9, 49.0]	632,500	393,211
Sonnet 5	61.0 [52.0, 69.6]	76.0 [67.7, 83.7]	697,281	218,376
Opus 5.5	68.0 [59.8, 76.2]	85.0 [78.6, 91.0]	510,187	78,546
DeepSeek R1 Qwen	22.0 [14.4, 30.1]	28.0 [20.0, 36.4]	428,199	215,305
Gemma 3 27B	26.0 [18.3, 33.7]	44.0 [35.7, 52.1]	340,082	17,552
GLM 4.5 Air	10.0 [4.9, 16.0]	11.0 [5.8, 17.2]	1,658,280	1,357,252
GPT-OSS 120B	49.0 [39.6, 58.3]	60.0 [51.0, 69.0]	450,420	150,198
GPT-OSS 20B	43.0 [34.3, 52.0]	47.0 [37.8, 56.6]	708,462	408,240
Kimi Linear	14.0 [7.2, 21.4]	27.0 [19.0, 35.7]	241,035	46,849
Nemotron 3.5 Lightning	27.0 [19.0, 35.6]	27.0 [18.8, 35.4]	1,676,079	1,361,911
Nemotron 3 Super	48.0 [40.2, 56.1]	48.0 [40.2, 56.0]	740,935	426,767
Gemini 3.1 Pro	73.0 [65.1, 80.6]	85.0 [78.2, 91.2]	1,446,420	556,279
GPT-5.6 Luna	63.0 [54.6, 71.1]	74.0 [66.7, 80.8]	390,142	98,876
Haiku 4.5 (continuation)	56.0 [46.9, 65.1]	69.0 [61.2, 76.5]	1,053,443	658,433
TF–IDF centroid	18.0 [11.0, 25.3]	—	—	—
Random line	6.7 [5.7, 7.8]	—	—	—

Table 16: Current primary results on 100 inputs from 92 problem clusters. Hit@3 counts a reference match among the first three candidate slots; the one-line baselines have no top-three result. The token columns report recorded usage for the selected primary attempts and are lower bounds where logging is incomplete. The public score file identifies complete totals and missing usage for each configuration.

Table 17: Local model configurations and exact checkpoint repositories.

Model Label	**Repository**
DeepSeek-R1-Distill-Qwen-32B	`deepseek-ai/DeepSeek-R1-Distill-Qwen-32B`
Gemma 3 27B-IT (greedy)	`unsloth/gemma-3-27b-it`
GLM-4.5-Air (FP8)	`zai-org/GLM-4.5-Air-FP8` (FP8 compressed-tensors)
GPT-OSS 120B (MXFP4, medium)	`openai/gpt-oss-120b`
GPT-OSS 20B (MXFP4, medium)	`openai/gpt-oss-20b`
Kimi-Linear 48B-A3B Instruct	`moonshotai/Kimi-Linear-48B-A3B-Instruct` (configuration dtype: BF16)
Nemotron 3.5 Lightning 30B-A3B (BF16)	`nvidia/NVIDIA-Nemotron-3.5-Lightning-30B-A3B-BF16` (BF16)
Nemotron 3 Super 120B-A12B (FP8)	`nvidia/NVIDIA-Nemotron-3-Super-120B-A12B-FP8` (ModelOpt FP8)

The GPT-OSS configurations use MXFP4 weights, medium reasoning effort, and instructions in the native Harmony developer block. All eight local configurations requested one B200 GPU and used `vLLM` 0.29.0 (Kwon et al., 2023).

Inputs that exceed the recorded context limit receive a setup refusal and count as misses. The five hosted configurations are `claude-sonnet-5`, `claude-opus-5-5`, `claude-haiku-4-5-20251001`, `gemini-3.1-pro`, and `gpt-5.6-luna`, queried through their command-line interfaces (Claude Code CLI, the agy CLI for Gemini, and Codex CLI for Luna).

Prompts and context. All configurations receive the same numbered text and benchmark instructions. Local models use their native chat templates, GPT-OSS places the instructions in the Harmony developer block, and Gemini places them at the start of the user turn. For local models, the prompt plus the 16,000-token output allowance must fit both the configured context and the 120,000-token attempt ceiling; longer inputs receive a recorded setup refusal and a score of zero, without truncation or a model call. The hosted runs preserve the CLI settings and receipts, whose saved configuration does not specify a numeric context-window limit.

Model	Context tokens	Generation setting	Configured execution interface
Haiku 4.5	n.r.	Medium effort	Claude CLI 2.1.275
Sonnet 5	n.r.	Medium effort	Claude CLI 2.1.275
Opus 5.5	n.r.	Medium effort	Claude CLI 2.1.280
DeepSeek R1 Qwen	32,768	$T = 0.6$, $p = 0.95$	Local vLLM
Gemma 3 27B	65,536	Greedy	Local vLLM
GLM 4.5 Air	65,536	$T = 0.6$, $p = 0.95$; thinking	Local vLLM
GPT-OSS 120B	131,072	$T = 0.6$, $p = 0.95$; medium	Local vLLM
GPT-OSS 20B	131,072	$T = 0.6$, $p = 0.95$; medium	Local vLLM
Kimi Linear	32,768	$T = 0.6$, $p = 0.95$	Local vLLM
Nemotron 3.5 Lightning	131,072	$T = 1.0$, $p = 0.95$; thinking	Local vLLM
Nemotron 3 Super	65,536	$T = 1.0$, $p = 0.95$; thinking	Local vLLM
Gemini 3.1 Pro	n.r.	Low effort	agy 1.2.8
GPT-5.6 Luna	n.r.	Medium effort	Codex CLI 0.155.1

Table 18: Saved execution settings for the 13 model configurations. Every attempt has a 16,000-token output allowance and a 600-second wall limit, with the interface-specific enforcement described below. T is temperature and p is top-p; local generation uses vLLM 0.29.0 and seed 20260921. "n.r." means the hosted CLI's numeric context-window limit was not recorded. Context limits, output-budget enforcement and recovery rules are specified below. Available version receipts agree across the initial, correction and recovery passes.

Output and time limits. Each attempt has a 600-second wall limit and a 120,000-token attributable-usage ceiling, and the common 16,000-token output allowance is enforced differently across interfaces. Local vLLM generation sets `max_tokens=16000`, including generated reasoning tokens, and reaching that limit counts as budget exhaustion. Claude requests a 16,000-token client limit and checks the event stream for continuations, output usage, and live estimates. Gemini and Luna have no provider-side output cap in these saved interfaces: their recorded native output usage is checked after generation, and a completed response above the allowance counts as budget exhaustion, with Gemini's output count including its recorded thinking tokens. The 120,000-token ceiling is checked during the Claude and Gemini streams and after receipt of Luna's native usage. Because of these interface differences, the shared allowances do not imply matched computation.

Tools, retries and recovery. Claude and Luna disable tools, and local generation has no tool executor. Gemini's CLI exposes tools, so the runner stops and rejects an attempt if a tool step or an additional turn appears. Ordinary runs and the input-correction pass allow one initial attempt and one retry for an eligible transport failure; responses that exhaust their budget or break the execution rules are terminal under that allowance. The separate Haiku continuation resumes the listed no-answer session-limit notices and carries forward completed answers and budget failures. The later environment-recovery pass grants one attempt per listed target, with no automatic retry, and its result replaces the notice regardless of outcome. Both Haiku reporting views use the same corrected and recovered outputs, and every attempt remains in campaign accounting.

Usage and budget charges. Recorded usage, accounting charges, and execution limits are distinct quantities. Complete native usage contributes its measured count, partial native usage contributes its recorded lower bound, and an attempt with incomplete usage receives a conservative accounting charge of at least 40,000 tokens, or a larger recorded lower bound or applicable reservation. A charge is a budget debit, not measured consumption. The results table reports selected-response usage; `benchmark/accounting/README.md` gives the full accounting rules and attempt ledger, and `benchmark/code/generation/README.md` documents the generation drivers, prompts, and settings.

F.3 Reference-line checks and review exclusions

Human review checked whether the accepted lines state the challenged claim. The initial 40-input check approved 35 line selections; after corrections, a fresh, disjoint sample approved 38 of 40 and met the review criterion. Appendix C gives the full review process and dispositions.

The current 100-input primary set and 131-input scorable set exclude every record with a retained negative or uncertain source, witness, or formalization review, and all three records with adverse judgments on their current line selections. Reapplying these exclusion lists leaves the current sets and scores unchanged. Appendix C records the individual decisions; `bugbank/reviews/version_resolution/README.md` supplies the complete exclusion lists and saved-answer sensitivity. Reference-line revisions and their replay are documented in `benchmark/README.md`.

F.4 Examining first-choice misses

A random sample of 30 was drawn from Sonnet 5's 58 readable first-choice misses in the earlier 191-input evaluation, using seed 20260921. LLM reviewers, with access to the source text and reference labels, classified the selected claims and assessed the accompanying explanations.

The final review identified 12 selections of another false assertion, 15 lines that were not false as written, and three source or annotation problems. The first LLM reviewer approved the supplied explanation in 11 of the 12 alternative-assertion cases. The three source or annotation problems were two omitted restatements of the challenged proposition and one garbled source.

The released records preserve the original classifications, later adjudications, sampling intervals, and disagreements on M08, M13, and M20. This analysis retained the original benchmark scores. The sampled responses and scientific reviews are in `bugbank/reviews/sonnet_misses/`; `bugbank/reviews/README.md` explains their numerical reproduction.

G What the criticism scores measure

This appendix distinguishes three questions relevant to interpreting the scores in Section 4. They are whether a formal counterexample refutes the source claim, whether a critic selected a false assertion, and whether the critic's explanation is valid. The propositions are elementary, and we developed their written proofs with AI tools; they are separate from the Lean-certified mathematics elsewhere in the paper. The worked comparison uses the saved reviews of 24 texts from an earlier evaluation set.

G.1 The logical connection from a formal witness to a source claim

Section 2 defines the source assertion Q_s, the formal witness, and its interpretation e, and Equations (1)–(3) state the source-correspondence requirements. The proposition below is the implication used there.

Proposition 4 (One-sided refutation transport)**.** *If Equation* (2) *and the witness-level implications in Equation* (3) *hold, then* $A_s(e(w)) \wedge \neg C_s(e(w))$*, so the source assertion* Q_s *is false.*

Proof. The first implication gives the source assumptions. If the source conclusion were true, the second implication would make the encoded conclusion true, contradicting the certificate. Hence the source conclusion is false at an allowed source object. □

Full equivalence between the formal and source statements is one way to satisfy these conditions, not the only one. The sorting example in Section C.3 illustrates the first condition: the witness must satisfy the source's endpoint inequality. Writing down X_s, A_s, C_s, and e makes the interpretation a reviewer must judge explicit.

G.2 Secondary reviews of generated objections

The Sonnet miss analysis in Section F.4 shows that a response can miss the designated reference line and still identify another false assertion. The study below checks, for each saved objection, whether it identifies its target and whether its evidence is valid.

Experiment B: LLM review of saved responses from an earlier evaluation set. Experiment B examined 48 saved responses, one from Gemini 3.1 Pro and one from Nemotron 3 Super 120B for each of 24 sampled texts of the earlier evaluation set. Two LLM reviewers separately read

anonymized packets and judged whether each explanation challenged the selected claim (J1) and whether its mathematical evidence was valid (J3).

The question is whether a response both identifies the assertion it challenges and supplies valid evidence under that assertion's assumptions. Table 19 reports this joint endpoint after the recorded LLM adjudications and the checks that each objection applies to the selected source claim under its assumptions; the first-reader judgments designated in advance are kept below. Six of the 10 reference misses supplied valid objections under the joint criterion.

Model	Reference matches	Valid obj. among matches	Valid obj. among misses	Total valid objections
Gemini 3.1 Pro	20	18	2	20 / 24
Nemotron 3 Super 120B	18	15	4	19 / 24

Table 19: LLM review of joint target identification and evidence validity on 24 paired texts. A counted valid objection is a response that met the rules, challenges its selected claim, and supplies mathematically valid evidence under the source assumptions, after the recorded adjudications and the checks that the objection applies to the selected source claim. Table 20 gives the first-reader judgments and the counts across the recorded review stages.

Detailed judgments. The upper panel of Table 20 gives the first-reader evidence-validity judgments (J3) designated in advance; the lower panel shows the joint positive-evidence counts across the recorded review stages: the first designated LLM reviewer alone, agreement of both reviewers, inclusion of the recorded adjudications, and finally the source-assumption checks.

First-reader evidence-validity judgments (J3) for the 48 saved responses

Model	First-choice outcome	Valid	Invalid	Unresolved
Gemini 3.1 Pro	Hit@1	18	0	2
Gemini 3.1 Pro	Miss	3	1	0
Nemotron 3 Super 120B	Hit@1	16	2	0
Nemotron 3 Super 120B	Miss	5	0	1

Responses meeting the rules that satisfy both J1 and J3

Review stage	Gemini 3.1 Pro	Nemotron 3 Super 120B
First reader	18 / 24	19 / 24
Both readers	18 / 24	18 / 24
After LLM adjudication	20 / 24	20 / 24
With source-assumption check	20 / 24	19 / 24

Table 20: Experiment B in detail. Upper panel: the first-reader evidence-validity judgments (J3), by whether the response's first choice was a reference line. Lower panel: the number of responses meeting the rules that satisfy both target identification (J1) and evidence validity (J3) across the recorded review stages (ending with the final joint endpoint after the source-assumption check).

A third LLM reviewer examined eight flagged responses. P06 contained a complete explanation, although the attempt exceeded the allowed budget. P43 supplied valid mathematical evidence but cited the wrong line. P35 challenged a stronger condition than the selected statement claimed, so its counterexample did not address that claim. For P31, the calculation refuted the implication stated in the model's explanation, while the supplied example violated two assumptions in the source. Applying the study's rule that counterexamples must satisfy the source assumptions classifies P31's evidence as invalid.

G.3 Partial labels and generated-objection soundness

A reference-selection score measures agreement with the designated target. Two broader outcomes are finding any false assertion and supplying a valid counterexample for it. Because the reference labels cover selected assertions only and the score does not assess explanations, the available data bound these outcomes without determining them, a partial identification problem (Manski, 2003).

In this subsection N is the number of benchmark inputs and all sums over i run from one to N. For item i, let F_i be the set of admissible line labels that locate genuinely false assertions under the source assumptions (a label may identify part of a displayed proposition); this set describes the truth of the assertions and need not be fully annotated. Let G_i be the set of reference lines and V_i the indicator that the response met the rules. For a first-choice label ℓ_i, define

$$h = \frac{1}{N}\sum_i V_i \mathbf{1}\{\ell_i \in G_i\}, \quad q = \frac{1}{N}\sum_i V_i \mathbf{1}\{\ell_i \in F_i\}, \quad c = \frac{1}{N}\sum_i V_i.$$

The fractions h and q count responses that meet the rules and select, respectively, a reference line or any genuinely false assertion; c counts all responses that meet the rules. All three use the full N-input denominator. Unlike h and c, q is not observable in general, because the full sets F_i are not annotated. Null labels belong to neither set.

Proposition 5 (Partial identification under sound references)**.** *If $G_i \subseteq F_i$ for every item, then $h \le q \le c$. If the inclusion may fail on at most m items, then*

$$\max(0, h - m/N) \le q \le c.$$

Let E_i indicate that the critic's supplied objection is valid for its selected target, and define $r = N^{-1}\sum_i V_i \mathbf{1}\{\ell_i \in F_i\} E_i$. Without evidence about E_i, no strictly positive lower bound on r follows from h alone.

Proof. On a sound item, reference membership implies false-assertion membership. At most m reference hits can fail that implication on the remaining items. No valid target can be credited without a valid output, giving the upper bound c. Finally, every E_i can be zero even when every chosen target matches its reference set: a critic may attach an invalid witness to the correct line. Hence $r = 0$ is compatible with positive or perfect reference localization. □

The same argument applies to top-k selection, with explanation validity assessed separately. When the designated references are sound, matching one is evidence that the critic selected a false assertion, and a miss can still identify a different false assertion, as the diagnostic in Section F.4 shows; estimating q or r requires the corresponding target and explanation judgments.

G.4 Comparing critics with partially reviewed line selections

If two critics select the same line, its truth status affects both scores equally, so only the selections on which they disagree can change their score difference. The following result turns that observation into exact bounds.

Set a critic's first-choice label to $\perp$ when its response does not meet the rules or its first-choice label is invalid, so that critic a or b supplies $p_i^a, p_i^b \in L_i \cup \{\perp\}$, where L_i is the addressable line set. Write $z_{i\ell} = \mathbf{1}\{\ell \in F_i\}$ and $z_{i\perp} = 0$. A line reviewed as false under the source assumptions has value one; a line reviewed as not false has value zero. Every other indicator is unknown. A reference label has value one only when its validity under the source assumptions is assumed or established.

For critic $m \in \{a, b\}$, let $\underline{z}_i^m, \overline{z}_i^m$ be the lower and upper possible values of z_{i,p_i^m}. They are $(1, 1)$ for known false assertions, $(0, 0)$ for known nonfalse or null predictions, and $(0, 1)$ for unadjudicated lines. Indicators of the same line on the same item are shared, not assigned separately to each critic.

Proposition 6 (Sharp comparison in the line-label model)**.** *Over assignments of unknown line indicators consistent with these known labels and with no further constraints, the false-assertion localization difference*

$$\Delta_F = \frac{1}{N}\sum_i \big(z_{i,p_i^a} - z_{i,p_i^b}\big)$$

has sharp lower and upper bounds

$$L = \frac{1}{N}\sum_{i:p_i^a \neq p_i^b} (\underline{z}_i^a - \overline{z}_i^b), \qquad U = \frac{1}{N}\sum_{i:p_i^a \neq p_i^b} (\overline{z}_i^a - \underline{z}_i^b). \tag{5}$$

Both endpoints are attained in this model. "Sharp" means that no narrower interval follows from these labels alone under the stated absence of further constraints.

Proof. Coincident predictions share an indicator, so their contribution is exactly zero even when it is unknown. For distinct predictions the difference is at least $\underline{z}_i^a - \overline{z}_i^b$ and at most $\overline{z}_i^a - \underline{z}_i^b$. To attain the lower bound, set each unknown indicator selected by a to zero and each selected by b to one on disagreement items; reverse these assignments for the upper bound. Distinct selected lines impose no conflicting assignment within an item, and this model imposes no cross-item constraints. Thus the extremal choices can be made simultaneously. □

Which additional reviews narrow the comparison. The identification width is

$$U - L = \frac{1}{N} \sum_{i:p_i^a \neq p_i^b} \left[(\overline{z}_i^a - \underline{z}_i^a) + (\overline{z}_i^b - \underline{z}_i^b) \right].$$

Only unknown predictions on disagreement items contribute: one unknown line gives width $1/N$, two give $2/N$, and a shared unknown prediction gives zero. Reviewing one such line narrows the interval whichever label it receives, so the disagreements identify the additional source judgments that matter most for the paired comparison.

The bounds are exact for the stated model, which permits every assignment consistent with the known line labels; logical relationships among assertions or repeated sources can narrow them further. An identification interval describes the possible score differences while some labels are unknown, whereas a sampling confidence interval describes uncertainty from sampling texts, and reviewing explanation validity answers a third question, whether the critic supplies valid evidence for its chosen assertion. Applying the bounds requires item-level predictions and judgments.

Numerical application on 24 texts. The proposition concerns which selected assertions are false, not whether the critic's own explanation proves them false. Using the saved judgments and source checks, we assign one shared truth indicator to each $(\text{input}, \text{line})$ pair: a valid objection can establish a positive line label, while an invalid explanation alone does not establish a negative one.

Gemini and Nemotron make the same normalized prediction on 15 of 24 inputs; their contribution is exactly zero regardless of the line's status. They disagree on nine inputs. Let z_{19} and z_{43} be the unknown false-assertion indicators for Nemotron's P19 line 24 and P43 line 230. The seven other disagreement contributions sum to zero based on the accepted labels or null predictions. These two contribute $-z_{19}$ and $1 - z_{43}$. Thus the Gemini-minus-Nemotron score difference is $(1 - z_{19} - z_{43})/24$, between $-1/24$ and $1/24$.

Positive line labels are accepted conditionally from the saved LLM judgments, all other non-null labels stay unknown, the references are not assumed sound, and no negative label is inferred from invalid evidence. These are the 24 paired texts of Experiment B, separate from the current 100-input benchmark. The interval $[-1/24, 1/24]$ is conditional on the recorded LLM judgments and summarizes the two unknown labels; it is not a 95% confidence interval. The saved inputs, judgments, and reproducible calculation are documented in `studies/criticism_endpoints/README.md`.

G.5 Sensitivity to revised reference lines

Let k be the number of candidate slots scored. Fix P_i^k, the set of valid line labels in the original first k slots for input i; null slots remain unfilled and later predictions are not promoted. Write $G_i \triangle G_i'$ for labels that belong to exactly one of the old and new reference sets. Let G_i, G_i' be old and revised reference sets and $r = |\{i : G_i \neq G_i'\}|$.

Proposition 7 (Prediction-aware reference perturbation)**.** *For fixed outputs and a fixed N-item population,*

$$|\mathrm{Hit}_G\ @k - \mathrm{Hit}_{G'}\ @k| \leq \rho(k) := \frac{1}{N} \sum_i \mathbf{1}\{P_i^k \cap (G_i \triangle G_i') \neq \varnothing\} \leq \frac{r}{N}. \tag{6}$$

For two critics a, b, their paired score difference changes by at most $\rho_a(k) + \rho_b(k) \leq 2r/N$.

Proof. An item cannot change hit status unless a selected label lies in the symmetric difference $G_i \triangle G_i'$. Even then, a retained reference hit may keep its status unchanged. Each binary hit changes

by at most one, so summing gives the first inequality. Only items whose reference set changed can contribute to ρ, giving the second. Applying the triangle inequality to two critics yields the paired bound. The coarse constants are attainable: a changed reference can turn one miss into a hit, or exchange reference membership between two distinct first-choice predictions, changing their difference by two. □

The prediction-aware bound counts outcomes that could change, not net changes: a gained hit and a lost hit can cancel in an aggregate score, and an edit away from every selected line cannot affect the score at all. Counting changed reference sets alone can therefore overstate the possible effect of a revision.

The bound applies to any recorded revision of the checked line sets, and rescoring the saved predictions against both versions gives the actual change. If m excluded inputs are misses for every reporting view, each rate changes from H/N to $H/(N-m)$ while the hit count H stays fixed. The release provides the line sets, exclusion lists, and per-input predictions for these calculations.

G.6 Why paired comparisons require individual outcomes

Suppose two models have a and b hits on the same N items, with $a \geq b$. Their common-hit count t can lie anywhere in

$$\max(0, a + b - N) \leq t \leq b.$$

The discordant counts are $a - t$ and $b - t$, so a paired test needs more than the marginals. In the current 100-input results, Gemini has 73 first-choice matches and Opus 68; these totals allow t anywhere from 41 to 68, that is, between 5 and 59 inputs on which exactly one model succeeds. The released per-input predictions determine the actual overlap. The main text therefore reports the highest observed scores, and paired comparisons use the item-level outcomes.

G.7 Costs with incomplete usage records

Let the total nonnegative costs in two conditions be $C_A = L_A + U_A$ and $C_B = L_B + U_B$, where L_A, L_B are recorded subtotals and $U_A, U_B \geq 0$ unrecorded usage. For positive denominators, L_A/L_B bounds C_A/C_B in neither direction in general, because either unrecorded remainder can be the larger one. A budget charge is an accounting debit, not a measurement of that remainder, and a cost total over selected successful attempts describes a different set of executions from one that includes failures and retries. We therefore name the included calls and report measured usage, lower bounds, and budget charges separately.

H From a counterexample to a complete explanation: thirteen follow-up packages

This study followed every dossier in the earlier 285-dossier collection whose annotations described both a failure family and a repair: thirteen in all. The square-root result had already been formalized; for the other twelve, human contributors and coding agents developed formal proofs, followed by separate adversarial model reviews. All thirteen packages contain Lean-checked family and repair statements with explicit assumptions.

Table 21 lists all thirteen. Ten of the source dossiers are in the 147-dossier human-approved core; the square-root, array-assignment, and recurrence-sum dossiers are outside it. The packages are mathematical developments of the selected examples, each with its own review record and scope (Appendix C).

Three outcomes. The packages illustrate three outcomes, which can overlap. One is a complete classification of failures: the square-root package proves that the original algorithm fails exactly at the positive perfect squares and, separately, that the corrected algorithm is correct on every allowed input. Another is a corrected local statement: in the signed-cut example, at the source-compatible values $n = 3$, $T = 2$, $d = 20$, $f = -1$, and $a = 19$, the source's total $T + f$ is 1 while the correct additive total $T + a - 2(n-2)d$ is -19. Because the surrounding argument needs a positive total, the package also proves an exact positivity condition (Lean statements `repair` and `exact_positivity`).

Record / topic	What the formal result establishes	Assumptions and scope
CS 97637/a97647 Square root	Failure iff the input is a positive square; strict-guard repair satisfies the specification for every natural input.	Completed before the other twelve packages; proves the corrected algorithm.
CS 1200/a1206 Array assignment	Every $2 :: 3 :: r$ reaches $2 :: 2 :: r$; repaired postcondition equals the reachable-state set.	A local program statement, with total correctness under the stated assignment rules.
CS 13118/a13122 Geometric sum	Correct sum is $(c^n - c)/(c - 1)$; exact offset and validity conditions are proved.	$n \geq 1$, $c \neq 1$ over a field; ordering is a separate real statement.
CS 156543 Liveness	Every finite prefix has a satisfying extension (applying the standard liveness framework (Alpern & Schneider, 1985)); a later commit exactly characterizes repair at a violated request.	The equivalence keeps the violated request in the fixed prefix.
CS 1957/a2388 Recurrence sum	Correct sum uses $\sum_{i=1}^{L} i^k$; the asserted equality holds iff $k = 0$ or $L \leq 1$.	Exact powers $n = b^L$, $b > 1$, $a > 0$, $k, L \in \mathbb{N}$; explicit Θ bounds.
TCS 2312/a2318 Signed cut	Total is $T + a - 2(n - 2)d$; exact conditions distinguish accounting error, negativity and loss of positivity.	Distinct vertices and additive edges; positivity has a separate condition.
Math 1291495 Cycle deletion	For a finite connected graph, terminal deletion outcomes are exactly its spanning trees; corrected edge counts recover the target tree theorem.	Delete one cycle edge at a time; the cyclic source premise requires $k \geq 2$.
Math 1868681 Bipartite edges	Maximum is $\lfloor p^2/4 \rfloor$; the rational value $p^2/4$ is attained iff p is even.	Distinguishes the exact maximum from the valid upper bound $p^2/4$.
Math 782216 Handshaking induction	The attempted step holds iff the graph has an isolated K_2 component; it fails for every connected graph with at least two edges. The zero-edge base case is also characterized.	Classifies the proposed induction step; the base case depends on the empty-graph convention.
Math 994847 Adjacency powers	$(I + A)^{n-1}_{ij} \neq 0$ iff i, j are reachable; a bipartite obstruction family has certified slack.	The full family includes diagonal entries; the source's off-diagonal assertion requires $n \geq 3$.
ProofWiki 25056 Polynomial content	$\operatorname{cont}(ag) = \lvert a \rvert \operatorname{cont}(g)$, with exact signed-form validity conditions.	Integer form; rational positive content needs a zero-polynomial convention.
Reddit 1bq1jtc Prime residues	$6q + 2 = 2(3q + 1)$, uniquely in the affine template; the intended prime-residue conclusion is also proved.	Corrects the local factorization and proves the target classification.
Reddit 8yjpp7 Recursive increment	Repaired expression equals $n + 1$ iff n is odd; the recursive program returns $n + 1$ for all n.	The source branch supplies oddness; the result extends the recorded $4k + 1$ family to every odd input.

Table 21: All thirteen mathematical follow-ups in the selected explanation study. The middle column states the formal result, and the last column gives the assumptions needed to apply it. The artifact index maps each abbreviated source identifier to its full record ID and selected Lean declarations.

The third is a proof of the intended final statement after the faulty step is replaced, which the cycle-deletion, prime-residue, and recursive-increment packages supply. Proving a family or repair statement is not the same as proving the source's conclusion; the middle column of the table says which was done.

Assumptions matter. The adjacency-power family includes diagonal matrix entries, while applying it to the source's off-diagonal assertion needs the narrower family with $n \geq 3$; in the liveness example, the equivalence fixes a prefix that includes the violated request, and letting the prefix end earlier changes the statement. The saved model reviews record these distinctions next to the formal proofs.

Formal proofs and reproduction. `mathematics/extensions/index.json` maps the statements to their Lean sources and recorded checks: the twelve later packages are in `mathematics/explanations/`, and the square-root proof is in `mathematics/extensions/sqrt_boundary/SqrtFailureBoundary.lean` with its repair proof in `mathematics/extensions/sqrt_repair/`. Files using only Lean's libraries have recorded checks under Lean 4.30.0 and 4.34.0, and files using Mathlib under Lean 4.30.0 with the pinned Mathlib project; the square-root package also has a separately written second proof and reproduction record, which the index distinguishes from the adversarial model reviews of the other twelve. `bugbank/lean/README.md` gives replay instructions.

I GRAPH COLORINGS AND RAINBOW SCHUR TRIPLES: STATEMENTS, PROVENANCE, AND PROOFS

This appendix states the two family theorems of Section 5 in full, records where their witnesses and certificates come from, and proves them. Both witnesses refute published conjectures. They were found on 4 September 2026 by `gpt-5.6-sol` at maximum reasoning effort, in runs that asked the model to prove or refute the statements without access to BugBank, and were accepted by separate model checks. They entered the research-discovery collection on 10 September, apart from the 285-dossier construction stage and the human-approved core. The Schur witness already supplied an infinite family with exact counts, and the follow-up optimizes the larger two-threshold family it belongs to and measures the loss from moving the thresholds. The graph witness was one cycle blowup, and the follow-up classifies the odd cycle lengths and counts all frozen six-colorings. We developed the family theorems and their proofs with the help of AI tools, and report them separately from the benchmark's dossier and input counts.

Graph certificates. Three theorems cover the original witness, the refutation of the source conjecture, and the family. The file `mathematics/supporting/sources/C9K2_frozen_coloring.lean` defines the adjacency of $C_9[K_2]$ and explicit five- and six-colorings; its theorem `C9K2.frozen_six_colouring_certificate` checks a proper six-coloring in which no vertex can change color, a proper coloring with at most five colors, and a second proper six-coloring, so the first six-coloring is isolated although another proper six-coloring exists. The theorem `Recol.not_conjecture1` in `mathematics/supporting/sources/PrimeBlowup.lean` supplies what a refutation of the conjecture needs beyond the obstruction: it checks that C_9 satisfies the conjecture's hypotheses, using a degeneracy lemma (Cereceda et al., 2008) that covers arbitrarily many colors and shows every induced subgraph of C_9 recolorable, and it refutes the conjecture. The theorem `Recol.cycle_blowup_frozen` in `mathematics/theorems/CycleBlowupAllN.lean`, which reuses the general lemmas of `PrimeBlowup.lean` verbatim, establishes Proposition 2 for every odd $n \geq 5$: the chromatic number, the classification, and the exact number of isolated six-colorings, which gives only a lower bound on the number of recoloring components. The later AllOdd and Q_3 certificates have their own formal scopes, stated below; combining those graph statements with the source conjecture's hypotheses is a written argument, not one additional Lean theorem.

Schur certificates. The proof in `mathematics/supporting/sources/RainbowSchur.lean` counts the rainbow triples of the displayed coloring of $[22m]$ for every positive integer m, ordered and normalized by $\binom{n}{2}$, and uses the count to contradict the proposed asymptotic bound; this lower bound comes from one explicit family, and optimizing over all colorings is a broader problem.

The composed certificate `mathematics/theorems/SchurTemplateComposed.lean` contains the two-threshold result of Proposition 1 and its connection to the literal coloring; `mathematics/builds/schur.json` records the intermediate proofs and compilation evidence.

Reviews and build evidence. Each proposition below is proved at its full stated scope by a Lean theorem using Mathlib under Lean 4.30.0. The finite $C_9[K_2]$ certificate has recorded builds under Lean 4.30.0 and 4.34.0 using only `propext` and `Quot.sound`. Build evidence for the finite example and the conjecture refutation is indexed in `mathematics/supporting/index.json`, and the family theorem's in `mathematics/builds/cycle_k2.json`. A second model instance checked each certificate adversarially and found no error; human reviews of the two extensions (C01 and S02) and of later formalizations are reported in Appendix C. The AllOdd and Q_3 certificates were checked by models without a further human-review round.

I.1 Rainbow Schur triples: improving and optimizing a coloring

Section 5 defines ordered Schur triples and rainbow colorings of $[n] = \{1, \dots, n\}$; the colors are red, blue, and green, or $1, 2, 3$, and integers with the same color form a *color class*.

We count ordered triples, so $(2, 3, 5)$ and $(3, 2, 5)$ count separately. The extremal problem asks how large a fraction of these triples a three-coloring can make rainbow as n grows.

In *An Unsure Note on an Un-Schur Problem*, Parczyk & Spiegel (2026) give a construction with limiting density $2/5$ and conjecture, immediately after their Theorem 1, that this is optimal. Their construction gives all even integers one color and splits the odd integers into two contiguous color classes.

The recorded counterexample improves that construction by assigning one color to both an initial and a final block of odd integers. For $n = 22m$, all evens have color 1, odds at most $8m$ or greater than $20m$ have color 2, and the remaining odds have color 3. The original refutation proves that this gives exactly $99m^2 + 3m$ rainbow ordered Schur triples for every positive integer m, with limiting density $9/22 > 2/5$. It therefore disproves the conjectured optimum by improving the constructive lower bound for the original extremal problem.

A preprint submitted on 16 September 2026 by Hegde et al. (2026) gives the same two-threshold construction and $9/22$ lower bound, together with a global upper bound of $8/15$; our saved model run is dated 4 September 2026. The two dates document the two chronologies, and we leave mathematical priority unassigned. The proposition below concerns the unique optimum and stability within the two-threshold family, not global optimality.

The split odd color class suggests a two-parameter search: move its two block boundaries and ask which placement works best. The following proposition turns that search into an exact description of the entire template. Here a *template* means this fixed rule for assigning colors, with only the two boundaries allowed to vary.

Proposition 1 (Optimal Schur thresholds within a template)**.** *For $0 \le a \le b \le 1$, color every even integer in $[n]$ color 1, and each odd integer k color 2 when $k/n \le a$ or $k/n > b$, and color 3 otherwise. The limiting rainbow ordered-Schur-triple fraction $D(a, b)$, normalized by $\binom{n}{2}$, has unique maximum $9/22$ at $(a, b) = (4/11, 10/11)$. On $a \le 1/2 \le b$ and $a + b \ge 1$, writing $u = a - 4/11$ and $v = b - 10/11$ gives*

$$D(a, b) = \frac{9}{22} - \frac{5}{2}u^2 + 2uv - \frac{3}{2}v^2. \tag{7}$$

The two contiguous odd-block subfamilies, $a = 0$ or $b = 1$, attain maximum $2/5$.

The public certificate `mathematics/theorems/SchurTemplateComposed.lean` combines the counting and optimization arguments. Its declarations `template_count` and `rational_count` give exact counts for integer thresholds and for $n = 2qm$ with rational thresholds in the stability region. The declaration `paper_limit` proves the limit for every real $0 \le a \le b \le 1$ and the literal coloring in the proposition. At odd n, two-sided counting bounds suffice to establish this limit.

Within the same file, namespace `CoPEER.SchurTemplate` contains `unique_global_optimum`, `left_boundary_optimum`, and `right_boundary_optimum`. The theorem

`sharp_stability` gives $9/22 - D \geq (4-\sqrt{5})(u^2+v^2)/2$ in the stated region. The declarations `quadratic_spectral_equality_iff` and `quadratic_spectral_coefficient_optimal` prove that equality holds exactly when $v = (1+\sqrt{5})u/2$ and that the coefficient is best possible for this quadratic form. Thus the certificate links the finite colorings to their limiting density and then proves its exact optimum. The written argument appears in Section I.3.1.

Every pair of differently colored odd integers supplies two rainbow triples with one even and one odd addend: their positive difference is even, and it can appear in either addend position. Two odd addends contribute only when their sum stays in $[n]$. Balancing these contributions gives the recorded thresholds: a final color-2 block of normalized width $1/11$ raises the best template density by $1/110$, or 2.27% relative to $2/5$. The quadratic identity gives the exact loss from moving the two thresholds within the stated region; every nonzero displacement reduces the limiting density. This explains why separating the odd color class into two blocks helps, and why the recorded boundaries are optimal within this template (Figure 4).

A better density requires a different coloring rule; finding the best unrestricted three-coloring remains the broader extremal question.

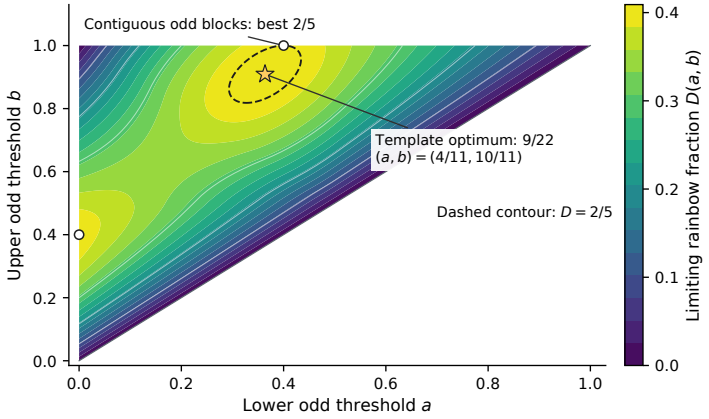


Figure 4: The two-threshold parity template has unique optimum $9/22$ at $(a, b) = (4/11, 10/11)$, compared with $2/5$ for a single contiguous odd block. The analytic proof establishes the optimum throughout the stated domain; the plot illustrates its dependence on the thresholds (generated using NumPy (Harris et al., 2020)).

I.2 Cycle blowups: explaining the failure of recolorability

The reconfiguration problem asks whether one proper coloring can be changed into another by recoloring one vertex at a time, keeping every intermediate coloring proper. To represent the allowed moves, the graph $R_k(G)$ has one vertex for each proper coloring of G using colors from a set of size k, and an edge joins two colorings that differ at one vertex; a coloring may use fewer than k of the available colors.

In these terms a graph is *recolorable* if $R_k(G)$ is connected for every $k \geq \chi(G)+1$, where $\chi(G)$ is the chromatic number, and a *frozen* coloring, one in which no vertex can change color without creating an equally colored adjacent pair, is an isolated vertex of $R_k(G)$.

In *Recoloring via Modular Decomposition*, Belavadi et al. (2026) ask whether recolorability of every induced subgraph of a prime graph guarantees recolorability of all its clique blowups; this is the preservation direction of their Conjecture 1, and the statement matches Section 6 of the originally reviewed arXiv v2 input. Section 5 defines prime graphs and clique blowups; $C_n[K_2]$ replaces each cycle vertex by a two-vertex clique, with all edges between neighboring cliques.

The original counterexample uses C_9, which is prime and whose induced subgraphs are all recolorable. Its blowup $C_9[K_2]$ has chromatic number 5, but repeating the color pairs $\{1,2\}, \{3,4\}, \{5,6\}$ around the cycle gives a frozen six-coloring. Each vertex has neighbors using every alternative color, so no first recoloring move is possible. A proper five-coloring provides another vertex of the

same reconfiguration graph, proving that it is disconnected. Thus the blowup fails the conjecture's conclusion even though the base graph satisfies its hypotheses.

Relation to fall colorings and the source construction. A frozen proper coloring is a fall coloring: every vertex sees all colors in its closed neighborhood (Dunbar et al., 2000). For cycles, $3 \in \mathrm{Fall}(C_n)$ exactly when $3 \mid n$ (Lauri & Mitillos, 2019, Section 4). Applying Shaebani (2009, Theorem 2) with $G = C_n$, $H = K_2$, and three two-color palettes gives a fall six-coloring of $C_n[K_2]$ whenever $3 \mid n$. Shaebani also explicitly discusses $C_9[K_2]$ immediately after Corollary 3 and gives a fall five-coloring, so even this particular graph predates our recorded witness. The source's Figure 3 displays the same periodic palettes on a doubled six-cycle; its base graph contains a non-recolorable induced C_6, so that example does not contradict Conjecture 1 (Belavadi et al., 2026, Section 6). Our case uses this known construction at $n = 9$ and verifies all the hypotheses of the conjecture. In the follow-up, we used AI tools to prove the odd-cycle characterization and labeled count below in Lean. The cited constructions establish the existence direction; the follow-up supplies the necessity and counting arguments. We report these connections without asserting literature-wide priority for the characterization or count.

The following proposition identifies exactly which odd-cycle blowups have frozen six-colorings and counts them, showing that the original example is part of an infinite family with exponentially many isolated colorings.

Proposition 2 (Frozen colorings in odd-cycle blowups)**.** *For every odd $n \geq 5$, C_n is prime and every induced subgraph is recolorable. Its blowup $H_n = C_n[K_2]$ has chromatic number 5 and has a frozen proper coloring with the six labeled colors $[6]$ if and only if $3 \mid n$. When $3 \mid n$, there are exactly $90 \cdot 2^n$ frozen labeled colorings, and $R_6(H_n)$ has at least $90 \cdot 2^n + 1$ connected components.*

The single $C_9[K_2]$ witness suggests looking at its repeated color pairs. Call the pair assigned to each replacement clique its *palette*. The key is that the inability to recolor a vertex imposes a local constraint on three neighboring palettes. In a frozen coloring, every three consecutive two-vertex cliques must collectively use all six colors. Overlapping triples of cliques therefore force their palettes to repeat with period three. An ordered partition of the six colors into pairs gives 90 choices; independently assigning each clique's pair to its two labeled vertices gives 2^n choices. Thus $C_{6r+3}[K_2]$, for every $r \geq 1$, is a counterexample to the same preservation claim. The original $C_9[K_2]$ has 46,080 isolated six-colorings, not merely the single exhibited state.

The witness thus becomes a classification: this six-color obstruction occurs exactly at the odd multiples of three, and its multiplicity is known. It persists for arbitrarily large prime base graphs while the blowups keep chromatic number five. Clique substitution creates the obstruction even though every induced subgraph of the base graph is recolorable. This identifies what a repaired preservation theorem must address: additional conditions on the substitution itself. The component count is a lower bound. At the remaining odd lengths, determining connectivity requires studying colorings that do admit individual moves.

Larger replacement cliques. The two-color palettes lead naturally to a further question: what happens when each cycle vertex is replaced by a clique of size three? For every natural $n \geq 4$, the graph $C_n[K_3]$ has a frozen proper eight-coloring exactly when $8\lceil n/3 \rceil \leq 3n$. This holds precisely for $n \in \{6, 8, 9, 11, 12\}$ and all $n \geq 14$.

The necessity argument tracks the cliques containing each color. Those cliques form an independent dominating set of the base cycle: no two are adjacent, and every other clique has a neighbor in the set. For sufficiency, explicit sequences of palettes can be joined to cover every allowed length. Three consecutive K_3 cliques have nine vertex positions for eight colors, so a color can repeat in the two outer cliques. This explains why the three-period argument for K_2 changes. At $n = 13$, the theorem rules out this particular obstruction; the connectivity of the other colorings is a separate question.

Allowing the clique size to vary gives a broader construction. For every odd $n \geq 9$, taking $q = \lceil n/3 \rceil$ yields a graph $C_n[K_q]$ with chromatic number $2q + 1 < n$ and a frozen n-coloring. Thus a suitable clique size removes the divisibility restriction on the base cycle. We developed these later results with the help of AI tools. Their graph statements are certified in `mathematics/theorems/Q3Law.lean` and `mathematics/theorems/AllOdd.lean`; `mathematics/index.json` records their exact scopes and build receipts. Combining these graph statements with the source

conjecture's hypotheses is the written mathematical argument, rather than a single additional Lean theorem linking all parts.

I.3 Proofs of the structural counterexample propositions

The rest of this appendix proves the two propositions above. The Schur proof counts rainbow triples, derives their limiting density, and optimizes the threshold pair; the cycle proof derives palette periodicity and counts the frozen colorings.

Both propositions also have Lean proofs under `mathematics/theorems/`, with statement and build records in `mathematics/index.json`: the Schur certificate connects finite counts to the limiting density and proves the family's unique optimum and sharp stability bound, and the cycle certificate covers every allowed cycle length. Finite checks of the original witnesses are supplied separately under `mathematics/discoveries/`.

I.3.1 Schur template: counting, optimization, and stability

Proof of Proposition 1. Count the two kinds of rainbow triple. Fix $0 \le a \le b \le 1$ and the coloring specified in the proposition. Write $t = b-a$ for the width of the middle odd block and $x_+ = \max(x, 0)$ for the positive part of x. The two odd color classes have cardinalities $(1-t)n/2 + O(1)$ and $tn/2 + O(1)$. There are $\binom{n}{2}$ ordered triples $x + y = z$ in $[n]$.

For each pair of differently colored odd integers, their positive even difference yields exactly two mixed-parity rainbow triples, one for each ordering of the addends. Conversely each mixed-parity rainbow triple determines this pair. Their number is therefore $t(1-t)n^2/2 + O(n)$, contributing $t(1-t)$ to the limiting normalized density.

For odd–odd addends, the sum is even and has color 1. Choose the color-2 addend from $[0, a] \cup [b, 1]$ and the color-3 addend from $[a, b]$ in normalized coordinates, subject to their sum being at most 1. The lattice of odd pairs has density $n^2/4$ per unit area. The factor two for ordering the addends and division by $\binom{n}{2} \sim n^2/2$ cancel this factor $1/4$. Thus the limiting contribution is the area of those two rectangles below the line $x + y = 1$. Boundary lattice squares number $O(n)$, giving an $O(n)$ error in the unnormalized count; this also justifies arbitrary real thresholds and their rounding. Even–even addends cannot be rainbow.

For a rectangle $[u, v] \times [s, t]$, its area below $x + y = 1$ is

$$\frac{1}{2}\big[(1-u-s)_+^2 - (1-v-s)_+^2 - (1-u-t)_+^2 + (1-v-t)_+^2\big].$$

This follows by inclusion–exclusion from the triangle with lower corner (u, s). Applying the formula to the two rectangles gives

$$\begin{aligned} D(a,b) &= (b-a)(1-b+a) \\ &\quad + \frac{1}{2}\big[(1-a)^2 - (1-b)^2 - (1-2a)_+^2 - (1-2b)_+^2 + 2(1-a-b)_+^2\big]. \end{aligned} \tag{8}$$

In particular the limit exists.

Optimize the density over all threshold pairs. The allowed pairs form the triangle $0 \le a \le b \le 1$. The lines $a = 1/2$, $b = 1/2$, and $a + b = 1$ divide it into four regions. Expanding Equation (8) gives the following quadratics and Hessian determinants:

Region	$D(a, b)$	$\det \nabla^2 D$
$0 \le a \le b \le \frac{1}{2}$	$-\frac{3}{2}a^2 + 4ab - \frac{5}{2}b^2 - 2a + 2b$	-1
$a \le \frac{1}{2} \le b$, $a + b \le 1$	$-\frac{3}{2}a^2 + 4ab - \frac{1}{2}b^2 - 2a + \frac{1}{2}$	-13
$a \le \frac{1}{2} \le b$, $a + b \ge 1$	$-\frac{5}{2}a^2 + 2ab - \frac{3}{2}b^2 + 2b - \frac{1}{2}$	11
$\frac{1}{2} \le a \le b \le 1$	$-\frac{1}{2}a^2 + 2ab - \frac{3}{2}b^2 - 2a + 2b$	-1

The first, second, and fourth Hessians are indefinite. At an interior local maximum a twice differentiable function must have negative semidefinite Hessian, so these three regions attain their maxima on

their boundaries. In the third region the Hessian is $\left(\begin{smallmatrix} -5 & 2 \\ 2 & -3 \end{smallmatrix}\right)$, which is negative definite. Its gradient vanishes uniquely at $(a, b) = (4/11, 10/11)$, strictly inside the region, with value $9/22$.

The following formulas cover every remaining boundary:

$$
\begin{aligned}
D(a, a) &= 0, \\
D(0, b) &= \begin{cases} 2b - \frac{5}{2}b^2, & b \le \frac{1}{2}, \\ (1 - b^2)/2, & b \ge \frac{1}{2}, \end{cases} \\
D(a, 1) &= \begin{cases} 2a - \frac{5}{2}a^2, & a \le \frac{1}{2}, \\ (1 - a^2)/2, & a \ge \frac{1}{2}, \end{cases} \\
D(a, \tfrac{1}{2}) &= \tfrac{3}{8} - \tfrac{3}{2}a^2 && (0 \le a \le \tfrac{1}{2}), \\
D(\tfrac{1}{2}, b) &= \tfrac{3}{8} - \tfrac{3}{2}(b - 1)^2 && (\tfrac{1}{2} \le b \le 1), \\
D(a, 1 - a) &= 3a - 6a^2 && (0 \le a \le \tfrac{1}{2}).
\end{aligned}
$$

The second and third lines have maximum $2/5$, attained at $b = 2/5$ and $a = 2/5$, respectively. Each of the final three has maximum $3/8$. Thus every boundary value is at most $2/5 < 9/22$, proving the unique global maximum over this template. The boundaries $a = 0$ and $b = 1$ are precisely the two contiguous odd-block subfamilies, proving their claimed optima.

Measure the loss from moving the thresholds. Completing the square in the third region, with $u = a - 4/11$ and $v = b - 10/11$, yields Equation (7). The loss matrix is $\left(\begin{smallmatrix} 5/2 & -1 \\ -1 & 3/2 \end{smallmatrix}\right)$ with eigenvalues $(4 \pm \sqrt{5})/2$, so in that region

$$
\frac{9}{22} - D(a, b) \ge \frac{4 - \sqrt{5}}{2} \left((a - \tfrac{4}{11})^2 + (b - \tfrac{10}{11})^2 \right).
$$

This inequality quantifies stability within the template: the loss in density is at least a fixed positive multiple of the squared distance from the optimal threshold pair. The wider problem allows arbitrary three-colorings and requires a separate upper bound. □

The public file `mathematics/theorems/SchurTemplateComposed.lean` contains the counting, limit, and optimization arguments. It checks Equation (8), including every boundary, using nonnegative polynomial decompositions in the four regions. For the sharp coefficient, writing $s = (1 + \sqrt{5})/2$, $\lambda = (4 - \sqrt{5})/2$, and $Q(u, v) = 5u^2/2 - 2uv + 3v^2/2$, it verifies $s[Q(u, v) - \lambda(u^2 + v^2)] = (su - v)^2$. Equality therefore holds exactly on $v = su$. Any nonzero vector on that line proves that the coefficient cannot be increased for this quadratic form.

The corresponding declarations are `CoPEER.SchurTemplate.unique_global_optimum`, `sharp_stability`, `quadratic_spectral_equality_iff`, and `quadratic_spectral_coefficient_optimal`. The declaration `paper_limit` proves the counting and limit correspondence preceding Equation (8); the composed theorem joins it to the optimization result. `mathematics/builds/schur.json` records the compiled source and log hashes.

I.3.2 Odd-cycle blowups: palette periodicity and component counts

Proof of Proposition 2. Verify the conjecture's hypotheses for the base cycle. A graph is d-degenerate if every nonempty induced subgraph has a vertex of degree at most d. The standard degeneracy recoloring argument (Cereceda et al., 2008) shows that if a finite graph X is d-degenerate, then $R_k(X)$ is connected for every $k \ge d + 2$. Here vertices of $R_k(X)$ are proper maps $V(X) \to [k]$, without requiring surjectivity, and an edge changes one vertex's color. To prove the lemma, induct on $|V(X)|$. Remove a vertex v of degree at most d, and connect the restrictions of any two colorings by induction. Lift that path to X. If a neighbor is about to acquire the current color of v, first move v to a different color absent from its neighborhood; at least two colors are absent because $k \ge d + 2$. Then perform the neighbor's move. Finally give v its target color. This proves connectivity for all required k.

For $n \ge 5$, C_n is prime. To see this, a *module* is a set of vertices that all have the same neighbors outside the set. A proper module S with at least two vertices would, by connectivity, have an outside

vertex v adjacent to a member of S. Modularity makes v adjacent to every member of S, so degree two forces $S = N(v)$. The other neighbor of either member of S lies outside S and distinguishes its two members, a contradiction. For odd n, C_n has chromatic number three and degeneracy two, so the lemma proves recolorability for every $k \geq 4$. Each proper induced subgraph is a disjoint union of paths. With an edge it has chromatic number two and degeneracy one, so the lemma applies for every $k \geq 3$. Nonempty edgeless graphs recolor one vertex at a time for every $k \geq 2$; the empty graph has one coloring. Thus all induced subgraphs satisfy the full definition of recolorability.

Determine the chromatic number of the blowup. Call each replacement clique a *bag*, and index the bags of $H_n = C_n[K_2]$ cyclically by $0, \ldots, n-1$. An independent set takes at most one vertex per bag, and its occupied bags form an independent set of C_n. Conversely every such set of bags lifts to an independent set. Therefore $\alpha(H_n) = (n-1)/2$, and for odd $n \geq 5$,

$$\chi(H_n) \geq \left\lceil \frac{2n}{(n-1)/2} \right\rceil = \left\lceil \frac{4n}{n-1} \right\rceil = 5.$$

For the matching upper bound, give the first $n-3$ bags alternating palettes $\{1,2\}, \{3,4\}$. The prefix length is even, so its last palette is $\{3,4\}$. Give the last three bags, in order, $\{1,5\}, \{2,3\}, \{4,5\}$. Adjacent palettes, including the last and first, are disjoint. Assign each palette's two colors once in its bag. This is a proper five-coloring for every specified n.

Identify exactly when a coloring is frozen. In any proper six-coloring, write P_i for the two-element palette on bag i. Every vertex of H_n has degree five, and its closed neighborhood consists exactly of the six vertices in bags $i-1, i, i+1$. It cannot change color precisely when its five neighbors use all five alternative colors. The coloring is therefore frozen if and only if, for every i, the palettes P_{i-1}, P_i, P_{i+1} are pairwise disjoint and partition $[6]$. Comparing this partition with that for $i+1$ forces $P_{i-1} = P_{i+2}$: both are the complement of $P_i \cup P_{i+1}$. Hence the palettes have period three. If $3 \nmid n$, then $\gcd(3, n) = 1$, so this periodicity around the cycle makes all palettes equal, contradicting adjacent disjointness. If $3 \mid n$, any ordered partition of $[6]$ into three pairs, repeated around the cycle, supplies a frozen coloring.

Count the isolated colorings. There are $6!/(2!)^3 = 90$ ordered partitions for bags $0, 1, 2$. Each determines all other palettes, and in each labeled bag we can assign the two colors to its two labeled vertices in two ways, independently. The exact frozen-coloring count is thus $90 \cdot 2^n$. Each is an isolated vertex of $R_6(H_n)$. The explicit five-coloring, regarded as a six-coloring, has unused color 6, so any vertex can move to that color. It lies in a further component, giving at least $90 \cdot 2^n + 1$ components. For odd n divisible by three, write $n = 6r + 3$ with $r \geq 1$. These graphs refute the same prime-blowup preservation claim at $6 = \chi(H_n) + 1$. □

The exact count distinguishes vertex labels and color labels, just as R_k does. It gives a lower bound on the number of connected components: each frozen coloring contributes one, and the five-coloring contributes another. For $3 \nmid n$, deciding connectivity requires examining the components formed by colorings that can make single-vertex moves.

J FURTHER CERTIFIED FAMILIES

Beyond the three case studies of Section 5, further witnesses from the research collection were extended to parameter families and formalized. Table 22 summarizes them, and the paragraphs that follow give each result's exact scope, its certificate, and where its witness came from. The prime-blowup and rainbow-Schur families are stated and proved in Appendix I. We developed these extensions with the help of AI tools. Separate adversarial model reviews checked their formal statements and their correspondence to the sources. They are mathematical results, not measurements of model discovery. None of the files uses `sorry` or `native_decide`. `mathematics/index.json`, `mathematics/extensions/index.json`, and `mathematics/supporting/index.json` link the statements, sources, hashes, and build records, and `mathematics/discoveries/` records the source versions and discovery provenance. Mathematical priority is left unassigned throughout.

Projective cubes. In the cyclic group $\mathbb{Z}_N$, the projective cube of a size-d multiset $S = \{a_1, \ldots, a_d\}$ is $\Sigma^* S = \{\sum_{i \in I} a_i : \emptyset \neq I \subseteq [d]\}$, the set of all sums over nonempty index sets. Repeated

Case	Refuted statement	What the selected Lean proof establishes	Toolchain
Projective cubes	Conjecture 4 of Meng (2026)	Cube-free sets above the conjectured density for $d = 5, 7, 8$ in infinitely many cyclic groups; the exact density boundary of the layered construction	core Lean 4.30.0 and 4.34.0; boundary results Lean 4.30.0 with Mathlib
Supertoken graphs	$\dim F_2(K_n) = n-1$ (Baskoro et al., 2025)	Resolving sets of size at most $n-2$ for every $n \geq 8$	core Lean 4.30.0 and 4.34.0
Binomial polynomials	second assertion of Sun (2026)	No coefficientwise q-log-convexity for every integer $m \geq 4$	core Lean 4.30.0 and 4.34.0
Fake-Gaussian positivity	Conjecture 4 of Gatzweiler & Krattenthaler (2025)	An exact integer polynomial with a negative coefficient	Lean 4.30.0 and 4.34.0
q-rious unimodality	unimodality assertion of Warnaar & Zudilin (2025)	A non-unimodal coefficient sequence; the Landau hypothesis certified	Lean 4.30.0 and 4.34.0
Permutation-grid rooks	necessity direction of Conjecture 3.10 of Lewis & Won (2026)	A placement count congruent to 3 modulo 4	Lean 4.30.0 and 4.34.0
Toda numerators	Conjecture 7.3 of Labelle (2026)	A numerator that is not unimodal under any common-exponent regrading	Lean 4.30.0 and 4.34.0

Table 22: Further certified families and refutations. Each row's scope, certificate file, provenance, and the human judgments on its dossier are given in the text; the prime-blowup and rainbow-Schur certificates are in Appendix I.

and zero generators are allowed, and the source prints $\subset$ in its definition but its three-generator example includes $x + y + z$, so we include the full index set. A set is cube-free if it contains no such cube. Conjecture 4 of Meng (2026) states that when $d \mid N$, every projective-d-cube-free set $A \subseteq \mathbb{Z}_N$ satisfies $|A| \leq (d-1)N/d$. For each row of Table 23 and every integer $k \geq 1$, let $A_N = \{x \in \mathbb{Z}_N : x \bmod M \notin F\}$ with F the excluded residues. Reducing a hypothetical cube in A_N modulo M would give a cube that avoids F, contradicting the verified obstruction in that smaller group. The corresponding Lean file proves $d \mid N$, that A_N contains no projective d-cube, the displayed cardinality, and $d|A_N| > (d-1)N$. Each row therefore refutes Conjecture 4 for infinitely many group orders.

Table 23: Three families of counterexamples verified by Lean. Every row holds for every integer $k \geq 1$. The last column compares the density of the constructed set with the conjectured upper bound.

d	M	Excluded residues F	N	$\lvert A_N\rvert$	Density
5	16	$\{0, 4, 12\}$	$80k$	$65k$	$13/16 > 4/5$
7	24	$\{0, 6, 18\}$	$168k$	$147k$	$7/8 > 6/7$
8	36	$\{0, 6, 12, 30\}$	$72k$	$64k$	$8/9 > 7/8$

The finite obstruction is checked in four steps. A Python generator enumerates the sets of subset sums reachable while all nonempty sums avoid F modulo M; these sets describe the possible states after each new generator is added. Lean checks the generated data: appending any residue must either produce a forbidden nonempty subset sum or lead to a listed state at the next depth, and no state survives to depth d. A shared soundness proof turns these checks into a theorem about every length-d sequence. Reduction modulo M then gives cube-freeness for every group order divisible by M, counting the repeated residue classes gives the cardinality, and $N = \operatorname{lcm}(M, d)k$ supplies the hypothesis $d \mid N$. The three self-contained files compiled under Lean 4.30.0 and 4.34.0 with Lean's bundled libraries and no Mathlib; the batch checks use kernel evaluation and no axioms, the assembled closure theorem uses `propext`, and the family theorems use `propext`, `Classical.choice`, and `Quot.sound`. The certificates are in `mathematics/theorems/`, with build evidence, generator and template hashes in `mathematics/builds/cube_families.json`. A separate model reader examined the argument adversarially, regenerated all three files byte for byte, rejected

thirteen deliberately incorrect variants, and reran the $d = 5$ and $d = 7$ files under both Lean versions; for $d = 8$ it inspected the recorded compilation logs and recomputed the mathematics independently.

The $d = 5$ set was found in the model runs of 10 September 2026, first with BugBank and then with plain prompting. It is the pullback under reduction modulo 16 of the published layered set C_5 of Long & Wagner (2019), so the search rediscovered that construction. We found the $d = 7$ and $d = 8$ sets by computer search, with AI tools, on 19 September and then formalized them; their construction priority is not established. Human reviews of the dimension-5, 7 and 8 dossiers (R02, R13, R14) are reported in Appendix C. The same source's auxiliary Conjecture 10 had been refuted publicly on 26 July 2026 by hypnopump (2026) with a different witness and a Lean proof; that witness contains a projective cube, so it concerns Conjecture 10, not Conjecture 4.

The three families raise the question in which dimensions the layered construction itself exceeds the conjectured density. Write $C_d \subseteq \mathbb{Z}_{2^n}$ for the recursively defined layered construction of Long & Wagner (2019); each layer has an index, and the construction fits in the group when all those indices are at most n, which $d \le n$ guarantees. Let $g(d)$ be the fraction of the group outside the construction, so that $g(1) = 1$ and $g(P + m - 1) = (1 + g(m))/(2P)$ for $P = 2^\ell$, $\ell \ge 1$, and $1 \le m \le P$. The Lean proofs establish, for every $d \ge 2$ and every n for which the layers fit, that $|C_d|/2^n = 1 - g(d)$ and that

$$\frac{|C_d|}{2^n} > \frac{d-1}{d} \quad \Longleftrightarrow \quad d \notin \{2^\ell - 2,\ 2^\ell - 1,\ 2^\ell : \ell \ge 1\};$$

equality holds exactly when d is a power of two, and at the other exceptional dimensions, one or two below a power of two, the density is strictly below the bound. The theorem `LongWagner.C_five_mod_sixteen` identifies C_5 in $\mathbb{Z}_{16}$ with the certified quotient excluding $\{0, 4, 12\}$; at $d = 7, 8$ the layered densities are $27/32 < 6/7$ and $7/8$, which is why the other constructions were needed there. Cube-freeness of the general C_d is supplied by Long & Wagner (2019, Section 3.1 and Claim 3.1) and enters the conditional Lean theorems as an explicit premise, while the certificates for $d = 5, 7, 8$ verify cube-freeness directly; a general pullback lemma proves that taking a pullback preserves cube-freeness and density. These results are in `mathematics/extensions/layered_boundary/`: `LayeredBoundaryAllD.lean` contains `density_gt_iff`, `density_eq_iff`, `card_C_iff_fits`, `card_C_gt_iff`, and `card_C_eq_iff`, and `PullbackLemma.lean` the pullback lemmas, under Lean 4.30.0 with Mathlib. The original all-dimensions file has a successful execution receipt; intermediate bounds were then appended, leaving the headline classification unchanged, and a replay of the unedited amended bytes on 21 September 2026 exited with status zero and reproduced the logs byte for byte (`LayeredBoundaryAllD_RECEIPT.json`). The scoped supplement L01 reviews this all-dimensional extension with the Long–Wagner cube-freeness premise explicit (Appendix C).

Supertoken graphs. A vertex of the supertoken graph $F_k(K_n)$ is a placement of k indistinguishable tokens among n types, allowing several tokens of the same type (Hammack & Smith, 2017, Definition 2.1), represented by a list of n natural numbers summing to k. Adjacent vertices differ by moving one token to another coordinate. The proof in `mathematics/extensions/06_supertoken_metric/SupertokenMetricGeneral.lean` starts from this adjacency and derives the shortest-walk distance as half the ℓ_1 distance, the formula stated by Baskoro et al. (2025, Proposition 3.1(i), Eq. (2)). A resolving set is a set of landmarks whose distance vectors distinguish all vertices (Harary & Melter, 1976; Slater, 1975). The finite witness supplies six landmarks resolving $F_2(K_8)$; padding each landmark with zero coordinates and adding $2e_i$ for each new coordinate gives at most $n - 2$ landmarks for every $n \ge 8$. The theorem `conjecture_4_2_fails_k2` thus refutes the claimed dimension $n - 1$ throughout that range, including the claim that it holds for all sufficiently large n at $k = 2$. The result is an upper bound at fixed $k = 2$; the exact dimensions, and the behavior as k grows with n fixed, are separate questions, and the proof is independent of the separate expensive lower-bound computation for $(8, 2)$. We later extended the family and formalized it with AI tools.

Binomial polynomials. Coefficientwise q-log-convexity requires every coefficient of $V_{n-1,m}V_{n+1,m} - V_{n,m}^2$ to be nonnegative (Liu & Wang, 2007), where $V_{n,m}(q) = \sum_{j=0}^{n} \binom{n+j}{j}^m q^j$. The recorded witness has $m = 11$ and $n = 3$. The coefficient responsible for that failure yields a uniform obstruction: for every $m \ge 11$, the coefficient of q^2 at $n = 3$ is $2 \cdot 15^m + 6^m - 2 \cdot 10^m - 16^m < 0$, and a kernel-checked calculation at $n = 23$ covers the

remaining exponents $4 \le m \le 10$. Together these give `not_qlogconvex_of_four_le` in `mathematics/extensions/09_sun_qlogconvex/SunQLogConvex.lean`, which refutes the second assertion in the cited arXiv v4 of Sun (2026) for every integer $m \ge 4$. The other three assertions, the cases $m = 2, 3$, and pointwise nonnegativity for real $q > 0$ are distinct questions.

Fake-Gaussian positivity. For the fixed-m positivity assertion of Gatzweiler & Krattenthaler (2025), let

$$F(q) = \prod_{i=1}^{21} \left(\frac{1 - q^{22+i}}{1 - q^i} \right)^{a_i}, \qquad a_6 = a_{21} = 1, \quad a_{14} = a_{18} = a_{20} = 2,$$

with all other $a_i = 0$. The certificate `mathematics/extensions/07_fake_gaussian/FakeGaussian.lean` checks the full numerator–denominator identity and proves that F is the exact integer polynomial represented by its 177 coefficients, with $[q^{43}]F = -1$. The declaration `not_conjecture4` refutes the encoded general statement, and a general quotient-uniqueness lemma ensures that every polynomial representative of the quotient has this same negative coefficient. The result concerns Conjecture 4; the symmetric positivity assertion, the source's other conjectures, and eventual positivity as m grows have separate mathematical scope. A model run produced the witness, and we formalized it with AI tools; Appendix C reports the human review of the dossier (R07) and the scoped supplement G07.

q-rious unimodality. For the factorial-ratio unimodality assertion of Warnaar & Zudilin (2025), the recorded tuples are $a = (2, 3, 5, 12)$ and $b = (1, 1, 1, 4, 6, 9)$. Writing $[j]_q = 1 + q + \cdots + q^{j-1}$, their quotient satisfies $D(a, b; q) = [10]_q[11]_q(1 - q^2 + q^4)$, and the full coefficient sequence of $(1 + q)D$ is not unimodal. The certificate `mathematics/extensions/04_qrious_unimodality/QriousUnimodality.lean` proves the quotient identity and its uniqueness and derives the general negations `not_conjecture5` and `not_conjecture5Sym`. The Landau hypothesis (Warnaar & Zudilin, 2025, Eq. (1)) is certified for every general floor profile, including every rational input, and for every structure satisfying the listed floor properties; the written argument identifies the real numbers with such a structure, while the retained Lean proof covers the abstract structure and rational inputs. The refutation concerns unimodality and leaves the separate positivity conjecture as a distinct question. The witness came from a model run; the human review of the dossier (R04) is reported in Appendix C.

Permutation-grid rooks. For the permutation $w = (2, 7, 4, 8, 1, 6, 3, 5)$, the Lean kernel computes 155 complete non-attacking rook placements on its permutation crossword grid; since $155 \equiv 3 \pmod 4$, this count violates the proposed condition. Using general definitions of permutation grids, words, and complete placements, the theorem `not_conjecture310` in `mathematics/supporting/sources/RookSpectrum.lean` refutes the necessity direction of Conjecture 3.10 of Lewis & Won (2026). Two size ranges matter here: the board has size eight, within the board sizes the source enumerated, whereas the count 155 exceeds the $r \le 108$ count range summarized immediately before the conjecture, and the restatement given to the model omitted that source sentence. The certificate establishes this witness and this direction of the conjecture; the converse direction, a corrected spectrum of possible counts, and the smallest possible witness are further questions. The human review of the dossier (R05) is reported in Appendix C.

Toda numerators. A coefficient sequence is unimodal if it never decreases before a peak and never increases after it. The recorded Toda witness of 10 September has numerator $1 + q^2$: its coefficients are not unimodal when consecutive powers of q are counted, but become unimodal when the polynomial is written in q^2, and under that substitution it is the source's type-A_2 example. This suggested a stronger target, a failure that persists when the polynomial is written in any fixed integer power of q, a change called common-exponent regrading. While formalizing the result on 19 September with AI tools, we found a type-B_2 witness at $\alpha = 2\alpha_1 + 2\alpha_2$ with numerator $1+q+3q^2+2q^3+5q^4+2q^5+3q^6+q^7+q^8$. Its nonzero coefficient of q forces the regrading step to be one, and $3 > 2 < 5$ gives an interior dip. The theorem `Exact.not_conjecture73_exact` in `mathematics/supporting/sources/TodaNumerator_Exact.lean` formalizes the refutation for general root-system data using the exact defining recurrence after denominators are cleared, checking full polynomial identities. The connection from that recurrence to the rational

functions in Definition 1.1 of Labelle (2026) uses algebra in $\mathbb{Q}(q)$ and the source's uniqueness result in the written argument. The original model witness and the later, stronger witness retain separate discovery records; the human review of the dossier (T11) is reported in Appendix C.

Checking definitions beyond the witness. A witness can satisfy two definitions that disagree elsewhere. For the C_2^4 short-zero-sum record, replacing the last invariant factor by the first in the definition of the exponent, reversing divisibility, or using modulus 2 throughout still lets the refutation compile, because all of these agree at $[2, 2, 2, 2]$. A companion file gives a separately written, Lean-equivalent formulation and checks true instances at $C_2 \oplus C_4$, which the altered definitions fail. These mutation probes (Jia & Harman, 2011) test the definitions beyond the refuting instance; correspondence to the source prose is reviewed separately. The base and companion proofs, with their successful compilation logs, are listed in `mathematics/supporting/index.json` as `ShortZeroSums_Conjecture.lean` and `ShortZeroSums_Conjecture_Sanity.lean`; the individual mutant logs were not retained.

What was contributed at each stage. The research collection contains eleven model-produced refutation cases and one finding already visible within its source. Subsequent checking identified known constructions, including the cube and Schur examples, an index-ratio conjecture that its author had revised before our run, and a semigroup contradiction present in the source's own table. We keep these distinctions so that a reader can see both the mathematical conclusion and what was contributed at each stage: identifying a witness, checking it against the source, extending its range, or formalizing the result. The cases are reported as source-linked mathematical criticism and formal completion, with discovery attribution stated individually.

K Corpus construction, audit procedures, and executable checks

Workflow and recorded search limits. This appendix gives the procedures behind the six-stage workflow of Section 3: the search limits, the prompts' requirements, the independent checks, and the model review. The original auditing protocol favors short objects: at most 12 graph vertices, 10 objects, or 10 significant trace steps. For each research conjecture we separately specify the types of examples to examine and the ranges of parameter values to test. A separately written checking program tests each proposed witness with exact arithmetic. Human contributors and coding agents translate surviving failures into Lean and inspect the selected theorem and its proof dependencies. A separate model-assisted review judges whether the objection applies to the selected source claim and records what it shows about the surrounding argument. The stages follow the principle of producing a result and checking its certificate separately (McConnell et al., 2011; de Moura & Ullrich, 2021; The mathlib Community, 2020).

A successful exhaustive check establishes the claim only for the example types and sizes it covers, and separately written checking programs and separate model calls can share mistaken assumptions. Appendix C documents the human reviews and the human-approved core that supplies the 100 primary benchmark inputs.

K.1 Generating and checking counterexamples

Let P be an informal proof, q a claim used in P, and w a proposed counterexample. Checking the objection requires three things: w must violate the stated mathematical condition, that condition must represent q under the source's assumptions, and the checking program must perform the intended calculation. The workflow checks the three separately and saves the evidence from each stage.

What each check establishes. A separately written program tests the proposed witness, and a Lean proof certifies the formal counterexample (de Moura & Ullrich, 2021; The mathlib Community, 2020); file hashes, source-passage matching, and axiom checks link that proof to its dossier. Human review, where performed, checks whether the objection represents the source claim and satisfies its assumptions. An executable Python check also depends on the Python interpreter. The division follows certifying algorithms and proof-carrying code: a producer supplies evidence that a smaller checker can verify (McConnell et al., 2011; Necula, 1997).

Selecting claims to investigate. Two separate calls to the same fixed model version receive the problem and a line-numbered proof, and each identifies a claim that may admit a short counterexample, within the size limits above and with a target Lean proof of roughly 50 lines. The response must quote the source exactly, describe a concrete candidate, and supply the required metadata. A deterministic validator checks the quotation and the response format, and a second model then reviews the claim, its assumptions, the relevance of the quotation, and the proposed size. These calls only choose candidates: the programs and Lean proofs that follow test the witnesses, and separate reviews check their connection to the source.

Searching, checking, and proving. A search agent receives the record identifier, the quoted claim, and the surrounding text, but not earlier witnesses or Lean files, so that it attempts a fresh construction. It writes and runs a deterministic program over integers and exact rationals, and records one of four outcomes: a witness, an exhaustive search with no witness in a specified finite set, an inconclusive search, or a claim outside the permitted search setting. A separately written checker reconstructs the mathematical condition from the claim and tests the proposed witness; both programs are replayed with two hash seeds. Human contributors and coding agents translated suitable witnesses into Lean 4 and built them with a fixed toolchain. The audit enumerates each theorem's axiom dependencies with `Lean.collectAxioms`, distinguishes computation trusted to the compiler from kernel-checked proofs, and detects `sorryAx`; every selected declaration in the current release has a kernel-checked proof. Separate stage inputs and fresh processes keep each stage's evidence inspectable.

Connecting files to claims. The record stores SHA-256 hashes of the problem, solution, quoted claim, certificate, and original source passage; when matching uses normalized text, an index map identifies the original bytes. A scanner checks commands, declarations, and imports against the allowed configuration. The reporting code is checked separately from the Lean kernel (Wiedijk, 2012), with an attack suite that exercises wrong targets, added axioms, `sorry`, altered source passages, imports, duplicate rows, aliases, and aggregate counts; the saved results distinguish completed tests from tests not run.

K.2 Reviewing the recorded objection

The model review assigns three complementary checks to fresh agents. A blind input contains the problem and the quoted claim; a full input adds the witness, explanation, and certificate. Review documents are assembled from these files, with hashes recomputed from disk and each quotation located in the numbered source text.

Three mathematical checks. The first agent uses only the blind input to seek a new counterexample with exact arithmetic, recording its finite search set and whether the search is exhaustive. The second reads the full input, writes a checker without seeing the first agent's work, and substitutes the recorded witness values. It reports whether the witness works and identifies any information needed to reproduce it. The third compares the Lean certificate with the quoted claim and witness: do the definitions encode the source's objects and conditions, does the construction use the recorded values, and does the theorem refute the intended assertion? Any partial match must name the modeling choice or discrepancy.

Resolving disagreements. A fourth model agent examines flagged records, the three verdicts, and the generated programs. It checks the disputed mathematics with its own exact-arithmetic program and reads the quotation in context. Its conclusion distinguishes an error in the record or checker from a misplaced quotation or a documented modeling choice, and it recommends keeping, repairing, removing, or referring the record for human review. An accepted adjudication includes an executable check and a mathematical justification. The saved decisions identify unresolved disputes and required repairs. Records pass after all checks succeed or an adjudication resolves every flag without requiring a change. A second model campaign repeated these checks on the admitted inputs with the same prompts and fresh processes; its verdicts were saved separately, and the intersecting passing sets supplied the initial human-review handout.

Human review and reproduction. Human-review packets include the source context, the selected theorem, the complete Lean file, and its project-local dependencies; Appendix C presents the review

process, and Section C.2 the initial review conditions and version evidence. In the repository, `bugbank/reviews/README.md` indexes the completed human judgments and `bugbank/core/README.md` explains current admission; the saved model-audit decisions and construction lineage are available through `bugbank/annotations/README.md` and `bugbank/annotations/initial401/results/audit_analysis.json`.

K.3 Executable checks of the proposed witnesses

Across 604 executable tasks, the search produced 351 witnesses, 12 exhaustive searches without a witness in the specified finite set, 221 inconclusive searches, and 20 claims outside the permitted search setting. Separately written checkers confirmed 348 of the 351 proposed witnesses and rejected three because they did not violate the reconstructed condition, and all 955 search and checker programs passed exact-arithmetic inspection and deterministic replay with two hash seeds. A separate stage checked 16 selected algorithmic failures and confirmed all 16. These were chosen examples, so the result describes their checks rather than the fraction of all failures that the search would find. The repository supplies the outcomes and a count-replay command in `studies/executable_checks/README.md`, with program bindings and the 16 selected records in `studies/executable_checks/bindings/README.md`.

L Related resources

Section 6 discusses the related work. Table 24 compares the resources closest to ours on two points: what they ask a system to do, and what evidence stands behind their reference answers. For Co-PEER's localization benchmark the reference evidence is the dossier itself, a concrete witness with a selected Lean proof, together with the recorded human review that defines the human-approved core from which the benchmark is drawn; the score evaluates the selected line.

Table 24: What selected mathematical criticism resources evaluate and what supports their references. Counts refer to different units. BugBank connects source assertions to concrete witnesses and Lean certificates, with version-linked human review for the 147-dossier human-approved core; the localization score evaluates the selected line.

Resource and scale	Evaluation target	Reference evidence and human review
Co-PEER / BugBank human-approved core: 147 dossiers; 100 primary texts	Accepted error line; Hit@1 and Hit@3	Concrete witnesses and selected Lean proofs; all core dossiers have all-YES human judgments on the three dossier criteria, with recorded version continuity. Exact line-label review is recorded separately.
FaithSieve (Wang et al., 2026) 550 problems	First erroneous step or no error	Expert reference labels. The evaluated system generates local Lean evidence and checks semantic alignment.
ArxivMathGrading-Bench (Barkallah et al., 2026) 40 errors in 35 papers	Locate an author-corrected error	Author correction history; LLM location matching.
CounterMath (Li et al., 2025) 1,216 statement–rationale pairs	Truth judgments and reasoning with examples	Textbook rationales and author checks; model evaluation includes a 100-output human check.
REFUTE (Sinha et al., 2025) 324 problem/program pairs	Construct an admissible input on which a program fails	Reference implementations and input-constraint validators check generated counterexamples.
Learning to Disprove (Li et al., 2026) Sets of 1,058, 3K and 3K	Generate counterexamples to formal statements	Lean verifies generated counterexample proofs.
ProcessBench (Zheng et al., 2025) 3,400 problem–solution cases	First erroneous step or no error	Expert labels with three agreeing judgments.
PRMBench (Song et al., 2025) 6,216 instances	Step-level simplicity, soundness and sensitivity	Constructed labels; human review of 10% of instances.
Hard2Verify (Pandit et al., 2026) 200 responses; 1,860 steps	Step correctness and first-error identification	Expert step annotations and three review rounds.